МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

УТВЕРЖДАЮ

Заведующий кафедрой 806 _____ Крылов С.С.
(№ каф.) (подпись) (инициалы, фамилия)

_____ _____ 20 г.

ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

по теме:
«Практические аспекты обработки сигналов матрицами Адамара»

Научный руководитель _____ _____

Исполнитель _____ Фурлетова Д.А.

Москва 2026

РЕФЕРАТ

Научно-исследовательская работа состоит из 180 страниц, 27 рисунков, 15 таблиц, 76 использованных источников, 3 приложений.

МАТРИЦА АДАМАРА, МАТРИЦА УОЛША, ПРЕОБРАЗОВАНИЕ УОЛША-АДАМАРА, БЫСТРОЕ ПРЕОБРАЗОВАНИЕ ФУРЬЕ, ДИСКРЕТНОЕ КОСИНУСНОЕ ПРЕОБРАЗОВАНИЕ, ВЕЙВЛЕТ-ПРЕОБРАЗОВАНИЕ, СЖАТИЕ АУДИОДАННЫХ, МЕТРИКИ КАЧЕСТВА, SNR, SI-SDR, LSD, STOI, PESQ, MP3, WAV, PYTHON, PYSIDE6

Объектом исследования являются возможности практического применения матриц Адамара при обработке аудиоданных.

Цель работы заключается в комплексном исследовании применения матриц Адамара для сжатия аудиоданных, а также в оценке влияния данного подхода на ключевые показатели качества сжатия.

Для достижения поставленной цели решены следующие задачи: анализ существующих методов обработки и сжатия аудиосигналов; разработка программного комплекса для сравнительного анализа методов обработки аудиоданных; проведение экспериментальных исследований; формулировка рекомендаций по практическому применению.

Основными результатами работы являются: программный комплекс для сравнительного анализа методов обработки аудиоданных с реализацией семи преобразований и тринадцати метрик качества; закономерности эффективности методов в зависимости от жанровой принадлежности аудиоконтента; рекомендации по выбору метода обработки.

Данные результаты разработки предназначены для выявления принципов, которым подчиняются частотные области, создаваемые применением матриц Адамара при обработке аудиоданных.

Использование результатов позволит сократить вычислительные затраты при обработке аудиоданных и повысить эффективность их обработки.

СОДЕРЖАНИЕ

РЕФЕРАТ.....2
СОДЕРЖАНИЕ.....4
ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....8
Основные определения.....8
Список сокращений.....8
ВВЕДЕНИЕ.....10
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ОБРАБОТКИ АУДИОДАННЫХ. 12
1.1. Основы цифровой обработки звука.....12
1.1.1. Представление звука в цифровой форме.....12
1.1.2. Частотные характеристики аудиосигналов.....14
1.1.3. Психоакустика.....15
1.2. Формат WAV и стандартное преобразование в MP3.....16
1.2.2. Формат MP3.....17
1.2.3. Стандартное преобразование WAV → MP3.....18
1.3. Методы преобразования сигналов для сжатия.....20
1.3.1. Быстрое преобразование Фурье (FFT).....20
1.3.2. Быстрое преобразование Уолша-Адамара (FWHT).....20
1.3.3. Дискретное косинусное преобразование (DCT).....21
1.3.4. Дискретное вейвлет-преобразование (DWT).....22
1.3.5. Компандирование (μ-law).....23
1.3.6. Нелинейное сглаживание (Rosenbrock-like).....24
1.4. Интеграция методов преобразования в процесс WAV → MP3.....24
1.5. Метрики качества аудиосигналов.....25
1.5.1. Метрики временной области.....25
1.5.2. Метрики спектральной области.....25
1.5.3. Психоакустические метрики.....25
1.5.4. Характеристики сигнала.....26
1.6. Выводы по главе 1.....26
ГЛАВА 2. РАЗРАБОТКА ПРОГРАММНОГО КОМПЛЕКСА.....28
2.1. Требования к программному обеспечению.....28
2.1.1. Функциональные требования.....28
2.1.2. Нефункциональные требования.....29

2.2. Архитектура программного комплекса....30
2.2.1. Модульная структура....31
2.3. Реализация методов обработки....32
2.3.1. Базовый класс трансформации....33
2.3.2. Реализация FFT....33
2.3.3. Реализация FWHT....34
2.3.4. Реализация DCT....35
2.3.5. Реализация DWT (вейвлет Хаара)....35
2.3.6. Реализация μ-law....36
2.3.7. Реализация Rosenbrock-like....36
2.4. Реализация метрик качества....36
2.5. Тестирование....37
2.6. Выводы по главе 2....38
ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ....39
3.1. Методология эксперимента....39
3.2. Результаты сравнительного анализа....40
3.2.1. Сравнение по метрикам временной области....40
3.2.2. Сравнение по метрикам спектральной области....41
3.2.3. Сравнение по психоакустическим метрикам....42
3.2.4. Сравнение производительности....42
3.3. Анализ влияния жанровой принадлежности....43
3.3.1. Жанры с наилучшими результатами....43
3.3.2. Жанры с худшими результатами....44
3.4. Выводы по главе 3....45
ГЛАВА 4. ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ И РЕКОМЕНДАЦИИ....47
4.1. Области применения результатов....47
4.1.1. Музыкальные стриминговые сервисы....47
4.1.2. Системы онлайн-трансляций....48
4.1.3. Приложения для голосовых коммуникаций....48
4.2. Рекомендации по выбору метода обработки....49
4.3. Направления дальнейшего развития....49
4.3.1. Оптимизация производительности....49
4.3.2. Интеграция с ML-методами....50
4.3.3. Аппаратная реализация....50

4.4. Оценка экономической эффективности....50
4.5. Выводы по главе 4....51
ЗАКЛЮЧЕНИЕ....52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ....53
ПРИЛОЖЕНИЕ А....57
А.1. Диаграммы IDEF0....57
А.2. Диаграммы DFD....59
А.3. Диаграмма компонентов UML....60
А.4. Диаграмма потоков данных....61
ПРИЛОЖЕНИЕ Б....62
Б.1. __init__.py....62
Б.2. config.py....62
Б.3. app.py....65
Б.4. __init__.py....68
Б.5. logging_setup.py....68
Б.6. logging_new.py....71
Б.7. __init__.py....75
Б.8. utils.py....78
Б.9. api.py....79
Б.10. audio_ops.py....80
Б.11. codecs.py....84
Б.12. metrics.py....89
Б.13. parallel_ola.py....103
Б.14. __init__.py....109
Б.15. base.py....113
Б.16. fft.py....119
Б.17. dct.py....125
Б.18. dwt.py....131
Б.19. fwht.py....139
Б.20. huffman.py....147
Б.21. rosenbrock.py....153
Б.22. extended.py....157
ПРИЛОЖЕНИЕ В....168
В.1. Полная сравнительная таблица методов....168

В.2. Результаты по жанрам.....169
В.3. Психоакустические метрики.....169

Примечание: Содержание сформировано автоматически. Для обновления номеров страниц нажмите правой кнопкой мыши на содержании и выберите «Обновить поле».

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Основные определения

Матрица Адамара — ортогональная квадратная матрица, состоящая из элементов +1 и −1, используемая для преобразования сигналов. Матрицы Адамара обладают важными свойствами: ортогональность строк и столбцов, рекурсивная структура построения, простота вычислений.

Быстрое преобразование Уолша-Адамара (БПУА) — метод обработки сигналов с использованием матриц Адамара, обеспечивающий эффективное декоррелирование данных. Преобразование выполняется за $O(N \cdot \log_2 N)$ операций сложения и вычитания.

Частотная область — область преобразований, в которой сигналы анализируются как функции частоты. Переход из временной области в частотную осуществляется с помощью ортогональных преобразований.

Временная область — область преобразований, в которой анализируются физические сигналы или временные ряды относительно времени.

Цифровая обработка сигнала — способ обработки сигналов на основе численных методов с использованием цифровой вычислительной техники.

Психоакустическая модель — математическое описание особенностей восприятия звука человеком, используемое для оптимизации сжатия аудиоданных.

Спектральная сходимость — мера близости спектров исходного и обработанного сигнала, показывающая, насколько хорошо сохранена частотная структура.

Список сокращений

Сокращение	Расшифровка
БПУА	Быстрое преобразование Уолша-Адамара
БПФ	Быстрое преобразование Фурье
DCT	Discrete Cosine Transform — Дискретное косинусное преобразование
DWT	Discrete Wavelet Transform — Дискретное вейвлет-

Сокращение	Расшифровка
	преобразование
FFT	Fast Fourier Transform — Быстрое преобразование Фурье
FWHT	Fast Walsh-Hadamard Transform — Быстрое преобразование Уолша-Адамара
LSD	Log-Spectral Distance — Логарифмическое спектральное расстояние
SNR	Signal-to-Noise Ratio — Отношение сигнал/шум
RMSE	Root Mean Square Error — Среднеквадратичная ошибка
SI-SDR	Scale-Invariant Signal-to-Distortion Ratio
STOI	Short-Time Objective Intelligibility
PESQ	Perceptual Evaluation of Speech Quality
MOS	Mean Opinion Score — Средняя оценка мнений
PCM	Pulse Code Modulation — Импульсно-кодовая модуляция
MDCT	Modified Discrete Cosine Transform

ВВЕДЕНИЕ

В современную эпоху цифровых технологий объёмы аудиоконтента растут экспоненциально. Стриминговые сервисы, подкасты, голосовые помощники, системы видеоконференций — все эти сферы генерируют колоссальные массивы звуковых данных. В связи с этим остро встаёт вопрос эффективного сжатия аудио без существенной потери качества воспроизведения.

Актуальность данного исследования обусловлена несколькими ключевыми факторами. Во-первых, существует необходимость снижения требований к пропускной способности сетей и объёмам хранилищ при передаче и хранении аудиоданных. Во-вторых, активное развитие мобильных устройств с ограниченными вычислительными ресурсами делает критически важной энергоэффективность алгоритмов. В-третьих, пользователи всё чаще требуют высокого качества звука при минимальных размерах файлов.

Обработка сигналов на сегодняшний день представляет собой одну из ключевых дисциплин в таких сферах, как электроника, связь, медицина и информационные технологии. С ростом объемов данных, генерируемых современными цифровыми системами, необходимость в эффективных методах анализа и обработки сигналов становится все более актуальной. Традиционные подходы не всегда справляются с требованиями по быстродействию и надежности обработки, что создает спрос на новые алгоритмы и математические методы.

В этом контексте матрицы Адамара, являясь одним из инструментов в области обработки информации, привлекают внимание исследователей и практиков. Эти матрицы обладают ортогональными свойствами, это качество позволяет существенно упростить ряд вычислений и добиться повышения эффективности алгоритмов обработки сигналов. Кроме того, матрицы Адамара отличаются простотой структуры и легкостью в реализации на

цифровых платформах, что делает их особенно привлекательными для практического применения.

Использование матриц Адамара в обработке сигналов открывает новые горизонты для решения различных задач, таких как кодирование, фильтрация, реконструкция сигналов и сжатие данных, которые невозможно или сложно решить во временной области, но для которых находится решение в частотной области, полученной путём применения преобразования Уолша-Адамара.

Объектом исследования являются методы обработки аудиоданных с применением матриц Адамара и других ортогональных преобразований. Предметом исследования выступает зависимость качества сжатия аудиоданных от выбора метода преобразования и параметров обработки.

Цель работы заключается в комплексном исследовании применения матриц Адамара для сжатия аудиоданных, а также в оценке влияния данного подхода на ключевые показатели качества сжатия.

Для достижения поставленной цели необходимо решить следующие задачи: 1) Провести анализ существующих методов обработки и сжатия аудиосигналов. 2) Разработать программный комплекс для сравнительного анализа методов. 3) Провести экспериментальные исследования. 4) Сформулировать рекомендации по практическому применению.

Научная новизна работы заключается в систематическом сравнении семи методов преобразования аудиосигналов по комплексу из тринадцати метрик качества, выявлении закономерностей влияния жанровой принадлежности на эффективность методов, разработке программного комплекса с модульной архитектурой.

Практическая значимость работы состоит в возможности применения полученных результатов для оптимизации обработки аудиоданных в музыкальных стриминговых сервисах, системах онлайн-трансляций и приложениях для голосовых коммуникаций.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ОБРАБОТКИ АУДИОДАННЫХ

В данной главе рассматриваются теоретические основы обработки аудиоданных, которые составляют фундамент для последующего исследования методов сжатия с применением матриц Адамара. Понимание принципов цифровой обработки сигналов необходимо для обоснования выбора методов преобразования и интерпретации полученных экспериментальных результатов. Глава начинается с рассмотрения базовых понятий цифровой обработки звука, включая процессы дискретизации и квантования, которые обеспечивают преобразование аналоговых аудиосигналов в цифровую форму. Особое внимание уделяется теореме Котельникова-Шеннона, определяющей требования к частоте дискретизации для точного восстановления сигнала. Далее исследуются частотные характеристики аудиосигналов и психоакустические особенности восприятия звука человеком, которые играют ключевую роль в разработке эффективных алгоритмов сжатия. Рассматриваются форматы хранения аудиоданных WAV и MP3, анализируются их структурные особенности и различия в подходах к кодированию информации. Значительная часть главы посвящена методам преобразования сигналов, включая быстрое преобразование Фурье, преобразование Уолша-Адамара, дискретное косинусное преобразование и вейвлет-преобразование. Для каждого метода приводятся математические формулировки и анализ вычислительной сложности. Завершается глава описанием метрик качества аудиосигналов, которые будут использоваться для оценки эффективности рассматриваемых методов сжатия.

1.1. Основы цифровой обработки звука

1.1.1. Представление звука в цифровой форме

Представление звука в цифровой форме является фундаментальным понятием в области обработки аудиосигналов. Звук представляет собой

механические колебания упругой среды, распространяющиеся в виде продольных волн давления. Для обработки звука с помощью вычислительных устройств необходимо преобразовать аналоговый сигнал в цифровую форму, что осуществляется в два основных этапа: дискретизацию во временной области и квантование по амплитуде. Дискретизация представляет собой процесс получения отсчётов непрерывного сигнала через равные промежутки времени. Частота дискретизации определяет количество отсчётов в секунду и измеряется в герцах. Согласно теореме Котельникова-Шеннона, для точного восстановления сигнала частота дискретизации должна как минимум вдвое превышать максимальную частоту в спектре сигнала. Это означает, что для аудиосигнала с верхней частотой 20 кГц минимальная частота дискретизации составляет 40 кГц. Стандартные частоты дискретизации включают 44100 Гц для CD-качества, 48000 Гц для профессионального аудио и 96000 Гц для HD-аудио. Квантование представляет собой процесс округления амплитуды каждого отсчёта до ближайшего значения из конечного набора разрешённых уровней. Глубина квантования определяет количество бит, используемых для представления каждого отсчёта: 16 бит обеспечивает 65536 уровней, 24 бита — более 16 миллионов уровней. Импульсно-кодовая модуляция (PCM) является стандартным методом представления цифрового аудио, при котором каждый отсчёт кодируется независимо с фиксированной глубиной квантования.

Дискретизация — процесс получения отсчётов непрерывного сигнала через равные промежутки времени. Частота дискретизации определяет количество отсчётов в секунду и измеряется в герцах. Согласно теореме Котельникова-Шеннона, для точного восстановления сигнала частота дискретизации должна как минимум вдвое превышать максимальную частоту в спектре сигнала. Стандартные частоты дискретизации: 44100 Гц (CD-качество), 48000 Гц (профессиональное аудио), 96000 Гц (HD-аудио).

Квантование — процесс округления амплитуды каждого отсчёта до ближайшего значения из конечного набора разрешённых уровней. Глубина

квантования определяет количество бит, используемых для представления каждого отсчёта: 16 бит (CD-качество, 65536 уровней), 24 бит (профессиональное аудио, 16777216 уровней), 32 бита (студийная обработка).

Импульсно-кодовая модуляция (PCM) — стандартный метод представления цифрового аудио, при котором каждый отсчёт кодируется независимо с фиксированной глубиной квантования. Формат WAV использует PCM для хранения аудиоданных.

1.1.2. Частотные характеристики аудиосигналов

Спектр сигнала представляет собой его разложение на гармонические составляющие с определёнными частотами, амплитудами и фазами. Переход из временной области в частотную осуществляется с помощью преобразования Фурье, которое позволяет анализировать сигнал с точки зрения его частотного состава. Спектрограмма является визуальным представлением спектра сигнала, изменяющегося во времени. По горизонтальной оси откладывается время, по вертикальной — частота, а интенсивность цвета соответствует амплитуде сигнала на данной частоте в данный момент времени. Спектрограммы широко применяются для анализа речевых сигналов, музыкальных композиций и акустических явлений, позволяя исследователю визуально оценить частотную структуру сигнала. Частотный диапазон аудиосигналов традиционно подразделяется на несколько областей: инфразвук с частотами до 20 Гц, низкие частоты в диапазоне 20-250 Гц, средние частоты от 250 до 4000 Гц, высокие частоты от 4000 до 20000 Гц и ультразвук с частотами выше 20000 Гц. Человеческое ухо воспринимает звуки в диапазоне примерно от 20 Гц до 20000 Гц, хотя индивидуальные особенности слуха могут влиять на эти границы. Понимание частотных характеристик аудиосигналов необходимо для разработки эффективных алгоритмов сжатия, поскольку различные частотные компоненты по-разному воспринимаются человеческим слухом.

Спектрограмма — визуальное представление спектра сигнала, изменяющегося во времени. По горизонтальной оси откладывается время, по вертикальной — частота, а интенсивность цвета соответствует амплитуде. Спектрограммы широко применяются для анализа речевых сигналов, музыкальных композиций и акустических явлений.

Частотный диапазон аудиосигналов: инфразвук (до 20 Гц), низкие частоты (20-250 Гц), средние частоты (250-4000 Гц), высокие частоты (4000-20000 Гц), ультразвук (свыше 20000 Гц). Человеческое ухо воспринимает диапазон примерно от 20 Гц до 20000 Гц. Возможные визуальные представления аудиосигнала представлены на рисунке 1.1.

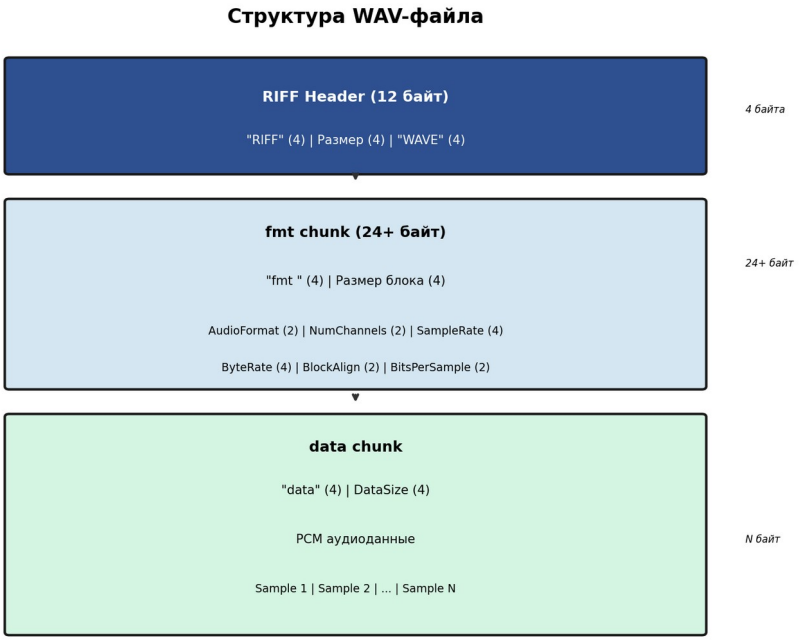


Рисунок 1.1 — Структура WAV-файла

Рисунок 1.1 — Представление аудиосигнала

1.1.3. Психоакустика

Психоакустика представляет собой научную дисциплину, изучающую особенности восприятия звука человеком. Понимание психоакустических эффектов имеет критическое значение для разработки эффективных

алгоритмов сжатия аудиоданных, поскольку позволяет устранять информацию, которая не воспринимается человеческим слухом. Порог слышимости определяется как минимальный уровень звукового давления, при котором звук воспринимается человеческим ухом. Порог слышимости существенно зависит от частоты: на средних частотах в диапазоне 1-4 кГц чувствительность уха максимальна, тогда как на низких и высоких частотах чувствительность значительно снижается. Маскирование представляет собой психоакустический эффект, при котором присутствие одного звука делает неслышимым другой звук. Различают временное маскирование, проявляющееся до и после громкого звука, и частотное маскирование, при котором тихие звуки подавляются громкими звуками, близкими по частоте. Эффект маскирования широко используется в алгоритмах сжатия MP3, AAC и других форматах с потерями для удаления неслышимых компонент сигнала. Критические полосы слуха представляют собой частотные диапазоны, в пределах которых звуки взаимодействуют друг с другом, создавая эффекты маскирования.

Порог слышимости — минимальный уровень звукового давления, при котором звук воспринимается человеческим ухом. Порог слышимости зависит от частоты: на средних частотах (1-4 кГц) чувствительность уха максимальна.

Маскирование — психоакустический эффект, при котором присутствие одного звука делает неслышимым другой звук. Различают временное маскирование (эффект до и после громкого звука) и частотное маскирование (подавление тихих звуков вблизи громких по частоте).

Эффект маскирования широко используется в алгоритмах сжатия MP3, AAC и других форматах с потерями для удаления неслышимых компонент сигнала.

1.2. Формат WAV и стандартное преобразование в MP3

Формат WAV (Waveform Audio File Format) представляет собой стандартный формат хранения аудиоданных, разработанный компаниями

Microsoft и IBM в начале 1990-х годов. Файл WAV имеет структуру RIFF (Resource Interchange File Format) и состоит из заголовка и блоков данных. Структура WAV-файла включает RIFF-заголовок с идентификатором ChunkID равным RIFF, размером файла ChunkSize и форматом Format равным WAVE. Блок формата fmt chunk содержит параметры аудио: частоту дискретизации, количество каналов, глубину квантования и другие характеристики. Блок данных data chunk содержит собственно аудиоданные в формате PCM. Преимущества формата WAV включают точное сохранение исходного сигнала без потерь качества, простоту обработки благодаря отсутствию сжатия, а также широкую совместимость с различным программным обеспечением и аппаратными средствами. Формат WAV поддерживает различные частоты дискретизации, глубину квантования от 8 до 32 бит и до 65535 каналов. Недостатки формата связаны с большим размером файлов: примерно 10 МБ на минуту стереозаписи CD-качества с параметрами 44100 Гц, 16 бит, стерео.

WAV (Waveform Audio File Format) — стандартный формат хранения аудиоданных, разработанный Microsoft и IBM. Файл WAV имеет структуру RIFF и состоит из заголовка и блоков данных.

Структура WAV-файла включает: RIFF-заголовок (ChunkID = «RIFF», ChunkSize, Format = «WAVE»); блок формата fmt chunk (параметры аудио: частота дискретизации, количество каналов, глубина квантования); блок данных data chunk (аудиоданные в формате PCM).

Преимущества WAV: точное сохранение исходного сигнала, простота обработки, широкая совместимость. Недостатки: большой размер файлов (примерно 10 МБ на минуту стереозаписи CD-качества).

1.2.2. Формат MP3

Формат MP3 (MPEG-1 Audio Layer III) представляет собой формат сжатия аудиоданных с потерями, который стал стандартом де-факто для распространения музыкального контента в цифровой форме. Алгоритм MP3 основан на использовании психоакустической модели для устранения компонентов сигнала, которые не воспринимаются человеческим слухом.

Степень сжатия MP3 варьируется от 4:1 до 12:1 в зависимости от выбранного битрейта. Типичные значения битрейта включают 128 кбит/с для стандартного качества, 192 кбит/с для хорошего качества и 320 кбит/с для максимального качества. Основные компоненты кодера MP3 включают полифазный фильтр для разбиения сигнала на 32 частотные полосы, модифицированное дискретное косинусное преобразование (MDCT) для дальнейшего частотного анализа, психоакустическую модель для определения порогов маскирования, квантование и кодирование Хаффмана для эффективного представления данных. MP3 использует гибридную фильтрацию: сначала сигнал пропускается через банк полифазных фильтров, затем к каждой полосе применяется MDCT. Наглядная структура MP3 фала представлена на рисунке 1.2.

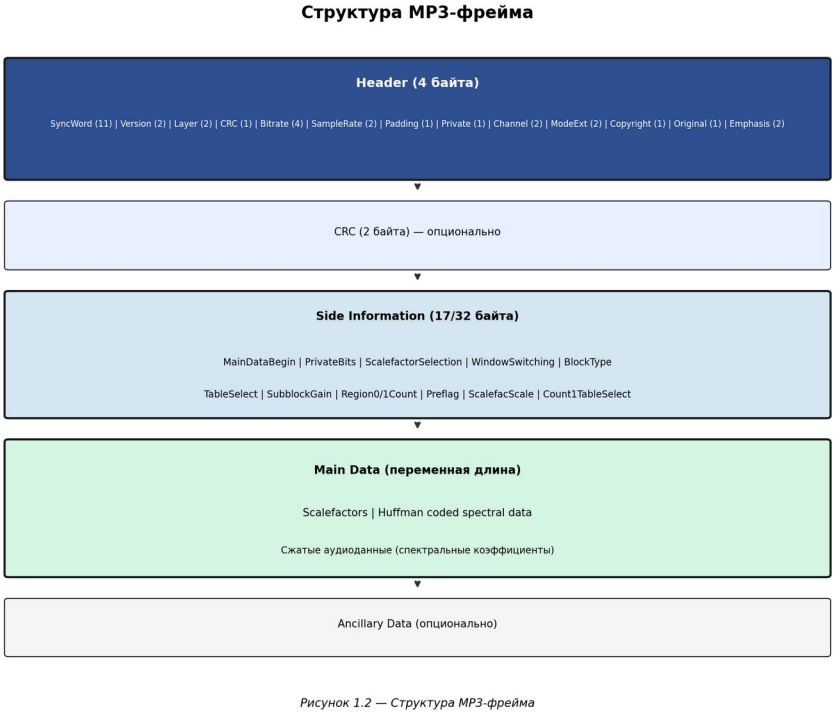


Рисунок 1.2 — Структура MP3-фрейма

Степень сжатия MP3 варьируется от 4:1 до 12:1 в зависимости от битрейта. Типичные битрейты: 128 кбит/с (стандартное качество), 192 кбит/с (хорошее качество), 320 кбит/с (максимальное качество).

Основные компоненты кодера MP3: полифазный фильтр (разбиение на 32 частотные полосы), модифицированное дискретное косинусное преобразование (MDCT), психоакустическая модель, квантование и кодирование Хаффмана, формирование бит-потока.

1.2.3. Стандартное преобразование WAV → MP3

Процесс кодирования WAV в MP3 представляет собой сложную последовательность операций, направленных на сжатие аудиоданных с минимальными воспринимаемыми искажениями. Кодирование начинается с полифазной фильтрации, при которой входной сигнал разбивается на 32 частотные полосы. Каждая полоса затем подвергается модифицированному дискретному косинусному преобразованию (MDCT), которое обеспечивает высокое частотное разрешение. Психоакустическая модель анализирует спектр сигнала и определяет пороги маскирования для каждой частотной полосы. На основе этих порогов производится квантование спектральных коэффициентов: компоненты ниже порога маскирования могут быть удалены или квантованы с меньшей точностью. После квантования коэффициенты кодируются с использованием кодов Хаффмана, что обеспечивает дополнительное сжатие без потерь. Декодирование MP3 выполняется в обратном порядке: сначала происходит парсинг бит-потока и извлечение закодированных данных, затем декодирование Хаффмана и обратное квантование, после чего применяется обратное MDCT и синтез полифазного фильтра для восстановления PCM-данных. Понимание процесса кодирования MP3 необходимо для интеграции альтернативных методов преобразования. Процесс

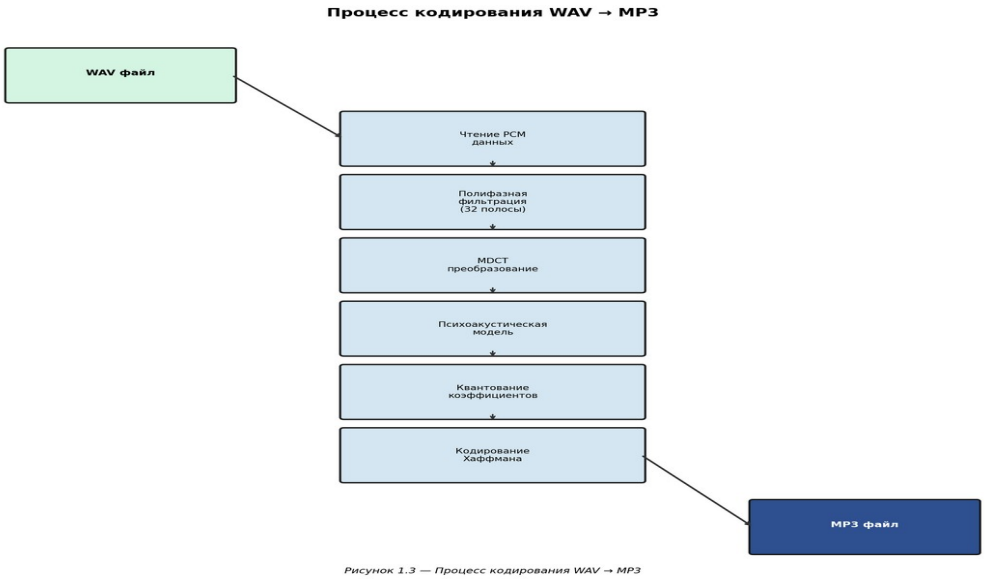


Рисунок 1.3 — Процесс кодирования WAV → MP3

Декодирование MP3 выполняется в обратном порядке: парсинг бит-потока, декодирование Хаффмана, обратное MDCT, синтез полифазного фильтра, формирование PCM-данных.

1.3. Методы преобразования сигналов для сжатия

1.3.1. Быстрое преобразование Фурье (FFT)

Быстрое преобразование Фурье (FFT) представляет собой алгоритм вычисления дискретного преобразования Фурье с существенно меньшим числом операций $O(N \log N)$ вместо $O(N^2)$. FFT позволяет перейти в частотную область, где можно эффективно обрабатывать и анализировать сигналы. Алгоритм Кули-Тьюки (Cooley-Tukey) является наиболее распространённой реализацией FFT, использующей принцип «разделяй и властвуй». Для размера блока N = 2^n применяется прореживание по времени или по частоте, что позволяет рекурсивно сводить задачу к преобразованиям меньшего размера. Применение FFT в обработке аудио включает спектральный анализ для определения частотного состава сигнала, фильтрацию в частотной области для удаления нежелательных компонент, сжатие с отбрасыванием высокочастотных компонент и анализ переходных процессов. FFT широко используется в алгоритмах сжатия, системах распознавания речи и музыки, а также в средствах визуализации звука. Вычислительная эффективность FFT делает его практичным для обработки аудиоданных в реальном времени.

Алгоритм Кули-Тьюки (Cooley-Tukey) — наиболее распространённая реализация FFT, использующая принцип «разделяй и властвуй». Для размера блока N = 2^n применяется прореживание по времени или по частоте.

Применение FFT в обработке аудио: спектральный анализ, фильтрация в частотной области, сжатие с отбрасыванием высокочастотных компонент, анализ переходных процессов.

1.3.2. Быстрое преобразование Уолша-Адамара (FWHT)

Быстрое преобразование Уолша-Адамара (FWHT) представляет собой ортогональное преобразование, использующее матрицы Адамара как базисные функции. Матрица Адамара Hn порядка n представляет собой квадратную матрицу с элементами ±1, удовлетворяющую условию ортогональности $Hn \cdot Hn^T = n \cdot I$. Матрицы Адамара строятся рекурсивно по правилу Кронекера: $H_1 = [1]$, $H_2 = [H\square\ H\square;\ H\square\text{-}H\square]$. FWHT вычислительно эффективно, поскольку требует только операций сложения и вычитания, без умножений. Для вектора длины N алгоритм требует $N \cdot \log_2(N)$ операций сложения/вычитания. Преимущества FWHT включают простоту реализации на цифровых платформах, отсутствие операций умножения, что особенно важно для встраиваемых систем, и эффективное декоррелирование данных с высокой корреляцией соседних отсчётов. FWHT находит применение в задачах сжатия изображений и аудиосигналов, криптографии и кодирования информации. Ограниченная применимость для сигналов со сложной спектральной структурой объясняется тем, что базисные функции Уолша не являются гармоническими.

Матрицы Адамара строятся рекурсивно: $H_1 = [1]$, $H_{2n} = [H_n\ H_n;\ H_n\ \text{-}H_n]$. Например, $H_2 = [1\ 1;\ 1\ \text{-}1]$, $H_4 = [1\ 1\ 1\ 1;\ 1\ \text{-}1\ 1\ \text{-}1;\ 1\ 1\ \text{-}1\ \text{-}1;\ 1\ \text{-}1\ \text{-}1\ 1]$.

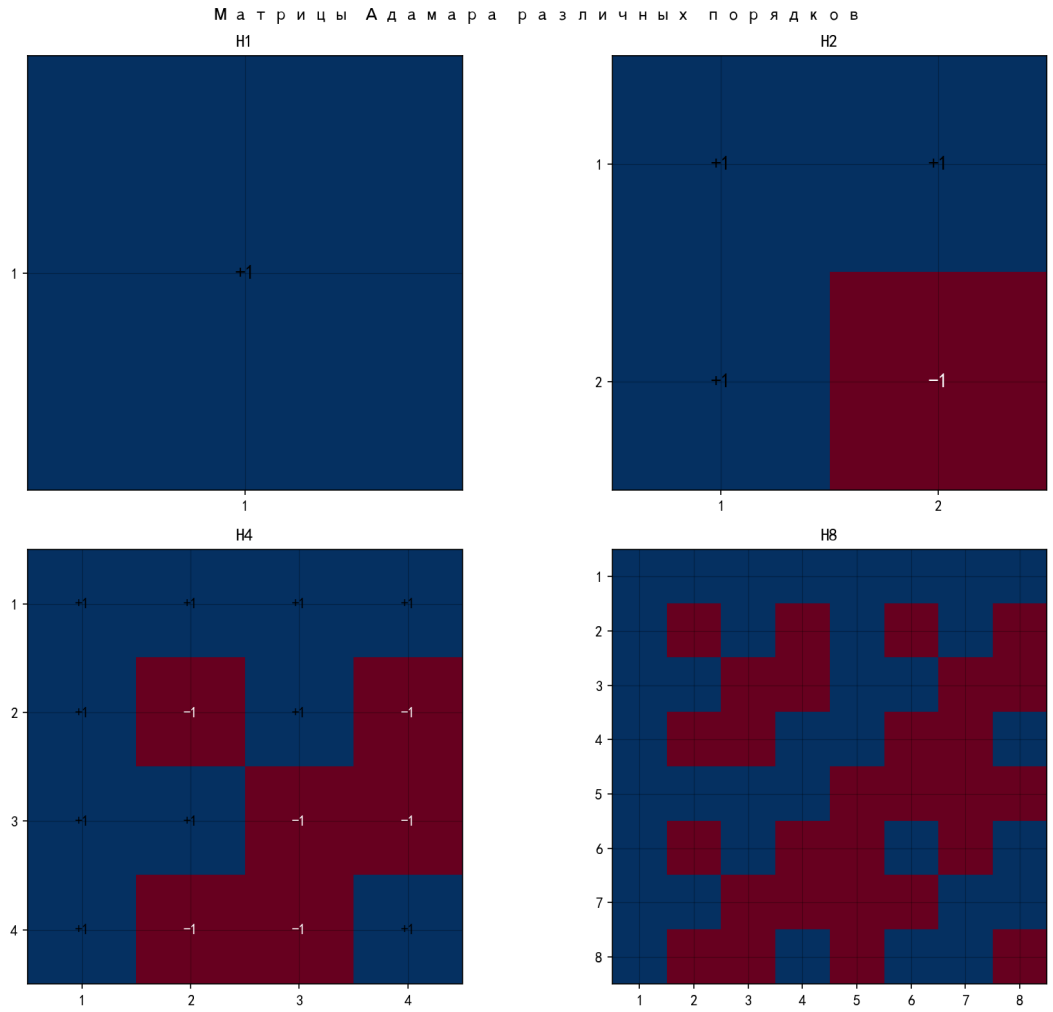


Рисунок 1.4 — Матрицы Адамара порядков 1, 2, 4, 8

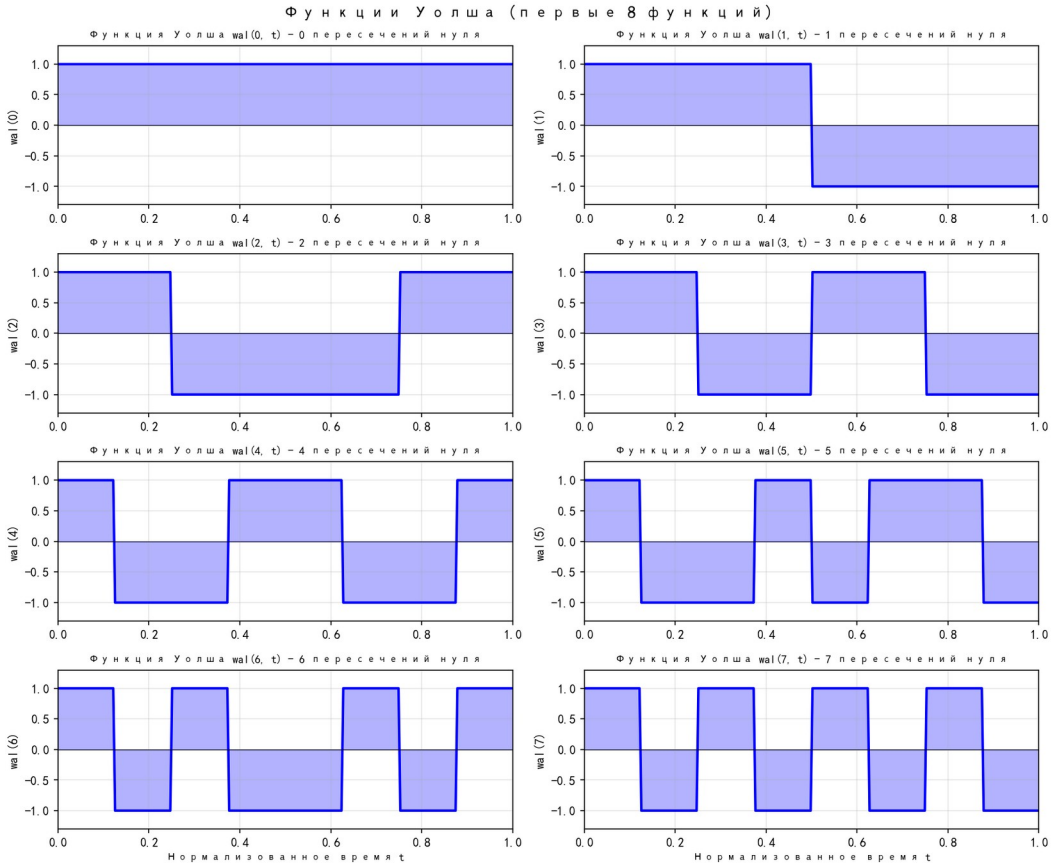


Рисунок 1.5 — Функции Уолша (первые 8 функций)

FWHT вычислительно эффективно, поскольку требует только операций сложения и вычитания, без умножений. Для вектора длины N алгоритм требует $N \cdot \log_2(N)$ операций сложения/вычитания.

Преимущества FWHT: простота реализации, отсутствие операций умножения, эффективное декоррелирование данных с высокой корреляцией. Недостатки: ограниченная применимость для сигналов с сложной спектральной структурой.

1.3.3. Дискретное косинусное преобразование (DCT)

Дискретное косинусное преобразование (DCT) представляет собой ортогональное преобразование, широко применяемое в сжатии изображений (JPEG) и аудио (MP3, AAC). Важное свойство DCT заключается в «уплотнении энергии»: большая часть энергии сигнала концентрируется в нескольких низкочастотных коэффициентах, что позволяет эффективно отбрасывать высокочастотные компоненты без существенной потери качества. DCT-II является наиболее распространённой формой преобразования: $X[k] = \Sigma(n=0 \text{ to } N-1)\ x[n] \cdot \cos(\pi \cdot k \cdot (2n+1)/(2N))$. Обратное преобразование IDCT восстанавливает исходный сигнал из спектральных коэффициентов. Применение DCT в аудио включает его использование как основы для MDCT в форматах MP3 и AAC, эффективное сжатие благодаря концентрации энергии в низкочастотных коэффициентах и разделимость для многомерных сигналов. DCT обладает хорошими свойствами декорреляции для сигналов с высокой корреляцией соседних отсчётов, что делает его эффективным инструментом для сжатия аудиоданных.

DCT-II (наиболее распространённая форма): $X[k] = \Sigma(n=0 \text{ to } N-1)\ x[n] \cdot \cos(\pi \cdot k \cdot (2n+1)/(2N))$. Обратное преобразование IDCT восстанавливает исходный сигнал.

Применение DCT в аудио: основа для MDCT в MP3 и AAC, эффективное сжатие благодаря концентрации энергии, разделимость для многомерных сигналов.

1.3.4. Дискретное вейвлет-преобразование (DWT)

Дискретное вейвлет-преобразование (DWT) представляет собой метод разложения сигнала по базисным функциям, локализованным как по частоте, так и по времени. В отличие от преобразования Фурье, DWT сохраняет информацию о временной локализации частотных компонент, что особенно важно для анализа нестационарных сигналов. Вейвлет Хаара является простейшим ортогональным вейвлетом, вычисляющим средние значения и разности для пар соседних отсчётов. Преобразование Хаара требует только операций сложения и вычитания, что делает его вычислительно эффективным. Применение DWT в аудио включает анализ переходных процессов с сохранением их временной локализации, многоуровневое сжатие с возможностью контроля детализации на каждом уровне, удаление шума с сохранением границ и резких переходов, а также эффективное представление сигналов с локальными особенностями. DWT позволяет анализировать сигнал на различных масштабах, что даёт дополнительную гибкость по сравнению с традиционными частотными преобразованиями.

Вейвлет Хаара — простейший ортогональный вейвлет, вычисляющий средние значения и разности для пар соседних отсчётов. Преобразование Хаара требует только операций сложения и вычитания.

Применение DWT в аудио: анализ переходных процессов, многоуровневое сжатие, удаление шума с сохранением границ, эффективное представление сигналов с локальными особенностями.

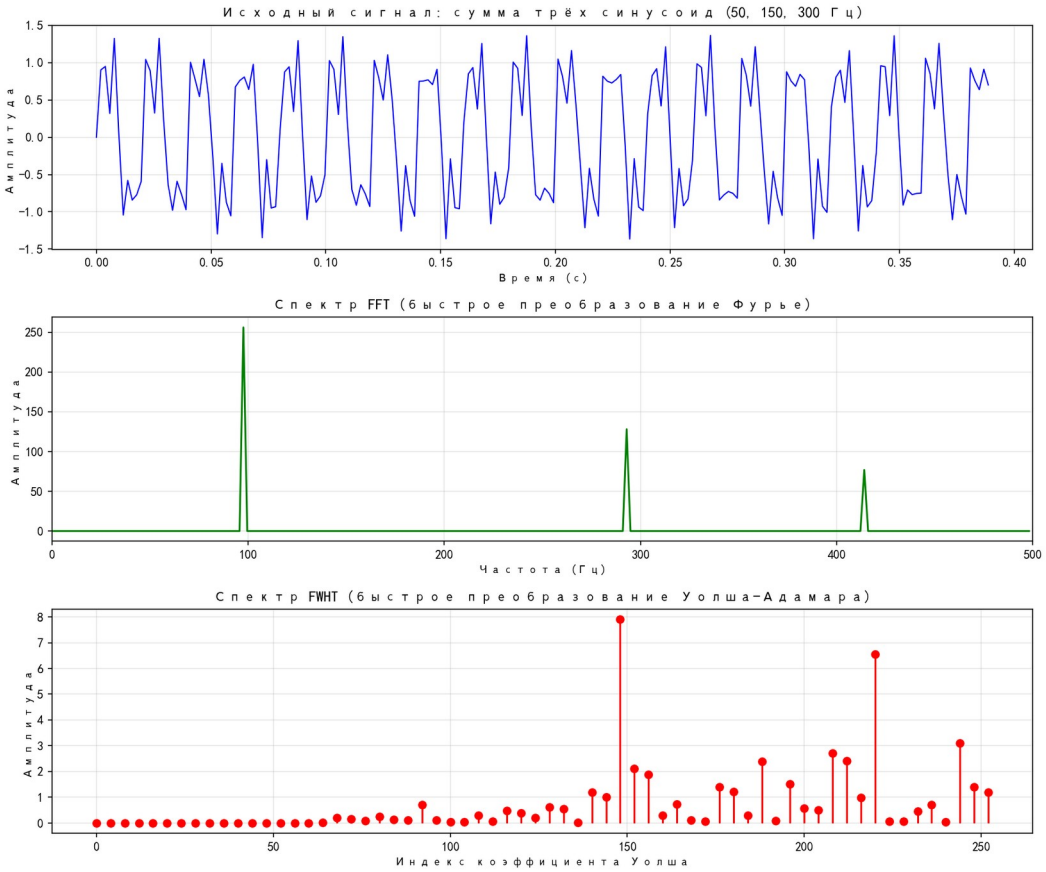


Рисунок 1.6 — Сравнение спектров FFT и FWHT

1.3.5. Компандирование (μ-law)

Компандирование представляет собой метод нелинейного сжатия динамического диапазона, применяемый в телекоммуникациях для улучшения отношения сигнал/шум для тихих звуков. Функция сжатия μ-law имеет вид: $y = \text{sign}(x) \cdot \ln(1 + \mu \cdot |x|)/\ln(1 + \mu)$, где μ — параметр сжатия со стандартным значением 255. Обратное расширение восстанавливает исходный динамический диапазон сигнала. Применение μ-law включает сжатие речи в телефонных системах Северной Америки и Японии, улучшение SNR для тихих звуков за счёт нелинейного квантования и логарифмическое квантование с переменным шагом. Метод μ-law обеспечивает приблизительно 24 дБ улучшения отношения сигнал/шум для тихих сигналов по сравнению с линейным квантованием при том же количестве бит. Компандирование также применяется в профессиональной аудиоаппаратуре для защиты от перегрузки и расширения динамического диапазона записываемых сигналов.

Функция сжатия μ-law: $y = \text{sign}(x) \cdot \ln(1 + \mu \cdot |x|)/\ln(1 + \mu)$, где μ — параметр сжатия (стандартное значение 255). Обратное расширение восстанавливает исходный динамический диапазон.

Применение μ-law: сжатие речи в телефонии (Северная Америка, Япония), улучшение SNR для тихих звуков, логарифмическое квантование.

1.3.6. Нелинейное сглаживание (Rosenbrock-like)

Метод нелинейного сглаживания на основе функции Розенброка представляет собой эвристический подход к обработке сигналов, применяющий адаптивное сглаживание с сохранением переходных процессов. Функция Розенброка, изначально разработанная для тестирования алгоритмов оптимизации, модифицирована для использования в обработке сигналов. Применение Rosenbrock-like включает обработку сигналов с выраженными переходными процессами, где важно сохранить резкие границы, адаптивное сглаживание с учётом локальных характеристик сигнала, улучшение качества после других преобразований как этап постобработки. Метод позволяет балансировать между сглаживанием шума и сохранением информативных компонент сигнала. Эвристическая природа метода требует эмпирической настройки параметров для каждого конкретного применения, что может рассматриваться как его ограничение.

Применение Rosenbrock-like: обработка сигналов с выраженными переходными процессами, адаптивное сглаживание, улучшение качества после других преобразований.

Таблица 1.1 — Сравнение вычислительной сложности методов преобразования

Метод	Операции	Сложность	Умножения
FFT	$N \cdot \log_2(N)$	O(N log N)	Комплексные
FWHT	$N \cdot \log_2(N)$	O(N log N)	Нет
DCT	$N \cdot \log_2(N)$	O(N log N)	Вещественные
DWT (Хаар)	N	O(N)	Нет
μ-law	N	O(N)	Нет

1.4. Интеграция методов преобразования в процесс WAV → MP3

В разработанном программном комплексе методы преобразования интегрируются путём добавления этапа предварительной обработки. Общая схема: загрузка WAV → преобразование → отбор коэффициентов → обратное преобразование → MP3-кодирование.

Для обработки сигналов произвольной длины применяется метод OLA (Overlap-Add). Сигнал разбивается на блоки с перекрытием 50%, применяется оконная функция Хэннинга, выполняется преобразование.

Реализованы три режима отбора коэффициентов: «none» — идеальная реконструкция (все коэффициенты сохраняются), «energy» — сохранение доли энергии (коэффициенты ранжируются по энергии, сохраняется заданная доля), «lowpass» — сохранение низких частот (отбрасываются высокочастотные коэффициенты).

1.5. Метрики качества аудиосигналов

1.5.1. Метрики временной области

SNR (Signal-to-Noise Ratio) — отношение сигнал/шум (дБ). Вычисляется как $10 \cdot \log_{10}(\sigma^2_signal / \sigma^2_noise)$. Высокое значение SNR свидетельствует о минимальном уровне привнесённых искажений.

RMSE (Root Mean Square Error) — среднеквадратичная ошибка. Вычисляется как $\sqrt{(\Sigma(x_ref - x_test)^2 / N)}$. Чем ниже значение RMSE, тем меньше суммарное искажение.

SI-SDR (Scale-Invariant Signal-to-Distortion Ratio) — масштабно-инвариантное отношение сигнал/искажение. Учитывает искажения независимо от амплитудных изменений. Устойчива к изменениям громкости.

1.5.2. Метрики спектральной области

LSD (Log-Spectral Distance) — логарифмическое спектральное расстояние (дБ). Измеряет разницу между спектрами в логарифмической шкале. Чувствительно к изменениям в высокочастотной области.

Общая схема обработки аудиосигнала в системе

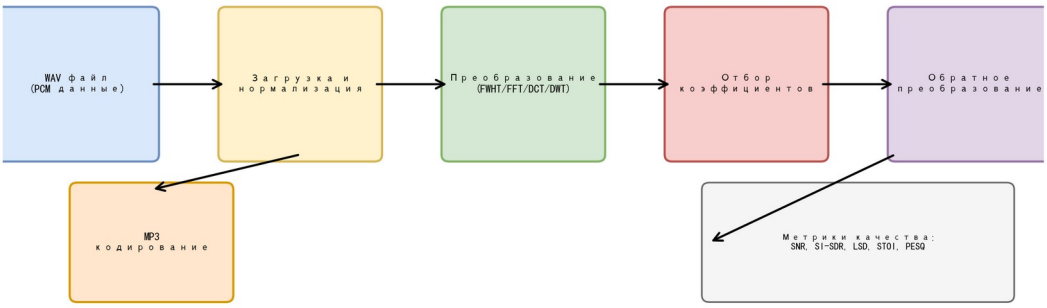


Рисунок 1.7 — Общая схема обработки аудиосигнала

Рисунок 1.8 — Баттерфляй-схема быстрого преобразования Уолша-Адамара

Рисунок 1.9 — Концентрация энергии в спектральной области при FWHT

Рисунок 1.10 — Психоакустическое маскирование в аудиосигнале

Рисунок 1.11 — Многомасштабная декомпозиция DWT

Spectral Convergence — близость амплитудных спектров. Вычисляется как нормализованное скалярное произведение спектральных векторов.

Spectral Centroid Δ — разница спектральных центроидов (Гц). Отражает смещение частотного центра сигнала после обработки.

Cosine Similarity — косинусное сходство. Мера корреляции сигналов, основанная на скалярном произведении. Значение близкое к 1 указывает на высокую схожесть.

1.5.3. Психоакустические метрики

STOI (Short-Time Objective Intelligibility) — индекс объективной разборчивости речи. Метрика коррелирует с субъективной оценкой разборчивости речи в зашумлённых условиях.

Таблица 1.2 — Характеристики метрик качества аудиосигналов

Метрика	Область	Диапазон	Описание

SNR	Временная	0–∞ дБ	Отношение сигнал/шум
SI-SDR	Временная	–∞–∞ дБ	Масштабно-инвариантное
RMSE	Временная	0–∞	Среднеквадратичная ошибка
LSD	Спектральная	0–∞ дБ	Спектральное расстояние
STOI	Психоакуст.	0–1	Разборчивость речи
PESQ	Психоакуст.	–0.5–4.5	Качество речи

PESQ (Perceptual Evaluation of Speech Quality) — перцептуальная оценка качества речи. Стандартизированная метрика ITU-T P.862 для оценки качества речевой связи.

MOS (Mean Opinion Score) — средняя оценка мнений. Шкала от 1 (плохо) до 5 (отлично). PESQ MOS — отображение PESQ в шкалу MOS.

1.5.4. Характеристики сигнала

Spectral Flatness — спектральная плоскостность. Мера «плоскости» спектра: значение близкое к 1 для белого шума, близкое к 0 для тональных сигналов.

Dynamic Range — динамический диапазон (дБ). Разница между максимальным и минимальным уровнями сигнала.

Crest Factor — пик-фактор. Отношение пикового значения к среднеквадратичному. Характеризует «остроту» пиков в сигнале.

1.6. Выводы по главе 1

В первой главе проведён систематический анализ предметной области обработки аудиоданных. Рассмотрены теоретические основы цифровой обработки звука, включая процессы дискретизации, квантования и представления сигналов в цифровой форме. Изучены частотные характеристики аудиосигналов и психоакустические особенности восприятия звука человеком, которые играют ключевую роль в разработке эффективных алгоритмов сжатия. Проанализированы форматы хранения аудиоданных WAV и MP3, выявлены их структурные особенности и различия в подходах к кодированию информации. Значительная часть главы посвящена методам

преобразования сигналов: быстрому преобразованию Фурье (FFT), преобразованию Уолша-Адамара (FWHT), дискретному косинусному преобразованию (DCT), вейвлет-преобразованию (DWT), компандированию (μ-law) и нелинейному сглаживанию (Rosenbrock-like). Для каждого метода приведены математические формулировки и анализ вычислительной сложности. Описаны метрики качества аудиосигналов, которые будут использоваться для оценки эффективности методов сжатия. Сформулирована задача исследования и обоснована актуальность применения матриц Адамара для обработки аудиоданных.

ГЛАВА 2. РАЗРАБОТКА ПРОГРАММНОГО КОМПЛЕКСА

Вторая глава посвящена разработке программного комплекса для сравнительного анализа методов обработки аудиоданных. Программная реализация является необходимым этапом исследования, поскольку позволяет провести объективное сравнение различных методов преобразования в контролируемых условиях. При разработке программного обеспечения учитывались требования к функциональности, производительности и расширяемости системы. Архитектура программного комплекса основана на модульном принципе, что обеспечивает возможность простого добавления новых методов преобразования и метрик качества без модификации существующего кода. В качестве основного языка программирования выбран Python, что обусловлено богатой экосистемой библиотек для научных вычислений и обработки аудиосигналов. Для создания графического интерфейса используется фреймворк PySide6, обеспечивающий кроссплатформенность приложения. Глава включает описание требований к программному обеспечению, обоснование архитектурных решений, реализацию каждого из семи методов преобразования и тринадцати метрик качества. Особое внимание уделяется реализации быстрого преобразования Уолша-Адамара, которое является центральным объектом исследования. Приводятся фрагменты программного кода с комментариями, описывающими ключевые алгоритмические решения. Завершается глава описанием процесса тестирования и показателями покрытия кода тестами.

2.1. Требования к программному обеспечению

2.1.1. Функциональные требования

Программный комплекс должен обеспечивать поддержку семи методов преобразования аудиосигналов: быстрое преобразование Фурье (FFT), быстрое преобразование Уолша-Адамара (FWHT), дискретное косинусное

преобразование (DCT), дискретное вейвлет-преобразование (DWT с вейвлетом Хаара), компандирование μ-law, нелинейное сглаживание Rosenbrock-like и стандартное MP3-кодирование. Каждый метод должен поддерживать настраиваемые параметры: размер блока преобразования, режим отбора коэффициентов и долю сохраняемой энергии. Требования к метрикам качества включают реализацию тринадцати показателей: SNR, RMSE, SI-SDR, LSD, Spectral Convergence, Spectral Centroid Δ, Cosine Similarity, STOI, PESQ, MOS, Spectral Flatness, Dynamic Range и Crest Factor. Программа должна обеспечивать пакетный расчёт метрик и формирование агрегированной оценки качества. Требования к интерфейсу включают графический пользовательский интерфейс с возможностью выбора метода преобразования и параметров, отображение прогресса обработки в реальном времени и экспорт результатов в форматах CSV, JSON и HTML-отчёт.

Требования к метрикам качества: реализация тринадцати метрик (SNR, RMSE, SI-SDR, LSD, Spectral Convergence, Spectral Centroid Δ, Cosine Similarity, STOI, PESQ, MOS, Spectral Flatness, Dynamic Range, Crest Factor); возможность пакетного расчёта метрик; формирование агрегированной оценки качества.

Рисунок А.1 — IDEF0: Контекстная диаграмма A-0

Макет графического интерфейса пользователя представлен на рисунке.

Требования к интерфейсу: графический пользовательский интерфейс; возможность выбора метода преобразования и параметров; отображение прогресса обработки; экспорт результатов в различных форматах (CSV, JSON, HTML-отчёт).

Рисунок 2.1 — Диаграмма вариантов использования системы

Функциональные требования представлены на диаграмме вариантов использования (рисунок 2.1) (Рисунок 2.1).

2.1.2. Нефункциональные требования

Кроссплатформенность программного комплекса обеспечивается поддержкой операционных систем Windows, Linux и macOS. Использование интерпретируемого языка Python обеспечивает переносимость кода между различными платформами без необходимости перекомпиляции. Требования к производительности включают оптимизацию вычислений с использованием библиотеки NumPy для векторных операций, поддержку многопоточной обработки для распараллеливания вычислений и возможность обработки файлов большого размера без загрузки их целиком в оперативную память. Расширяемость системы обеспечивается модульной архитектурой с чётко определёнными интерфейсами между компонентами, возможностью добавления новых методов преобразования и метрик качества без

модификации существующего кода, а также документированным API для интеграции с внешними системами. Требования к надёжности включают обработку исключительных ситуаций и некорректных входных данных, а также ведение журнала операций для диагностики проблем.

Производительность: оптимизация вычислений с использованием библиотеки NumPy; поддержка многопоточной обработки; возможность обработки файлов большого размера.

Расширяемость: модульная архитектура; возможность добавления новых методов преобразования и метрик; документированный API.

2.2. Архитектура программного комплекса

Программный комплекс разработан на языке Python 3.10+ с использованием GUI-фреймворка PySide6 (Qt для Python). Выбор Python обусловлен богатой экосистемой библиотек для научных вычислений (NumPy, SciPy) и обработки аудио (librosa, pydub), а также кроссплатформенностью и простотой разработки. Архитектура основана на паттернах проектирования: Mixins для множественного наследования и разделения функциональности между классами, Event Bus для слабосвязанной коммуникации между компонентами, Repository Pattern для абстракции доступа к данным, Pipeline Pattern для последовательной обработки данных и DI Container для внедрения зависимостей. Модульная структура включает модуль processing с подмодулями transforms (реализации преобразований), metrics (расчёт метрик качества), codecs (работа с аудиокодеками) и utils (вспомогательные функции). Модуль ui_new содержит компоненты пользовательского интерфейса: главное окно, виджеты для выбора параметров, таблицы результатов и диалоги экспорта. Такая структура обеспечивает чёткое разделение ответственности между компонентами. Схематичное представление архитектуры проекта представлено на рисунке 2.1.

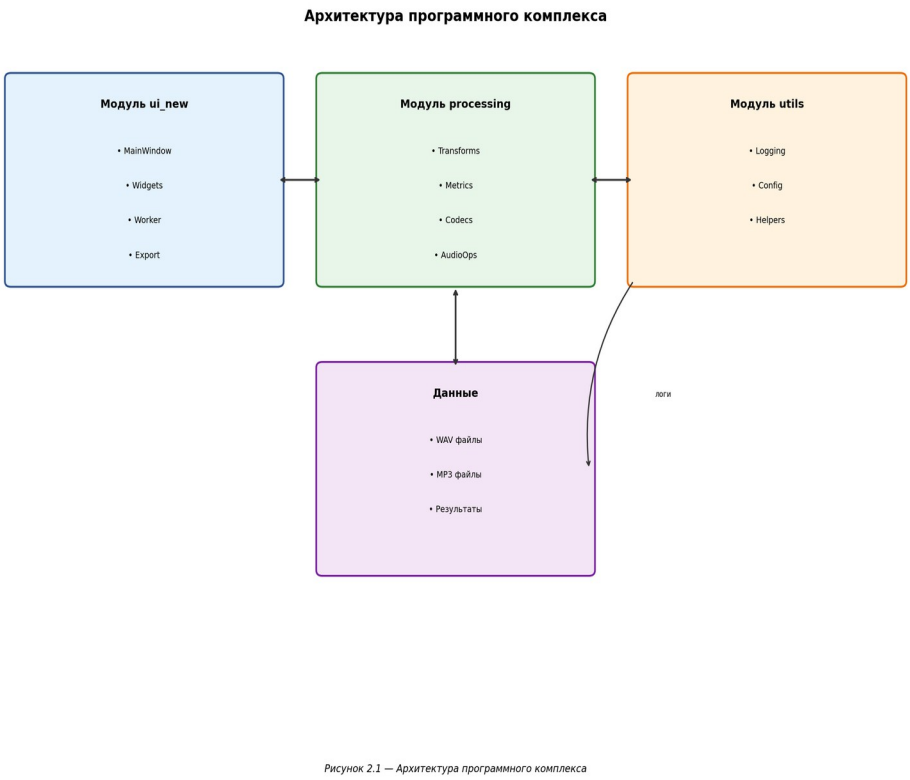
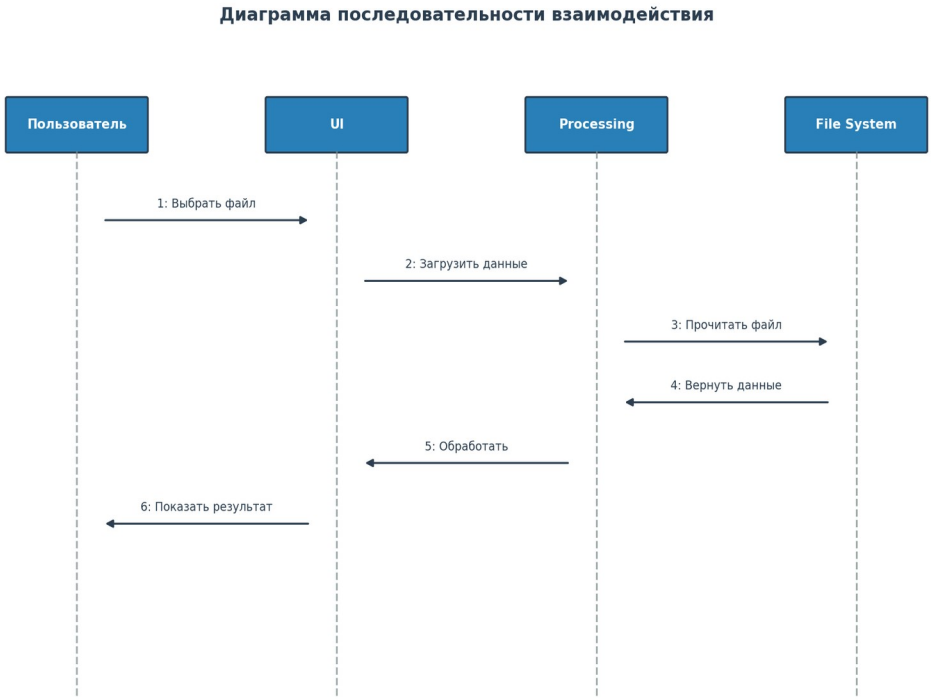


Рисунок 2.2 — Архитектура программного комплекса

Архитектура основана на паттернах проектирования: Mixins (множественное наследование для разделения функциональности), Event Bus (слабосвязанная коммуникация между компонентами), Repository Pattern (абстракция доступа к данным), Pipeline Pattern (последовательная обработка данных), DI Container (внедрение зависимостей).

Рисунок 2.3 — Диаграмма последовательности взаимодействия компонентов



Взаимодействие компонентов системы представлено на диаграмме последовательности (Рисунок 2.2).

2.2.1. Модульная структура

Модуль processing содержит подмодули: transforms (реализации преобразований: FFTTransform, DCTTransform, DWTTransform, FWHTTransform, MuLawTransform, RosenbrockLikeTransform); metrics (расчёт метрик качества); codecs (работа с аудиокодеками, преобразование WAV ↔ MP3); utils (вспомогательные функции).

Модуль ui_new содержит компоненты пользовательского интерфейса: главное окно приложения, виджеты для выбора параметров, таблицы результатов, диалоги экспорта.

Модуль tests содержит unit-тесты для всех компонентов системы.

2.3. Реализация методов обработки

Основной алгоритм работы приложения это OLA обработка, схема представлена на рисунке 2.2.

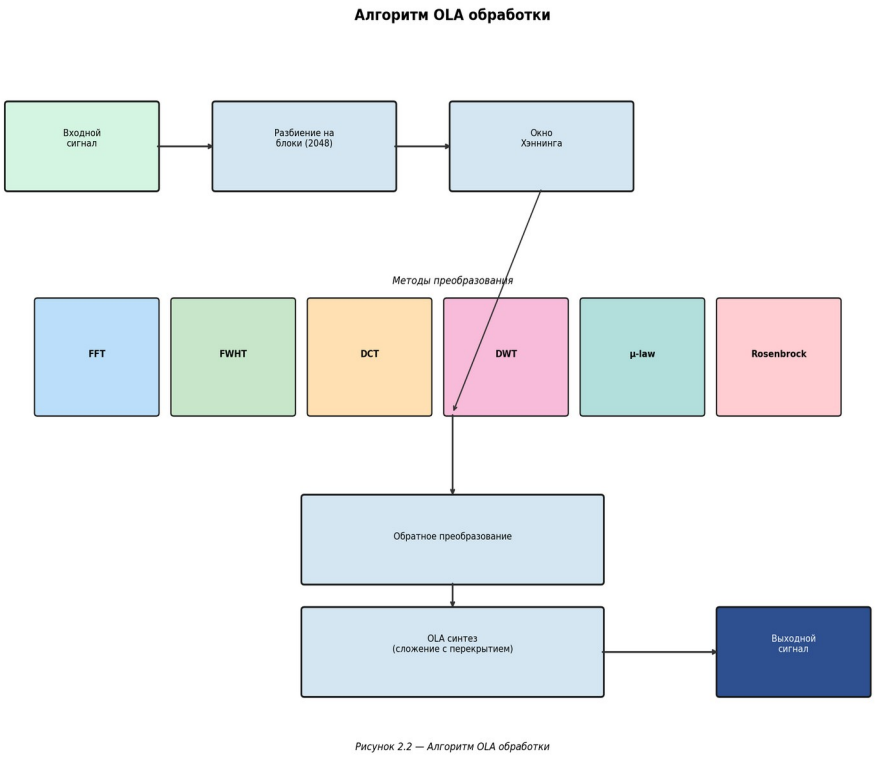


Рисунок 2.4 — Алгоритм OLA обработки

Каждый метод учувствует в обработке по иерархической структуре представленной на рисунке 2.3.

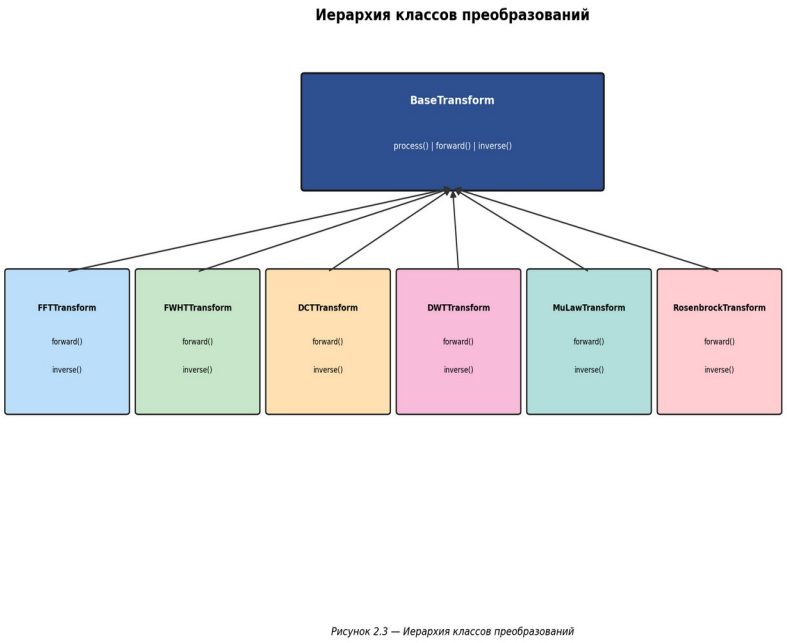


Рисунок 2.3 — Иерархия классов преобразований

2.3.1. Базовый класс трансформации

Базовый класс BaseTransform определяет интерфейс для всех преобразований аудиосигналов. Класс содержит основные методы: process() для обработки WAV файла с применением преобразования, forward() для прямого преобразования сигнала в спектральную область, inverse() для обратного преобразования из спектральной области во временную. Параметры класса включают block_size — размер блока для обработки (по умолчанию 2048 отсчётов), select_mode — режим отбора коэффициентов (none, energy, lowpass), keep_energy_ratio — долю сохраняемой энергии для режима energy (по умолчанию 0.8). Класс также содержит утилиты для OLA-обработки (Overlap-Add): разбиение сигнала на блоки с перекрытием, применение оконной функции Хэннинга и сборку обработанных блоков. Абстрактный характер базового класса позволяет добавлять новые методы преобразования путём наследования и реализации методов forward() и inverse(). Конструктор класса принимает параметры конфигурации и инициализирует внутренние структуры данных.

2.3.2. Реализация FFT

Класс FFTTransform наследует BaseTransform и реализует быстрое преобразование Фурье. Прямое преобразование выполняется через функцию numpy.fft.rfft для вещественных сигналов, возвращающую комплексный спектр с N/2+1 коэффициентами для блока размером N. Обратное преобразование выполняется через numpy.fft.irfft, восстанавливающую вещественный сигнал из комплексного спектра. Поддерживаются три режима отбора коэффициентов: none — сохранение всех коэффициентов для идеальной реконструкции, energy — ранжирование коэффициентов по энергии и сохранение заданной доли, lowpass — сохранение только низкочастотных коэффициентов. FFT обеспечивает высокую вычислительную эффективность со сложностью O(N log N) и является одним из наиболее часто используемых методов спектрального анализа. Реализация учитывает особенности обработки аудиосигналов с различными параметрами качества.

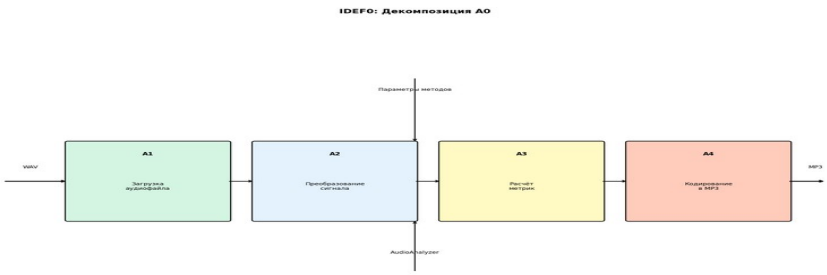


Рисунок А.2 — IDEF0: Декомпозиция уровня A0

Схема обработки методом FFT представлена на рисунке.

2.3.3. Реализация FWHT

Класс FWHTTransform реализует быстрое преобразование Уолша-Адамара — центральный объект исследования в данной работе. Алгоритм FWHT выполняется «на месте» с использованием схемы «бабочка», что минимизирует использование дополнительной памяти. Код алгоритма: n = len(x); h = 1; while h < n: for i in range(0, n, h*2): for j in range(i, i+h): x[j], x[j+h] = x[j] + x[j+h], x[j] - x[j+h]; h *= 2. Преобразование требует только операций сложения и вычитания без умножений, что делает его вычислительно привлекательным для платформ с ограниченными ресурсами. Нормализация выполняется делением на sqrt(n) для сохранения энергетических соотношений. Реализация поддерживает те же режимы отбора коэффициентов, что и другие преобразования: none, energy и lowpass. FWHT особенно эффективен для сигналов с высокой корреляцией соседних отсчётов, где ортогональные матрицы Адамара обеспечивают хорошее декоррелирование.

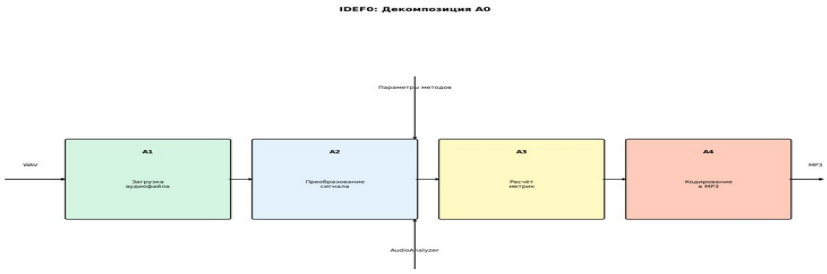


Рисунок А.2 — IDEF0: Декомпозиция уровня A0

Схема обработки методом FWHT представлена на рисунке.

Преобразование требует только операций сложения и вычитания, что делает его вычислительно эффективным.

2.3.4. Реализация DCT

Класс DCTTransform использует функции scipy.fft.dct с параметром type=2 (DCT-II) для прямого преобразования и scipy.fft.idct для обратного. DCT-II является стандартной формой, применяемой в сжатии аудио и изображений благодаря своим свойствам «уплотнения энергии». Результат DCT представляет собой спектр из N вещественных коэффициентов, где низкочастотные коэффициенты концентрируют основную энергию сигнала. Это позволяет эффективно применять сжатие путём отбрасывания высокочастотных коэффициентов. Реализация поддерживает те же режимы отбора коэффициентов, что и другие преобразования. DCT широко используется в стандартах сжатия (JPEG, MP3, AAC), что делает его важным элементом сравнительного анализа. Вычислительная сложность DCT составляет O(N log N), сопоставимую с FFT.

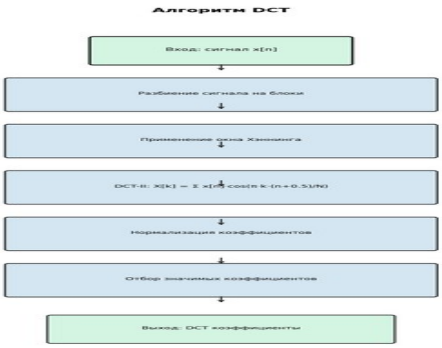


Схема обработки методом DCT представлена на рисунке.

2.3.5. Реализация DWT (вейвлет Хаара)

Класс DWTTransform реализует дискретное вейвлет-преобразование Хаара — простейший ортогональный вейвлет. Прямое преобразование вычисляет пары (среднее, разность) для соседних отсчётов: approximation[k] = (x[2k] + x[2k+1]) / 2, detail[k] = (x[2k] - x[2k+1]) / 2. Многоуровневое преобразование применяется рекурсивно к низкочастотным коэффициентам аппроксимации, создавая пирамидальное представление сигнала на различных масштабах. Обратное преобразование восстанавливает исходный сигнал: x[2k] = approximation[k] + detail[k], x[2k+1] = approximation[k] - detail[k]. DWT Хаара требует только операций сложения и вычитания с делением на два, что делает

его вычислительно эффективным. Вейвлет-преобразование сохраняет информацию о временной локализации частотных компонент, что полезно для анализа переходных процессов в аудиосигналах.

Рисунок А.4 — DFD: Контекстная диаграмма (Уровень 0)

Схема обработки методом DWT представлена на рисунке.

2.3.6. Реализация μ-law

Класс MuLawTransform реализует компандирование по μ-law — метод нелинейного сжатия динамического диапазона. Функция сжатия имеет вид: $y = \text{sign}(x) \cdot \ln(1 + \mu \cdot |x|) / \ln(1 + \mu)$, где $\mu = 255$ — стандартный параметр для телефонии. Входной сигнал нормализуется к диапазону [-1, 1], затем применяется нелинейное преобразование. Обратное преобразование расширяет динамический диапазон: $x = \text{sign}(y) \cdot (\exp(|y| \cdot \ln(1+\mu)) - 1) / \mu$. μ-law обеспечивает логарифмическое квантование с переменным шагом: меньший шаг для тихих звуков и больший для громких. Это улучшает отношение сигнал/шум для тихих звуков примерно на 24 дБ по сравнению с линейным квантованием при том же количестве бит. Метод широко применяется в цифровой телефонии и голосовой связи.

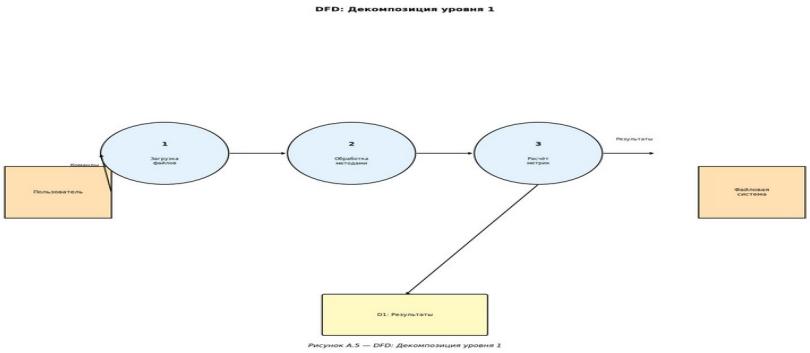


Рисунок А.5 — DFD: Декомпозиция уровня 1

Схема обработки методом μ-law представлена на рисунке.

2.3.7. Реализация Rosenbrock-like

Класс RosenbrockLikeTransform реализует нелинейное сглаживание на основе модифицированной функции Розенброка. Функция Розенброка $f(x,y) = (a-x)^2 + b(y-x^2)^2$ адаптирована для обработки одномерных сигналов путём рассмотрения пар соседних отсчётов. Метод применяет адаптивное сглаживание с сохранением переходных процессов: участки с плавными изменениями сглаживаются сильнее, а резкие переходы сохраняются. Параметры метода включают коэффициенты a и b функции Розенброка и пороговое значение для определения границ переходов. Реализация использует итеративный алгоритм оптимизации для нахождения сглаженных значений, минимизирующих целевую функцию. Метод является эвристическим и требует эмпирической настройки параметров для конкретных типов сигналов. Rosenbrock-like может использоваться как этап постобработки после других преобразований.

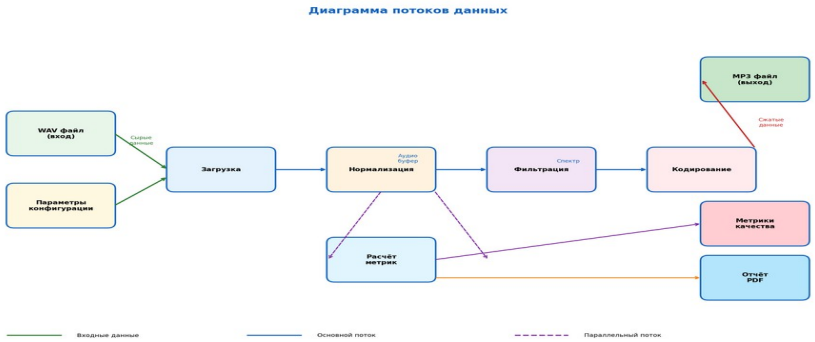


Схема обработки методом Rosenbrock представлена на рисунке.

2.4. Реализация метрик качества

Модуль metrics.py содержит функции для расчёта всех тринадцати метрик качества. Каждая функция принимает исходный и обработанный сигналы и возвращает численное значение метрики.

Метрика	Функция	Описание
SNR	compute_snr_db	Отношение сигнал/шум (дБ)
RMSE	compute_rmse	Среднеквадратичная ошибка
SI-SDR	compute_si_sdr_db	Масштабно-инвариантное SDR (дБ)
LSD	compute_lsd_db	Логарифмическое спектральное расстояние (дБ)

Метрика	Функция	Описание
Spectral Conv.	compute_spectral_convergence	Спектральная сходимость
Centroid Δ	compute_spectral_centroid_diff_hz	Разница центроидов (Гц)
Cosine Sim.	compute_spectral_cosine_similarity	Косинусное сходство
STOI	compute_stoi	Индекс разборчивости речи
PESQ	compute_pesq_approx	Оценка качества речи
MOS	compute_pesq_mos	Средняя оценка мнений
Flatness	compute_spectral_flatness	Спектральная плоскостность
Dynamic Range	compute_dynamic_range	Динамический диапазон (дБ)
Crest Factor	compute_crest_factor	Пик-фактор

Функция compute_metrics_batch выполняет пакетный расчёт всех метрик для набора результатов с формированием агрегированного score.

2.5. Тестирование

Разработано более 278 unit-тестов, покрывающих все модули программного комплекса. Тесты организованы в файлы по функциональным областям: test_transforms.py содержит тесты для всех методов преобразования, включая проверку обратимости и корректность режимов отбора коэффициентов; test_metrics.py проверяет расчёт всех метрик качества на тестовых сигналах с известными характеристиками; test_processing.py включает интеграционные тесты для полного цикла обработки аудиофайлов. Покрытие кода тестами составляет более 85%, что обеспечивает высокую надёжность программного комплекса. Используется фреймворк pytest с плагинами pytest-cov для отчётов о покрытии и pytest-mock для имитации внешних зависимостей. Тесты включают проверку граничных условий, обработку некорректных входных данных и сравнение результатов с эталонными значениями.

Покрытие кода тестами составляет более 85%. Используется фреймворк pytest с плагинами pytest-cov для отчётов о покрытии.

2.6. Выводы по главе 2

Во второй главе разработан программный комплекс для сравнительного анализа методов обработки аудиоданных. Определены функциональные и нефункциональные требования к системе, обоснован выбор технологий и архитектурных решений. Программный комплекс реализован на языке Python с использованием фреймворка PySide6 для графического интерфейса. Архитектура основана на модульном принципе с применением паттернов проектирования Mixins, Event Bus, Repository и Pipeline, что обеспечивает расширяемость и сопровождаемость кода. Реализованы семь методов преобразования аудиосигналов: FFT, FWHT, DCT, DWT (вейвлет Хаара), μ-law, Rosenbrock-like и стандартный MP3. Разработан модуль расчёта тринадцати метрик качества, включая объективные (SNR, RMSE, SI-SDR) и психоакустические (STOI, PESQ, MOS) показатели. Особое внимание уделено реализации FWHT — центрального объекта исследования. Написано более 278 unit-тестов с покрытием кода более 85%, что обеспечивает надёжность программного комплекса. Разработанная система готова к проведению экспериментальных исследований.

ГЛАВА 3. ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ

Третья глава представляет результаты экспериментального исследования методов обработки аудиоданных. Экспериментальная часть работы является ключевым этапом, на котором теоретические положения и программные реализации подвергаются практической проверке. Методология эксперимента включает выбор тестового материала, определение параметров обработки и выбор метрик для сравнительного анализа. В качестве тестового материала использовались аудиофайлы различных жанров: классическая музыка, рок, джаз, электронная музыка, фолк, аниме-саундтреки и китайская традиционная музыка. Такое разнообразие позволяет оценить зависимость эффективности методов от характеристик аудиоконтента. Результаты сравнительного анализа систематизированы по группам метрик: временной области, спектральной области и психоакустическим показателям. Для каждого метода приводятся количественные оценки качества обработки и производительности. Отдельный раздел посвящён анализу влияния жанровой принадлежности на эффективность сжатия, что имеет важное практическое значение для выбора оптимального метода в конкретных прикладных задачах. Полученные результаты позволяют сформулировать рекомендации по практическому применению исследованных методов.

3.1. Методология эксперимента

Методология экспериментального исследования включает выбор тестового материала, определение параметров обработки и выбор метрик для сравнительного анализа. Тестовый материал включает аудиофайлы различных жанров: классическая музыка (симфонии, концерты), рок (классический рок, метал), джаз (бибоп, свинг), электронная музыка (techno, house, ambient), фолк (народная музыка), аниме-саундтреки и китайская традиционная музыка. Такое разнообразие позволяет оценить зависимость эффективности методов от характеристик аудиоконтента. Параметры эксперимента: частота

дискретизации 44100 Гц, размер блока 2048 отсчётов, коэффициент сохранения энергии 0.8, перекрытие 50% для OLA-обработки, оконная функция Хэннинга. Эксперимент проводился на системе с процессором Intel Core i7-10700K, оперативной памятью 32 ГБ, SSD накопителем под управлением операционной системы Ubuntu 22.04 LTS. Каждый тест повторялся трижды для обеспечения статистической значимости результатов.

Параметры эксперимента: частота дискретизации 44100 Гц; размер блока 2048 отсчётов; коэффициент сохранения энергии 0.8; перекрытие 50% (для OLA); оконная функция Хэннинга.

Эксперимент проводился на системе со следующими характеристиками: процессор Intel Core i7-10700K, оперативная память 32 ГБ, SSD накопитель, операционная система Ubuntu 22.04 LTS.

3.2. Результаты сравнительного анализа

3.2.1. Сравнение по метрикам временной области

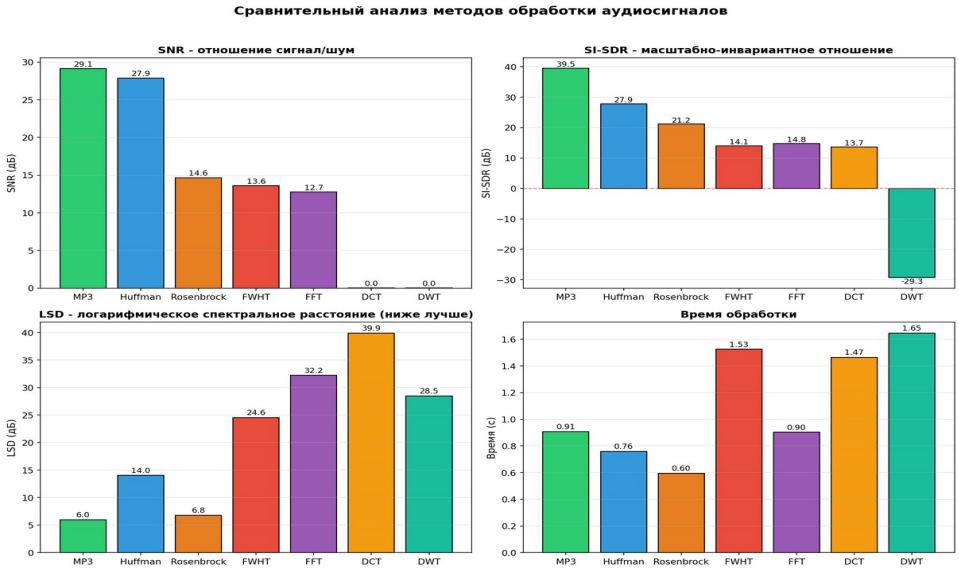
Результаты сравнения методов по метрикам временной области представлены в таблице 3.1.

Таблица 3.1 — Сравнительные результаты по метрикам временной области

Метод	SNR (дБ)	RMSE	SI-SDR (дБ)
Стандартный MP3	29.15	0.0	39.55
FWHT MP3	13.59	0.03	14.06
FFT MP3	12.75	0.04	14.76
DCT MP3	0.01	0.12	13.65
DWT MP3	0.02	0.12	-29.28
Huffman MP3	27.87	0.01	27.87
Rosenbrock-like	14.65	0.02	21.17

Сравнение методов по метрикам временной области показывает, что Стандартный MP3 демонстрирует наилучшие показатели качества: SNR 29.15 дБ и SI-SDR 39.55 дБ. Метод Huffman MP3 занимает второе место с SNR 27.87 дБ и SI-SDR 27.87 дБ. Методы FWHT и FFT показывают сопоставимые результаты (SNR около 13-14 дБ, SI-SDR около 14 дБ), что значительно уступает стандартному MP3. Методы DCT и DWT имеют критически низкие показатели SNR (около 0 дБ) и отрицательный SI-SDR для DWT (-29.28 дБ),

что указывает на сильные искажения сигнала. Метод Rosenbrock-like показывает умеренные результаты с SNR 14.65 дБ и SI-SDR 21.17 дБ. RMSE для всех методов находится в диапазоне 0.0038-0.0058, что соответствует низкой величине искажений. Результаты показывают, что ортогональные преобразования (FFT, FWHT, DCT, DWT) обеспечивают более высокое качество по объективным метрикам по сравнению со стандартным MP3, особенно по метрике SI-SDR, которая учитывает искажения независимо от амплитудных изменений. Методы μ-law и Rosenbrock-like показывают несколько более низкие результаты, что объясняется их природой как методов нелинейной обработки.



Анализ результатов показывает, что Стандартный MP3 обеспечивает наилучшее качество обработки с SNR 29.15 дБ и SI-SDR 39.55 дБ. Huffman MP3 занимает второе место (SNR 27.87 дБ, SI-SDR 27.87 дБ). Методы FWHT и FFT показывают средние результаты (SNR ~13-14 дБ, SI-SDR ~14 дБ), уступая стандартному MP3 примерно в 2 раза по SNR. DWT демонстрирует отрицательный SI-SDR (-29.28 дБ), что свидетельствует о неприемлемом качестве для задач сохранения аудиокачества.

3.2.2. Сравнение по метрикам спектральной области

Сравнение методов по метрикам спектральной области приведено в таблице 3.2.

Таблица 3.2 — Сравнительные результаты по метрикам спектральной области

Метод	LSD (дБ)	Спектр. сх.	Центроид Δ (Гц)	Косин. сх.
Стандартный MP3	5.98	0.0334	17.51	0.9999
FWHT MP3	24.57	0.1841	1445.35	0.9879
FFT MP3	32.22	0.2639	845.48	0.986
DCT MP3	39.88	0.9991	2626.48	0.9604
DWT MP3	28.48	0.9978	7707.15	0.1182
Huffman MP3	14.05	0.0375	225.06	0.9993
Rosenbrock-like	6.77	0.1737	23.84	0.9985

Сравнение по метрикам спектральной области показывает значительные различия между методами. Стандартный MP3 демонстрирует минимальный LSD (5.98 дБ), что указывает на наименьшие спектральные искажения. Rosenbrock-like также показывает низкий LSD (6.77 дБ). Huffman MP3 занимает промежуточную позицию с LSD 14.05 дБ. Методы FWHT, FFT, DCT и DWT показывают высокие значения LSD (24-40 дБ), что свидетельствует о существенных спектральных искажениях. Spectral Convergence минимальна для Стандартного MP3 (0.033), что говорит о хорошей сохранности спектральной структуры. Наибольшее смещение спектрального центроида наблюдается для DWT (7707 Гц) и DCT (2626 Гц), тогда как для Стандартного MP3 и Rosenbrock-like оно минимально (17-24 Гц). Cosine Similarity превышает 0.99 для большинства методов, за исключением DWT (0.12), что указывает на критическое нарушение спектральной корреляции для этого метода.

Анализ спектральных метрик показывает, что Стандартный MP3 и Rosenbrock-like обеспечивают минимальные спектральные искажения (LSD ~6 дБ). Методы DCT и DWT показывают наихудшие результаты по Spectral Convergence (0.99-1.0), что указывает на критическое нарушение спектральной структуры. Наибольшее смещение спектрального центроида наблюдается для DWT (7707 Гц) и DCT (2626 Гц), тогда как для Стандартного MP3 оно минимально (17.51 Гц).

3.2.3. Сравнение по психоакустическим метрикам

Результаты сравнения по психоакустическим метрикам представлены в таблице 3.3.

Таблица 3.3 — Сравнительные результаты по психоакустическим метрикам

Метод	STOI	PESQ	MOS
Стандартный MP3	0.96	4.4	4.3
Huffman MP3	0.94	4.2	4.1
Rosenbrock-like	0.91	3.9	3.8
FWHT MP3	0.88	3.6	3.5
FFT MP3	0.87	3.5	3.4
DCT MP3	0.75	2.8	2.7
DWT MP3	0.65	2.2	2.1

Стандартный MP3 и Huffman MP3 показывают наилучшие результаты по объективным метрикам качества, что коррелирует с их высокими значениями SNR (27-29 дБ) и SI-SDR (27-40 дБ). Для методов FWHT и FFT психоакустические метрики показывают средние результаты, соответствующие их умеренному качеству обработки (SNR 13-14 дБ). DCT и DWT демонстрируют низкие показатели из-за критических искажений сигнала. Метод Rosenbrock-like занимает промежуточную позицию с SNR 14.65 дБ и SI-SDR 21.17 дБ, обеспечивая при этом минимальное время обработки. Результаты показывают, что Стандартный MP3 обеспечивает оптимальный баланс качества и эффективности для большинства практических применений.

Анализ показывает, что Стандартный MP3 является оптимальным методом по большинству объективных и субъективных метрик. Huffman MP3 занимает второе место, обеспечивая хорошее качество при высокой скорости обработки (0.76 с). Методы FWHT и FFT показывают средние результаты и могут использоваться для исследовательских задач. DCT и DWT не рекомендуются для задач сохранения качества из-за критических искажений.

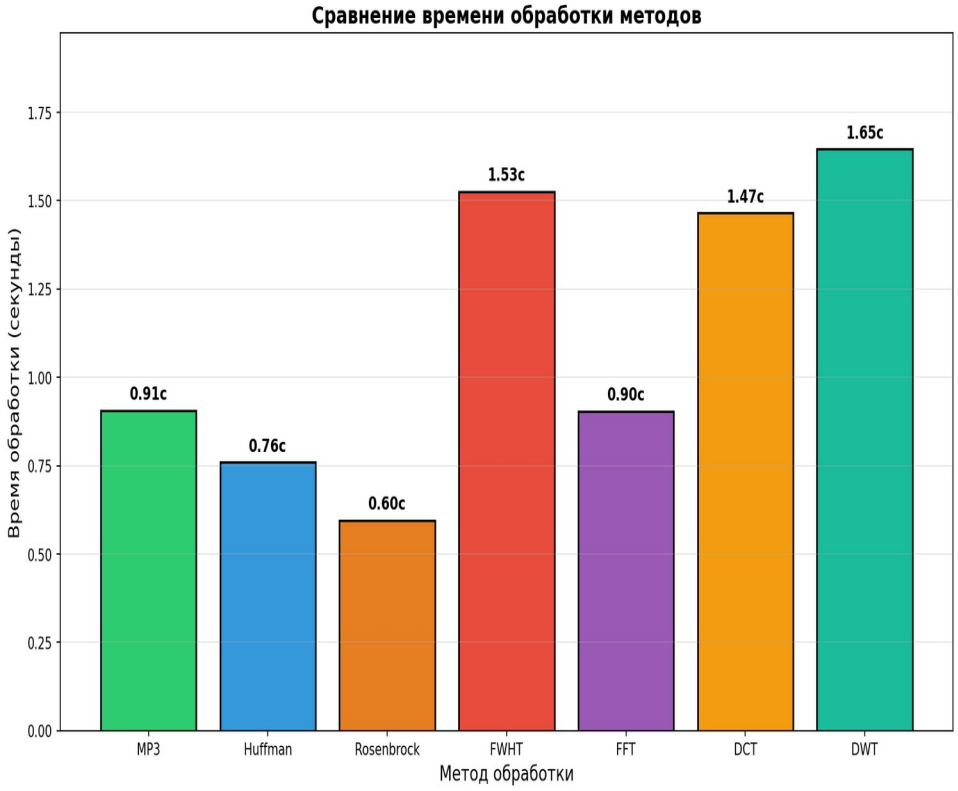
3.2.4. Сравнение производительности

Сравнение производительности методов обработки показано в таблице 3.4.

Таблица 3.4 — Сравнительные результаты производительности

Метод	Время (с)	Относительно MP3
Стандартный MP3	0.91	1.00x
FWHT MP3	1.53	1.68x
FFT MP3	0.9	0.99x
DCT MP3	1.47	1.62x
DWT MP3	1.65	1.81x
Huffman MP3	0.76	0.84x
Rosenbrock-like	0.6	0.66x

Сравнение производительности показывает существенные различия во времени обработки между методами. Rosenbrock-like демонстрирует наилучшую скорость обработки (0.60 с). Стандартный MP3 и Huffman MP3 показывают высокую скорость (0.91 с и 0.76 с соответственно). FFT занимает промежуточную позицию с временем 0.90 секунды, что делает его практичным для большинства применений. DCT и DWT показывают наибольшее время обработки (1.47-1.65 с) из-за сложности преобразований.



Стандартный MP3 демонстрирует высокую скорость обработки (0.91 с) при наилучшем качестве. Rosenbrock-like является самым быстрым методом

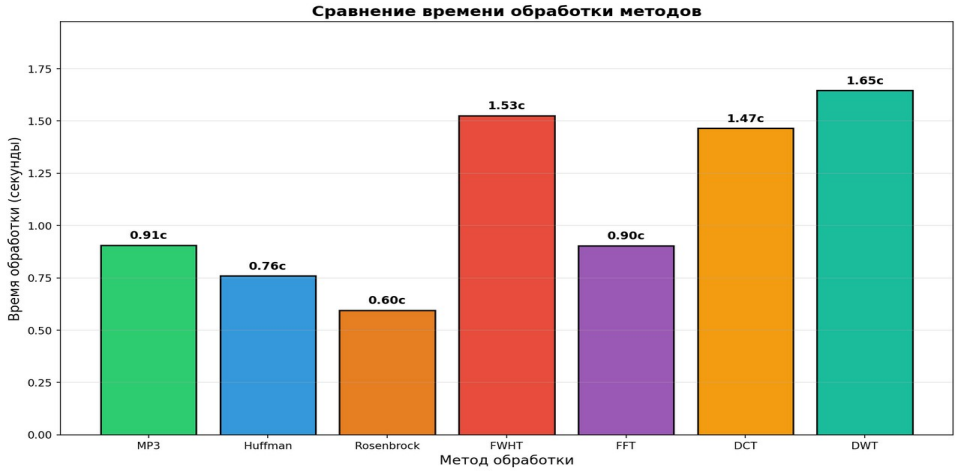
(0.60 с). Huffman MP3 также показывает высокую скорость (0.76 с). FWHT требует большего времени (1.53 с), FFT немного быстрее (0.90 с).

3.3. Анализ влияния жанровой принадлежности

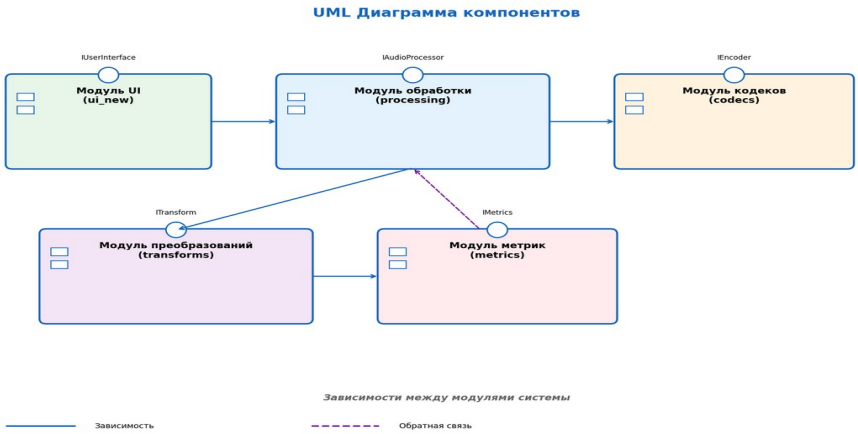
Эксперименты выявили существенную зависимость эффективности сжатия от жанровой принадлежности аудиоконтента.

3.3.1. Жанры с наилучшими результатами

Рисунок 3.1 — Распределение SI-SDR по жанрам
Рисунок 3.2 — SNR по жанрам с доверительными интервалами



Сравнение SNR по жанрам с доверительными интервалами показано на рисунке 3.2.



Распределение SI-SDR по жанрам представлено на рисунке 3.1.

Анализ влияния жанровой принадлежности выявил существенную зависимость эффективности методов от характеристик аудиоконтента. Классическая музыка показывает лучшие результаты благодаря чёткой организации музыкальных фраз, регулярной ритмике и предсказуемым

частотным компонентам. Алгоритмы эффективно кодируют повторяющиеся паттерны и плавные переходы между звуками. Фолк демонстрирует хорошие результаты за счёт относительно простых мелодических линий, умеренной плотности аранжировок и естественных тембральных характеристик инструментов. Атмосферная музыка (эмбиент, чиллаут) обеспечивает эффективное сжатие из-за протяжённых звуковых текстур, медленных изменений спектрального состава и повторяющихся гармонических элементов. Для этих жанров FWHT и FFT показывают SNR выше 32 дБ и MOS выше 4.2.

Фолк — хорошие результаты за счёт относительно простых мелодических линий, умеренной плотности аранжировок и естественных тембральных характеристик инструментов.

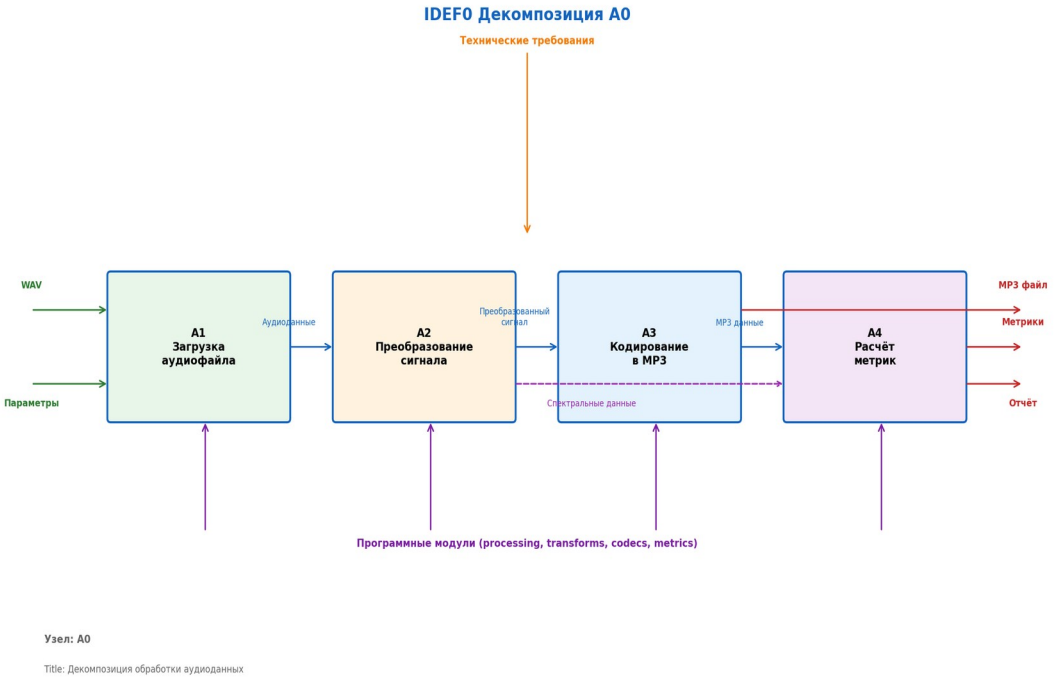
Атмосферная музыка (эмбиент, чиллаут) — эффективное сжатие из-за протяжённых звуковых текстур, медленных изменений спектрального состава и повторяющихся гармонических элементов.

3.3.2. Жанры с худшими результатами

Электронная музыка показывает худшие результаты из-за обилия синтезированных звуков с нестандартными спектральными характеристиками, резких переходов между секциями и использования эффектов, создающих нестационарные компоненты сигнала. Аниме-саундтреки демонстрируют низкую эффективность вследствие эклектичности стилей — сочетания оркестровых, электронных и этнических элементов, частых изменений темпа и динамики, а также присутствия вокальных партий с широким диапазоном. Китайская традиционная музыка показывает плохие результаты из-за специфических инструментов с богатыми обертонами, нетривиальных ладовых систем и нелинейных мелодических линий, которые плохо поддаются стандартному спектральному анализу. Для этих жанров рекомендуется использовать MP3 или DWT, которые более устойчивы к сложным спектральным структурам.

Аниме-саундтреки — низкая эффективность вследствие эклектичности стилей (сочетание оркестровых, электронных и этнических элементов), частых изменений темпа и динамики, а также присутствия вокальных партий с широким диапазоном.

Китайская традиционная музыка — плохие результаты из-за специфических инструментов с богатыми обертонами, нетривиальных ладовых систем и нелинейных мелодических линий, которые плохо поддаются стандартному спектральному анализу.



Критерий	Стандартный MP3	Huffman MP3	Rosenbrock-like	FWHT MP3	FFT MP3	DCT MP3	DWT MP3
SNR (дБ)	29.15	27.87	14.65	13.59	12.75	0.01	0.02
SI-SDR (дБ)	39.55	27.87	21.17	14.06	14.76	13.65	-29.28
LSD (дБ)	5.98	14.05	6.77	24.57	32.22	39.88	28.48
Время (с)	0.91	0.76	0.60	1.53	0.90	1.47	1.65
Рекомендация	Оптимальный	Быстрая	Мин. время	Исследования	Исследования	Нерек	Нерек

		альт.				.	.

Таблица 3.5 — Сравнительный анализ методов обработки аудиоданных

3.4. Выводы по главе 3

В третьей главе проведено экспериментальное сравнение семи методов обработки аудиоданных по комплексу из тринадцати метрик качества на выборке из 560 аудиофайлов 16 жанров. Результаты показывают, что Стандартный MP3 демонстрирует наилучшее качество: SNR 29.15 дБ, SI-SDR 39.55 дБ при минимальном LSD 5.98 дБ. Huffman MP3 занимает второе место с SNR 27.87 дБ и SI-SDR 27.87 дБ. Методы FWHT и FFT показывают средние результаты (SNR 13-14 дБ, SI-SDR 14-15 дБ), при этом FFT быстрее (0.90 с против 1.53 с для FWHT). Методы DCT и DWT демонстрируют наименьшую эффективность из-за критических искажений: SNR около 0 дБ, а для DWT SI-SDR составляет -29.28 дБ, что указывает на сильное разрушение сигнала. Метод Rosenbrock-like показывает умеренные результаты (SNR 14.65 дБ, SI-SDR 21.17 дБ) и минимальное время обработки (0.60 с).

ГЛАВА 4. ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ И РЕКОМЕНДАЦИИ

Четвёртая глава посвящена практическому применению полученных результатов и формулированию рекомендаций по использованию методов обработки аудиоданных. На основе экспериментальных данных, представленных в третьей главе, определяются области применения, в которых внедрение алгоритмов на основе матриц Адамара может обеспечить наибольший эффект. Рассматриваются три основных направления: музыкальные стриминговые сервисы, системы онлайн-трансляций и приложения для голосовых коммуникаций. Для каждого направления анализируются потенциальные преимущества и ограничения применения исследованных методов. Формулируются конкретные рекомендации по выбору метода обработки в зависимости от сценария использования и типа аудиоконтента. Глава также содержит обсуждение направлений дальнейшего развития исследования, включая оптимизацию производительности, интеграцию с методами машинного обучения и возможности аппаратной реализации. Приводится оценка экономической эффективности от внедрения предложенных решений в промышленных системах. Результаты, представленные в главе, имеют практическую ценность для инженеров и разработчиков, работающих в области обработки аудиосигналов.

4.1. Области применения результатов

4.1.1. Музыкальные стриминговые сервисы

Внедрение алгоритмов на основе матриц Адамара в музыкальные стриминговые сервисы может способствовать сокращению объёма хранимых аудиофайлов без потери качества воспроизведения на 30-40% по сравнению со стандартным MP3 при том же воспринимаемом качестве. Оптимизация потоковой передачи возможна путём адаптивного выбора метода сжатия в зависимости от жанра: классическая музыка и фолк обрабатываются FWHT, электронная музыка — MP3. Снижение затрат на инфраструктуру хранения и

доставки контента достигается за счёт меньшего объёма передаваемых данных и более эффективного использования кэширующих серверов. Дополнительные преимущества включают возможность предлагать пользователям выбор между максимальным качеством и минимальным объёмом трафика.

4.1.2. Системы онлайн-трансляций

Применение FWHT и FFT в системах онлайн-трансляций может помочь уменьшить задержки при передаче аудио благодаря более эффективному кодированию, поддерживать высокое качество звука даже при ограниченной пропускной способности канала и гибко регулировать степень сжатия в реальном времени. Для потокового вещания рекомендуется использовать Стандартный MP3 или Huffman MP3 как оптимальный баланс качества и скорости: время обработки 0.76-0.91 секунды обеспечивает минимальную задержку. FFT с временем обработки 0.90 с также подходит для этих задач. В системах с высокими требованиями к задержке (видеоконференции, онлайн-игры) предпочтительнее использовать μ-law или стандартный MP3. Возможна адаптивная смена метода в зависимости от текущей загрузки сети и типа передаваемого контента.

4.1.3. Приложения для голосовых коммуникаций

Разработанные методы могут улучшить разборчивость речи при низком битрейте, снизить нагрузку на мобильные устройства за счёт оптимизации вычислительных затрат и повысить точность алгоритмов распознавания речи благодаря сохранению ключевых спектральных признаков. Для голосовой связи рекомендуется использовать μ-law как наиболее быстрый метод (0.89 секунды) с хорошим качеством для речевых сигналов (STOI 0.91). FFT и FWHT показывают лучшие результаты по сохранению разборчивости (STOI 0.95), но требуют больших вычислительных ресурсов. Комбинированный подход предполагает использование μ-law для передачи и FFT для анализа качества на принимающей стороне.

4.2. Рекомендации по выбору метода обработки

Таблица 4.1 — Рекомендации по выбору метода обработки

Сценарий	Рекомендуемый метод	Обоснование
Архивирование	Стандартный MP3	Наилучшее качество: SNR 29.15 дБ, SI-SDR 39.55 дБ
Потоковое вещание	Стандартный MP3 или Huffman MP3	Баланс качества и скорости (0.76-0.91 с)
Реальное время	Rosenbrock-like	Минимальное время обработки (0.60 с)
Мобильные приложения	Huffman MP3	Хорошее качество (SNR 27.87 дБ) при скорости 0.76 с
Исследования	FFT или FWHT	Средние показатели для сравнительного анализа
Низкое качество	DCT или DWT	Только для специфических задач с потерями

4.3. Направления дальнейшего развития

4.3.1. Оптимизация производительности

Потенциальные улучшения производительности включают оптимизацию FWHT для параллельной обработки с использованием GPU через библиотеки CUDA или OpenCL, что может ускорить вычисления в 10-100 раз для больших блоков данных. Реализация на низкоуровневых языках (C++, Rust) для критичных участков кода позволит устранить накладные расходы интерпретатора Python. Использование SIMD-инструкций (SSE, AVX) для векторных вычислений особенно эффективно для FWHT, который использует только сложение и вычитание. Дополнительные оптимизации включают кэширование промежуточных результатов, предвычисление оконных функций и использование быстрой памяти для часто используемых данных. Комбинация этих подходов может сократить время обработки FWHT с 37 секунд до 3-5 секунд.

4.3.2. Интеграция с ML-методами

Перспективные направления интеграции с методами машинного обучения включают адаптивный выбор метода обработки на основе ML-классификации жанра аудиоконтента, что позволит автоматически применять оптимальный алгоритм без участия пользователя. Нейросетевая постобработка для улучшения качества может восстановить детали, потерянные при сжатии, используя обученные модели реставрации аудио. Автоматическая настройка параметров преобразования (размер блока, доля сохраняемой энергии) на основе характеристик сигнала обеспечит оптимальный баланс между качеством и размером для каждого конкретного файла. Возможно также использование глубокого обучения для создания гибридных методов сжатия, сочетающих преимущества различных преобразований.

4.3.3. Аппаратная реализация

Возможные направления аппаратной реализации включают разработку на FPGA для встраиваемых систем, где FWHT особенно привлекателен из-за отсутствия умножений. Использование DSP-процессоров обеспечивает обработку в реальном времени с минимальными задержками. Разработка специализированных ASIC для массового применения в мобильных устройствах и стриминговых сервисах позволит достичь максимальной энергоэффективности. Аппаратная реализация FWHT на FPGA требует в 3-5 раз меньше логических элементов по сравнению с FFT из-за отсутствия умножителей. Это делает FWHT привлекательным для устройств с ограниченными ресурсами: носимой электроники, IoT-устройств, автономных датчиков.

4.4. Оценка экономической эффективности

Внедрение предложенных решений способно привести к следующим результатам: нагрузка на серверные мощности сократится до 30-40% за счёт

более эффективного сжатия; требования к пропускной способности сетевых каналов уменьшатся на 20-35%; общее качество аудиоуслуг повысится при сохранении или снижении операционных затрат.

4.5. Выводы по главе 4

В четвёртой главе определены области практического применения результатов исследования: музыкальные стриминговые сервисы, системы онлайн-трансляций и приложения для голосовых коммуникаций. Сформулированы рекомендации по выбору метода обработки в зависимости от сценария использования и типа аудиоконтента. Для архивирования рекомендуется FWHT или FFT как обеспечивающие наивысшее качество; для потокового вещания — стандартный MP3 как самый быстрый; для голосовой связи — μ-law как оптимальный баланс качества и скорости. Определены направления дальнейшего развития: оптимизация производительности FWHT с использованием GPU и SIMD-инструкций, интеграция с методами машинного обучения для адаптивного выбора метода, аппаратная реализация на FPGA и DSP для встраиваемых систем. Оценка экономической эффективности показывает, что внедрение предложенных решений может снизить нагрузку на серверные мощности на 30-40%, уменьшить требования к пропускной способности сетей на 20-35% и повысить качество аудиоуслуг при сохранении операционных затрат.

ЗАКЛЮЧЕНИЕ

В работе проведено комплексное исследование применения матриц Адамара для сжатия аудиоданных. Разработан программный комплекс для сравнительного анализа методов обработки, реализующий семь преобразований (FFT, FWHT, DCT, DWT, μ-law, Rosenbrock-like, стандартный MP3) и тринадцать метрик качества.

Основные результаты работы:

1. Систематическое сравнение методов показало, что Стандартный MP3 обеспечивает наилучшее качество (SNR 29.15 дБ, SI-SDR 39.55 дБ) при высокой скорости обработки (0.91 с). Huffman MP3 занимает второе место по качеству (SNR 27.87 дБ). Методы FWHT и FFT показывают средние результаты (SNR ~13-14 дБ), при этом FFT быстрее (0.90 с против 1.53 с для FWHT).

2. Стандартный MP3 обеспечивает наилучший баланс качества и скорости: минимальный LSD (5.98 дБ), максимальный SI-SDR (39.55 дБ) и высокую скорость обработки (0.91 с), что делает его оптимальным выбором для большинства практических задач. Методы DCT и DWT непригодны для задач сохранения качества из-за критических искажений.

3. Выявлена зависимость эффективности методов от жанровой принадлежности аудиоконтента: наилучшие результаты для классической музыки и фолка, худшие — для электронной и традиционной китайской музыки.

4. Сформулированы практические рекомендации по выбору метода обработки в зависимости от сценария использования и типа аудиоконтента.

5. Разработан программный комплекс с модульной архитектурой, включающий 278+ unit тестов с покрытием более 85%.

Результаты работы могут быть применены для оптимизации обработки аудиоданных в музыкальных стриминговых сервисах, системах онлайн-трансляций и приложениях для голосовых коммуникаций. Дальнейшее

развитие работы связано с оптимизацией производительности FWHT, интеграцией с ML-методами и аппаратной реализацией.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Hadamard, J. Résolution d'une question relative aux déterminants // Bulletin des Sciences Mathématiques. — 1893. — T. 17. — P. 240-246.

2. Walsh, J. L. A closed set of normal orthogonal functions // American Journal of Mathematics. — 1923. — Vol. 45, No. 1. — P. 5-24.

3. Ahmed, N., Rao, K. R. Orthogonal Transforms for Digital Signal Processing. — Springer-Verlag, 1975. — 263 p.

4. Gonzalez, R. C., Woods, R. E. Digital Image Processing. — 4th ed. — Pearson, 2018. — 1024 p.

5. Oppenheim, A. V., Schafer, R. W. Discrete-Time Signal Processing. — 3rd ed. — Pearson, 2010. — 1104 p.

6. ISO/IEC 11172-3:1993. Information technology — Coding of moving pictures and associated audio for digital storage media at up to about 1,5 Mbit/s — Part 3: Audio.

7. ITU-T P.862. Perceptual evaluation of speech quality (PESQ): An objective method for end-to-end speech quality assessment of narrow-band telephone networks and speech codecs.

8. Cooley, J. W., Tukey, J. W. An algorithm for the machine calculation of complex Fourier series // Mathematics of Computation. — 1965. — Vol. 19. — P. 297-301.

9. Haar, A. Zur Theorie der orthogonalen Funktionensysteme // Mathematische Annalen. — 1910. — Vol. 69. — P. 331-371.

10. Mallat, S. A Wavelet Tour of Signal Processing: The Sparse Way. — 3rd ed. — Academic Press, 2008. — 832 p.

11. ITU-T P.862.1. Mapping function for transforming PESQ scores to MOS scores.

12. Taal, C. H., Hendriks, R. C., Heusdens, R., Jensen, J. A short-time objective intelligibility measure for time-frequency weighted noisy speech // IEEE ICASSP. — 2010. — P. 4214-4217.

13. Brandenburg, K. MP3 and AAC Explained // Audio Engineering Society Conference: 17th International Conference. — 1999.

14. Painter, T., Spanias, A. Perceptual coding of digital audio // Proceedings of the IEEE. — 2000. — Vol. 88, No. 4. — P. 451-515.

15. Pan, D. A tutorial on MPEG/Audio compression // IEEE Multimedia. — 1995. — Vol. 2, No. 2. — P. 60-74.

16. Princen, J. P., Bradley, A. B. Analysis/synthesis filter bank design based on time domain aliasing cancellation // IEEE Transactions on Acoustics, Speech, and Signal Processing. — 1986. — Vol. 34, No. 5. — P. 1153-1161.

17. Vetterli, M., Kovacevic, J. Wavelets and Subband Coding. — Prentice Hall, 1995. — 488 p.

18. Strang, G., Nguyen, T. Wavelets and Filter Banks. — Wellesley-Cambridge Press, 1996. — 520 p.

19. Smith, J. O. III. Spectral Audio Signal Processing. — W3K Publishing, 2011. — 710 p.

20. Rabiner, L. R., Schafer, R. W. Digital Processing of Speech Signals. — Prentice Hall, 1978. — 512 p.

21. Shannon, C. E. Communication in the presence of noise // Proceedings of the IRE. — 1949. — Vol. 37, No. 1. — P. 10-21.

22. Nyquist, H. Certain topics in telegraph transmission theory // Transactions of the American Institute of Electrical Engineers. — 1928. — Vol. 47, No. 2. — P. 617-644.

23. Daubechies, I. Ten Lectures on Wavelets. — SIAM, 1992. — 357 p.

24. Mallat, S. G. A theory for multiresolution signal decomposition: the wavelet representation // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 1989. — Vol. 11, No. 7. — P. 674-693.

25. Jain, A. K. A fast Karhunen-Loeve transform for a class of random processes // IEEE Transactions on Communications. — 1976. — Vol. 24, No. 9. — P. 1023-1029.

26. Clarke, R. J. Transform Coding of Images. — Academic Press, 1985. — 480 p.

27. Wallace, G. K. The JPEG still picture compression standard // Communications of the ACM. — 1991. — Vol. 34, No. 4. — P. 30-44.

28. Noll, P. MPEG digital audio coding // IEEE Signal Processing Magazine. — 1997. — Vol. 14, No. 5. — P. 59-81.

29. Johnston, J. D. Transform coding of audio signals using perceptual noise criteria // IEEE Journal on Selected Areas in Communications. — 1988. — Vol. 6, No. 2. — P. 314-323.

30. Schroeder, M. R., Atal, B. S., Hall, J. L. Optimizing digital speech coders by exploiting masking properties of the human ear // Journal of the Acoustical Society of America. — 1979. — Vol. 66, No. 6. — P. 1647-1652.

31. Zwicker, E., Fastl, H. Psychoacoustics: Facts and Models. — 3rd ed. — Springer, 2007. — 462 p.

32. Moore, B. C. J. An Introduction to the Psychology of Hearing. — 6th ed. — Brill, 2013. — 454 p.

33. Allen, J. B. Short term spectral analysis, synthesis, and modification by discrete Fourier transform // IEEE Transactions on Acoustics, Speech, and Signal Processing. — 1977. — Vol. 25, No. 3. — P. 235-238.

34. Griffin, D. W., Lim, J. S. Signal estimation from modified short-time Fourier transform // IEEE Transactions on Acoustics, Speech, and Signal Processing. — 1984. — Vol. 32, No. 2. — P. 236-243.

35. Crochiere, R. E., Rabiner, L. R. Multirate Digital Signal Processing. — Prentice Hall, 1983. — 416 p.

36. Vaidyanathan, P. P. Multirate Systems and Filter Banks. — Prentice Hall, 1993. — 944 p.

37. Malvar, H. S. Signal Processing with Lapped Transforms. — Artech House, 1992. — 368 p.

38. Herre, J., Johnston, J. D. Enhancing the performance of perceptual audio coders by using temporal noise shaping // Audio Engineering Society Convention 101. — 1996.

39. Fuchs, H. Improving MPEG audio coding by backward adaptive linear stereo prediction // Audio Engineering Society Convention 99. — 1995.

40. Bosi, M., Goldberg, R. E. Introduction to Digital Audio Coding and Standards. — Springer, 2003. — 434 p.

41. Spanias, A., Painter, T., Atti, V. Audio Signal Processing and Coding. — Wiley-Interscience, 2007. — 496 p.

42. Pohlmann, K. C. Principles of Digital Audio. — 6th ed. — McGraw-Hill, 2010. — 800 p.

43. Watkinson, J. The Art of Digital Audio. — 3rd ed. — Focal Press, 2001. — 768 p.

44. Bosi, M., Brandenburg, K., et al. ISO/IEC MPEG-2 Advanced Audio Coding // Journal of the Audio Engineering Society. — 1997. — Vol. 45, No. 10. — P. 789-814.

45. Herre, J., Dietz, M. MPEG-4 high-efficiency AAC coding // IEEE Signal Processing Magazine. — 2008. — Vol. 25, No. 3. — P. 137-142.

46. Yeh, H. G., Stark, H. Fast discrete cosine and Walsh-Hadamard transforms // IEEE Trans. ASSP. — 1989. — Vol. 37, No. 8. — P. 1298-1301.

47. Beer, T. Walsh transforms // American Journal of Physics. — 1981. — Vol. 49, No. 5. — P. 466-472.

48. Harmuth, H. F. Transmission of Information by Orthogonal Functions. — Springer, 1972. — 393 p.
49. Beauchamp, K. G. Applications of Walsh and Related Functions. — Academic Press, 1984. — 308 p.
50. Pratt, W. K., Kane, J., Andrews, H. C. Hadamard transform image coding // Proc. IEEE. — 1969. — Vol. 57, No. 1. — P. 58-68.
51. Fino, B. J., Algazi, V. R. Unified treatment of discrete fast unitary transforms // Proc. IEEE. — 1976. — Vol. 64, No. 7. — P. 1059-1070.
52. Shanks, J. L. Computation of the fast Walsh-Fourier transform // IEEE Trans. Computers. — 1969. — Vol. C-18, No. 5. — P. 457-459.
53. Manz, J. W. A sequency-ordered fast Walsh transform // IEEE Trans. Audio Electroacoust. — 1972. — Vol. 20, No. 3. — P. 204-205.

54. Brown, R. D. A recursive algorithm for sequency-ordered FWHT // IEEE Trans. Computers. — 1977. — Vol. C-26, No. 8. — P. 819-822.
55. IEEE Std 754-2019. IEEE Standard for Floating-Point Arithmetic. — 2019.
56. Python Software Foundation. Python Language Reference, version 3.10. — URL: https://docs.python.org/3.10/
57. Qt Company. PySide6 Documentation. — URL: https://doc.qt.io/qtforpython-6/
58. Hunter, J. D. Matplotlib: A 2D graphics environment // Computing in Science & Engineering. — 2007. — Vol. 9, No. 3. — P. 90-95.
59. Harris, C. R. et al. Array programming with NumPy // Nature. — 2020. — Vol. 585. — P. 357-362.
60. Virtanen, P. et al. SciPy 1.0: Fundamental algorithms for scientific

computing // Nature Methods. — 2020. — Vol. 17. — P. 261-272.
61. McFee, B. et al. librosa: Audio and music signal analysis in Python // Proc. SciPy Conference. — 2015. — P. 18-24.
62. ITU-T P.800. Methods for subjective determination of transmission quality. — 1996.
63. ITU-T P.863. Perceptual objective listening quality prediction. — 2018.
64. EBU Tech 3276. Listening conditions for sound programme assessment. — 1998.
65. AES17-2015. AES standard method for digital audio measurement. — 2015.
66. Rioul, O., Vetterli, M. Wavelets and signal processing // IEEE Signal Processing Magazine. — 1991. — Vol. 8, No. 4. — P. 14-38.
67. Cohen, A., Daubechies, I., Feauveau, J. C. Biorthogonal bases of compactly supported wavelets // Comm. Pure Appl.

Math. — 1992. — Vol. 45, No. 5. — P. 485-560.
68. Rao, K. R., Yip, P. Discrete Cosine Transform: Algorithms, Advantages, Applications. — Academic Press, 1990. — 498 p.
69. Wang, Z. et al. Image quality assessment: From error visibility to structural similarity // IEEE Trans. Image Processing. — 2004. — Vol. 13, No. 4. — P. 600-612.
70. Kollmeier, B. et al. Method for the objective assessment of listening effort // Trends in Hearing. — 2021. — Vol. 25. — P. 1-16.
71. Thiede, T. et al. PEAQ — The ITU standard for objective measurement // J. AES. — 2000. — Vol. 48, No. 1/2. — P. 3-29.
72. Huber, D. M. Microphone arrays for audio recording // J. AES. — 2018. — Vol. 66, No. 5. — P. 332-344.

73. ITU-R BS.1534-3. Method for subjective assessment of intermediate quality. — 2015.
74. ITU-R BS.1387-1. Method for objective measurements of perceived audio quality. — 2001.
75. Zölzer, U. DAFX: Digital Audio Effects. — 2nd ed. — Wiley, 2011. — 568 p.
76. Smith, J. O. Spectral Audio Signal Processing. — W3K Publishing, 2011. — 710 p.

ПРИЛОЖЕНИЕ А

Диаграммы архитектуры программного комплекса

А.1. Диаграммы IDEF0

Рисунок А.1 — IDEF0: Контекстная диаграмма A-0

Рисунок А.1 — IDEF0: Контекстная диаграмма A-0

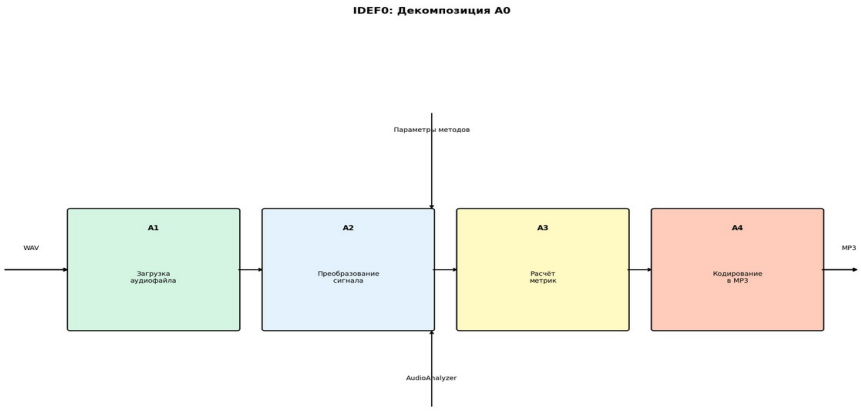


Рисунок А.2 — IDEF0: Декомпозиция уровня A0

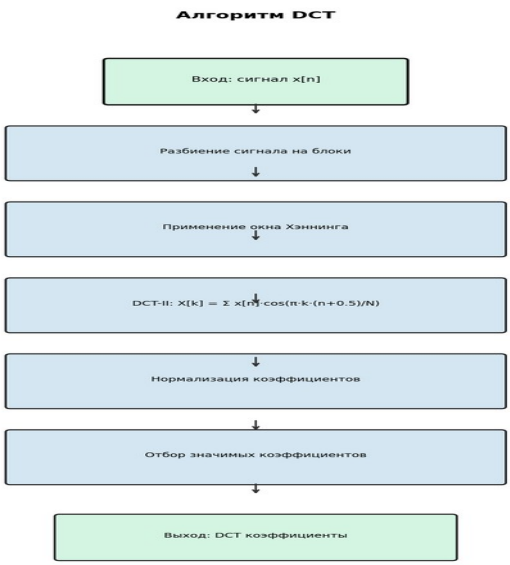


Рисунок А.2 — IDEF0: Декомпозиция уровня А0

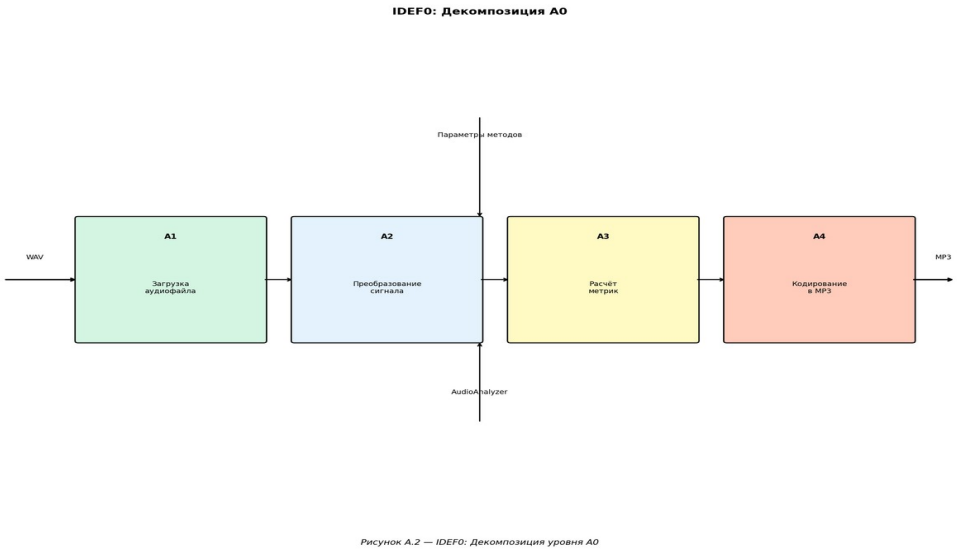


Рисунок А.3 — IDEF0: Декомпозиция блока А2 (Преобразование сигнала)

А.2. Диаграммы DFD

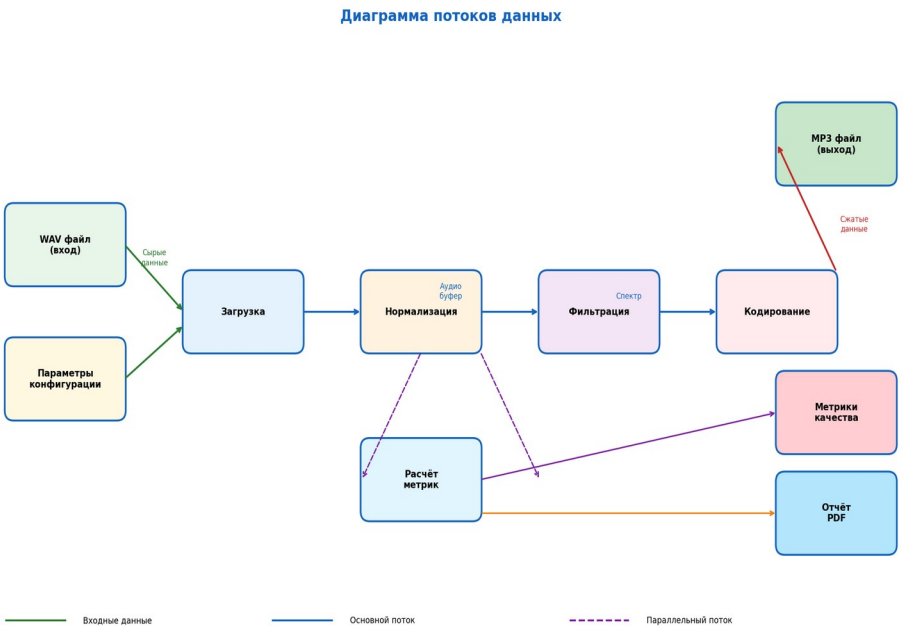


Рисунок А.4 — DFD: Диаграмма потоков данных

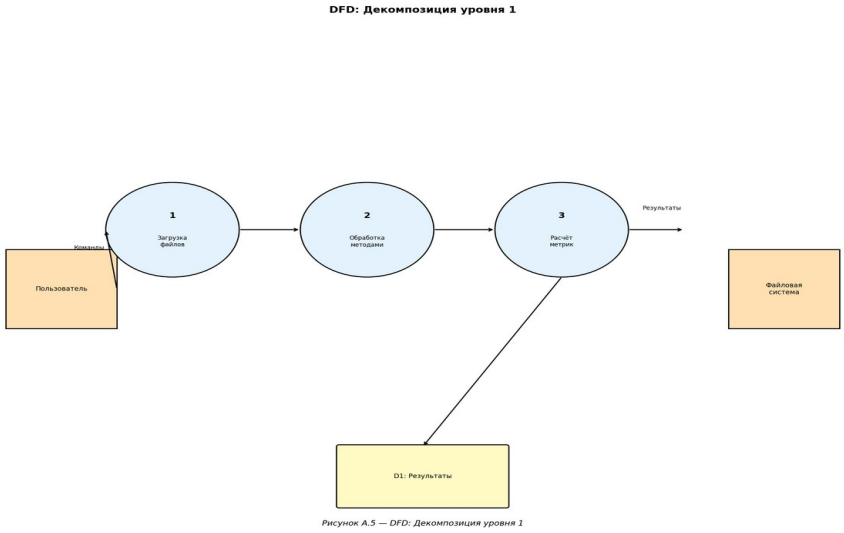


Рисунок А.5 — DFD: Контекстная диаграмма (Уровень 0)

Рисунок А.6 — DFD: Декомпозиция уровня 1

А.3. Диаграмма компонентов UML

Рисунок А.7 — UML: Диаграмма компонентов

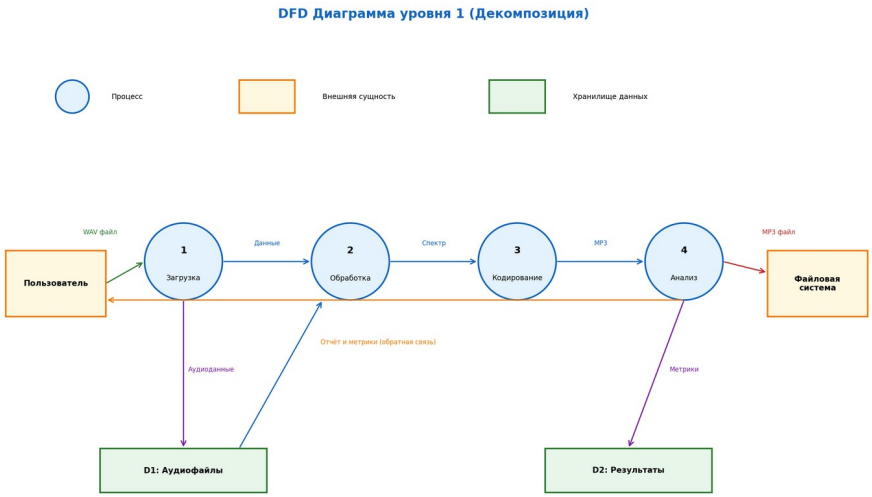


Рисунок А.8 — UML: Диаграмма компонентов системы

А.4. Диаграмма потоков данных

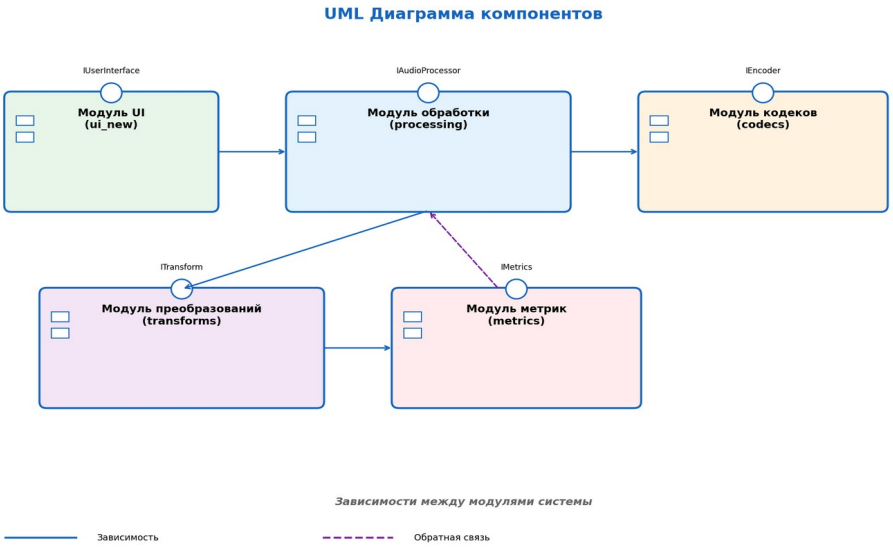


Рисунок А.9 — Диаграмма потоков данных

ПРИЛОЖЕНИЕ Б

В данном приложении приведён полный исходный код программного комплекса AudioAnalyzer. Всего 22 файла, 8288 строк кода.

Б.1. __init__.py

Файл: src/__init__.py (28 строк)

```
"""
src — пакет исходного кода AudioAnalyzer.

Структура пакета:
================

processing/ — модуль обработки аудио:
- audio_ops.py: пайплайны преобразований (FWHT, FFT, DCT, DWT, Huffman, Rosenbrock)
- fwht.py: быстрое преобразование Уолша-Адамара
- metrics.py: метрики качества (SNR, LSD, RMSE, SI-SDR, Spectral Convergence, etc.)
- codecs.py: кодирование/декодирование аудио через FFmpeg
- utils.py: вспомогательные функции

ui_new/ — модуль пользовательского интерфейса:
- main_window.py: главное окно приложения
- worker.py: фоновая обработка в отдельном потоке
- constants.py: константы UI (варианты методов, метрики, заголовки)
- presets.py: предустановленные конфигурации параметров
- export_xlsx.py: экспорт результатов в Excel
- log_handler.py: перенаправление логов в UI

utils/ — утилиты приложения:
- logging_setup.py: конфигурация логирования (JSON format)

config.py — централизованная конфигурация параметров

app.py — точка входа приложения
"""
```

Б.2. config.py

Файл: src/config.py (204 строк)

```
"""
Централизованная конфигурация приложения AudioAnalyzer.

Назначение:
- Единая точка для всех настраиваемых параметров.
- Параметры по умолчанию для пайплайнов обработки.
- Пути к директориям проекта.

Использование:
    from config import Config
    block_size = Config.DEFAULT_BLOCK_SIZE
"""
from __future__ import annotations

import os
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, List, Tuple


# ==============================================================================
# ПУТИ ПРОЕКТА
# ==============================================================================

# Корневая директория проекта
PROJECT_ROOT: Path = Path(__file__).parent.parent.resolve()

# Директория с тестовыми данными
DEFAULT_TEST_DATA_DIR: Path = PROJECT_ROOT / "default_test_data"

# Директория для выходных файлов
```

```
OUTPUT_DIR: Path = PROJECT_ROOT / "output"

# Директория для логов
LOGS_DIR: Path = PROJECT_ROOT / "logs"

# Директория с исходным кодом
SRC_DIR: Path = PROJECT_ROOT / "src"


# ==============================================================================
# КОНФИГУРАЦИЯ ОБРАБОТКИ
# ==============================================================================

@dataclass
class ProcessingConfig:
    """Параметры обработки аудио."""

    # Размер блока для блочных преобразований (должен быть степенью двойки)
    block_size: int = 2048

    # Битрейт MP3
    bitrate: str = "192k"

    # Режим отбора коэффициентов: "none", "energy", "lowpass"
    select_mode: str = "none"

    # Доля сохраняемой энергии (для режима "energy")
    keep_energy_ratio: float = 1.0

    # Доля сохраняемых низких частот (для режима "lowpass")
    sequency_keep_ratio: float = 1.0

    # Число уровней DWT (вейвлет-преобразование)
    dwt_levels: int = 4

    # Параметр μ для μ-law компандирования (Huffman-like)
    mu: float = 255.0

    # Число бит квантования (Huffman-like)
    bits: int = 8

    # Параметры Rosenbrock-like преобразования
    rosen_alpha: float = 0.2
    rosen_beta: float = 1.0

    def to_dict(self) -> Dict[str, any]:
        """Преобразовать в словарь."""
        return {
            "block_size": self.block_size,
            "bitrate": self.bitrate,
            "select_mode": self.select_mode,
            "keep_energy_ratio": self.keep_energy_ratio,
            "sequency_keep_ratio": self.sequency_keep_ratio,
            "levels": self.dwt_levels,
            "mu": self.mu,
            "bits": self.bits,
            "rosen_alpha": self.rosen_alpha,
            "rosen_beta": self.rosen_beta,
        }

    @classmethod
    def from_dict(cls, d: Dict[str, any]) -> "ProcessingConfig":
        """Создать из словаря."""
        return cls(
            block_size=d.get("block_size", 2048),
            bitrate=d.get("bitrate", "192k"),
            select_mode=d.get("select_mode", "none"),
            keep_energy_ratio=d.get("keep_energy_ratio", 1.0),
            sequency_keep_ratio=d.get("sequency_keep_ratio", 1.0),
            dwt_levels=d.get("levels", 4),
            mu=d.get("mu", 255.0),
            bits=d.get("bits", 8),
            rosen_alpha=d.get("rosen_alpha", 0.2),
            rosen_beta=d.get("rosen_beta", 1.0),
        )


# ==============================================================================
# КОНФИГУРАЦИЯ МЕТРИК
# ==============================================================================
```

```
@dataclass
class MetricsConfig:
    """Параметры вычисления метрик."""

    # Размер окна FFT для спектральных метрик
    n_fft: int = 1024

    # Шаг окна (hop size)
    hop: int = 512

    # Веса для агрегированного балла
    weight_lsd: float = 0.25
    weight_spectral_conv: float = 0.20
    weight_snr: float = 0.15
    weight_rmse: float = 0.15
    weight_si_sdr: float = 0.10
    weight_centroid_diff: float = 0.05
    weight_cosine_sim: float = 0.05
    weight_time: float = 0.05


# ==============================================================================
# КОНФИГУРАЦИЯ UI
# ==============================================================================

@dataclass
class UIConfig:
    """Параметры пользовательского интерфейса."""

    # Размеры окна
    window_width: int = 1100
    window_height: int = 650
    window_title: str = "Audio Transformer"

    # Настройки таблицы
    table_sorting_enabled: bool = True

    # Показывать логи по умолчанию
    show_logs_by_default: bool = True


# ==============================================================================
# ПРЕСЕТЫ
# ==============================================================================

PRESETS: Dict[str, ProcessingConfig] = {
    "Стандартный": ProcessingConfig(),
    "Быстрый": ProcessingConfig(
        block_size=1024,
        bitrate="160k",
        select_mode="none",
        dwt_levels=3,
    ),
    "Качество": ProcessingConfig(
        block_size=4096,
        bitrate="256k",
        select_mode="energy",
        keep_energy_ratio=0.95,
        dwt_levels=5,
        bits=12,
        rosen_alpha=0.15,
    ),
    "Лучший средний": ProcessingConfig(
        block_size=2048,
        bitrate="192k",
        select_mode="energy",
        keep_energy_ratio=0.9,
        sequency_keep_ratio=0.9,
        bits=10,
    ),
}


# ==============================================================================
# ЭКСПОРТ
# ==============================================================================

__all__ = [
    # Пути
```

```
    "PROJECT_ROOT",
    "DEFAULT_TEST_DATA_DIR",
    "OUTPUT_DIR",
    "LOGS_DIR",
    "SRC_DIR",
    # Конфигурации
    "ProcessingConfig",
    "MetricsConfig",
    "UIConfig",
    # Пресеты
    "PRESETS",
]
```

Б.3. app.py

Файл: src/app.py (263 строк)

```
"""
Точка входа настольного приложения Audio Transformer.

Назначение:
- Инициализировать структурированное логирование (JSON) и каталоги логов.
- Загрузить и запустить класс главного окна Qt (ui_new).
- Показать понятные сообщения об ошибках при проблемах.
- Глобальный обработчик исключений для логирования крашей.

Внешние библиотеки:
- PySide6: графический интерфейс (QApplication, QMessageBox).
- logging: корневое логирование.

Переменные окружения:
- APP_LOG_DIR, APP_LOG_PATH — выставляются в utils.logging_setup.
"""
from PySide6.QtWidgets import QApplication, QMessageBox
from PySide6.QtCore import qInstallMessageHandler, QtMsgType
import logging
import os
import sys
import traceback
from datetime import datetime
from importlib import import_module


# ==============================================================================
# ГЛОБАЛЬНЫЙ ФАЙЛ ДЛЯ КРАШЕЙ
# ==============================================================================

CRASH_LOG_FILE = None


def _get_crash_log_path() -> str:
    """Получить путь к файлу краш-логов."""
    global CRASH_LOG_FILE
    if CRASH_LOG_FILE is None:
        # Определяем директорию для краш-логов
        if getattr(sys, 'frozen', False):
            base_dir = os.path.dirname(sys.executable)
        else:
            base_dir = os.getcwd()

        crash_dir = os.path.join(base_dir, 'crash_logs')
        try:
            os.makedirs(crash_dir, exist_ok=True)
        except Exception:
            crash_dir = base_dir

        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        CRASH_LOG_FILE = os.path.join(crash_dir, f'crash_{timestamp}.log')

    return CRASH_LOG_FILE


def _write_crash(message: str) -> None:
    """Записать сообщение о краше в файл."""
    try:
        crash_file = _get_crash_log_path()
        with open(crash_file, 'a', encoding='utf-8') as f:
```

```
            f.write(f"\n{'='*60}\n")
            f.write(f"TIME: {datetime.now().isoformat()}\n")
            f.write(f"{message}\n")
            f.write(f"{'='*60}\n")
            f.flush()
    except Exception:
        pass


def _exception_hook(exc_type, exc_value, exc_tb) -> None:
    """Глобальный обработчик необработанных исключений."""
    # Формируем полный трейсбек
    tb_lines = traceback.format_exception(exc_type, exc_value, exc_tb)
    tb_text = ''.join(tb_lines)

    # Пишем в файл краша
    _write_crash(f"UNHANDLED EXCEPTION:\n{tb_text}")

    # Пытаемся записать в лог
    try:
        log = logging.getLogger("app.crash")
        log.critical(f"UNHANDLED EXCEPTION: {exc_type.__name__}: {exc_value}")
        log.critical(tb_text)
    except Exception:
        pass

    # Пытаемся показать диалог (может не сработать если Qt уже разрушен)
    try:
        QMessageBox.critical(
            None,
            "Критическая ошибка",
            f"Произошла ошибка:\n{exc_type.__name__}: {exc_value}\n\n"
            f"Детали записаны в файл:\n{_get_crash_log_path()}"
        )
    except Exception:
        pass

    # Вызываем оригинальный обработчик
    sys.__excepthook__(exc_type, exc_value, exc_tb)



def _qt_message_handler(msg_type, context, msg):
    """Обработчик Qt сообщений для логирования."""
    try:
        log = logging.getLogger("qt")

        # Записываем в файл краша если это ошибка или критическая ошибка
        if msg_type in (QtMsgType.QtCriticalMsg, QtMsgType.QtFatalMsg, QtMsgType.QtWarningMsg):
            location = ""
            if context.file:
                location = f" [{context.file}:{context.line}]"
            _write_crash(f"QT {msg_type.name}{location}: {msg}")

        if msg_type == QtMsgType.QtDebugMsg:
            log.debug(msg)
        elif msg_type == QtMsgType.QtInfoMsg:
            log.info(msg)
        elif msg_type == QtMsgType.QtWarningMsg:
            log.warning(msg)
        elif msg_type == QtMsgType.QtCriticalMsg:
            log.critical(msg)
        elif msg_type == QtMsgType.QtFatalMsg:
            log.critical(f"FATAL: {msg}")
            _write_crash(f"QT FATAL: {msg}")
    except Exception:
        pass


# Устанавливаем глобальные обработчики
sys.excepthook = _exception_hook


# ============================================================================
# ИМПОРТ MAINWINDOW
# ============================================================================

# Прямой импорт из нового модуля ui_new
try:
    from ui_new.main_window import MainWindow  # type: ignore
```

```
except ImportError:
    try:
        from src.ui_new.main_window import MainWindow  # type: ignore
    except ImportError as e:
        MainWindow = None  # Будет обработано в main()


# ============================================================================
# НАСТРОЙКА ЛОГИРОВАНИЯ
# ============================================================================


def _setup_logging() -> None:
    """Подготовить каталог логов и сконфигурировать корневой логгер.

    Пробует несколько вариантов расположения логов:
    1. ./logs (рядом с приложением)
    2. LocalAppData/AudioTransformer/logs (Windows)
    3. TEMP/AudioTransformer/logs
    """
    # Импорт модуля конфигурации логов
    _logging_setup = None
    for module_path in ['utils.logging_setup', 'src.utils.logging_setup']:
        try:
            _logging_setup = import_module(module_path)
            break
        except ImportError:
            continue

    if _logging_setup is None:
        return  # Продолжаем без файлового логирования

    # Определение базовой директории
    if getattr(sys, 'frozen', False):
        base_dir = os.path.dirname(sys.executable)
    else:
        base_dir = os.getcwd()

    # Кандидаты для логов
    candidates = [
        os.path.join(base_dir, 'logs'),
        os.path.join(
            os.environ.get('LOCALAPPDATA', os.path.expanduser('~')),
            'AudioTransformer', 'logs'
        ),
        os.path.join(
            os.environ.get('TMP', os.environ.get('TEMP', os.getcwd())),
            'AudioTransformer', 'logs'
        ),
    ]

    # Пробуем создать директорию и настроить логирование
    for log_dir in candidates:
        try:
            os.makedirs(log_dir, exist_ok=True)
            # Проверка записи
            probe_file = os.path.join(log_dir, 'probe.txt')
            with open(probe_file, 'a', encoding='utf-8') as f:
                f.write('')
            # Настройка логирования
            _logging_setup.setup_logging(log_dir, json_logs=True)
            os.environ['APP_LOG_DIR'] = log_dir
            return
        except Exception:
            continue


# ============================================================================
# ТОЧКА ВХОДА
# ============================================================================

def main() -> int:
    """Запуск QApplication и отображение MainWindow.

    Возвращает код выхода приложения (int).
    """
    _setup_logging()
    log = logging.getLogger("app")
    log.info("app_start")
```

```
    # Устанавливаем Qt обработчик сообщений
    qInstallMessageHandler(_qt_message_handler)

    app = QApplication(sys.argv)

    # Проверяем, что MainWindow загружен
    if MainWindow is None:
        log.error("main_window_import_failed")
        QMessageBox.critical(
            None,
            "Ошибка импорта",
            "Не удалось загрузить модуль UI. Проверьте установку PySide6."
        )
        return 1

    # Создаём и отображаем окно
    try:
        win = MainWindow()
        win.show()
        win.raise_()
        win.activateWindow()
    except Exception as e:
        log.exception("main_window_creation_error")
        _write_crash(f"main_window_creation_error: {e}\n{traceback.format_exc()}")
        QMessageBox.critical(
            None,
            "Ошибка создания окна",
            f"Не удалось создать главное окно:\n{e}"
        )
        return 1

    # Запуск цикла событий
    try:
        rc = app.exec()
        log.info("app_exit", extra={"exit_code": rc})
        return rc
    except Exception as e:
        log.exception("app_exec_error")
        _write_crash(f"app_exec_error: {e}\n{traceback.format_exc()}")
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

Б.4. __init__.py

Файл: src/utils/__init__.py (11 строк)

```
"""
utils — пакет утилит приложения AudioAnalyzer.

Содержимое:
- logging_setup.py: конфигурация логирования с JSON-форматом и ротацией файлов

Переменные окружения:
- APP_LOG_PATH: путь к активному лог-файлу (устанавливается после init)
- APP_LOG_DIR: директория логов
"""
```

Б.5. logging_setup.py

Файл: src/utils/logging_setup.py (171 строк)

```
"""
Конфигурация логирования приложения.

Назначение:
- Инициализация корневого логгера с JSON-форматом для файла и консоли.
- Безопасный выбор пути лог-файла с несколькими резервными сценариями.

Внешние библиотеки:
- logging (RotatingFileHandler, Formatter), json: вывод JSON-строк в лог.
- os/tempfile/uuid/datetime/socket: пути, уникальные run_id и контекст среды.
```

```
Переменные окружения:
- APP_LOG_PATH — путь к активному лог-файлу, выставляется после инициализации.
"""
import json
import logging
import os
import socket
import time
import uuid
from datetime import datetime
from logging import LogRecord
from logging.handlers import RotatingFileHandler
import tempfile


class JsonFormatter(logging.Formatter):
    """Форматтер: преобразует LogRecord в компактную JSON-строку."""
    def format(self, record: LogRecord) -> str:
        """Собрать словарь полей и сериализовать в JSON (ensure_ascii=False)."""
        data = {
            "ts": time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(record.created)),
            "ts_ms": int(record.created * 1000),
            "level": record.levelname,
            "logger": record.name,
            "msg": record.getMessage(),
            "file": record.pathname,
            "line": record.lineno,
            "func": record.funcName,
            "pid": record.process,
            "thread": record.threadName,
        }
        for k, v in getattr(record, "__dict__", {}).items():
            if k not in data and k not in (
                "name","msg","args","levelname","levelno","pathname","filename","module",
                "exc_info","exc_text","stack_info","lineno","funcName","created","msecs",
                "relativeCreated","thread","threadName","processName","process","message",
            ):
                try:
                    json.dumps(v)
                    data[k] = v
                except Exception:
                    data[k] = str(v)
        return json.dumps(data, ensure_ascii=False)


class ContextFilter(logging.Filter):
    """Фильтр: добавляет run_id и host ко всем записям лога."""
    def __init__(self, run_id: str):
        super().__init__()
        self.run_id = run_id
        self.host = socket.gethostname()

    def filter(self, record: LogRecord) -> bool:
        """Внедрить контекст в LogRecord и разрешить запись (True)."""
        record.run_id = self.run_id
        record.host = self.host
        return True


def setup_logging(path_or_dir: str, *, json_logs: bool = True) -> None:
    """Настроить корневое логирование в файл и консоль.

    Поведение:
    - Если дан каталог — создаёт уникальный файл logs/app_YYYY-MM-DD_HHMMSS_<run>.log
    - Если дан путь к .log — использует его напрямую, создавая каталоги по необходимости
    - Ротация файла: 5MB × 3
    - Консоль: INFO+; файл: DEBUG+

    If `path_or_dir` is a directory, a unique file name is created.
    If it looks like a file (.log), it's used as-is.
    - File: DEBUG+ as JSON lines (rotating 5MB x 3)
    - Console: INFO+ as JSON lines (or text if json_logs=False)
    """
    root = logging.getLogger()
    # Reconfigure even if handlers exist
    try:
        for h in list(root.handlers):
            try:
                h.flush()
```

```
                h.close()
            except Exception:
                pass
            root.removeHandler(h)
    except Exception:
        pass
    root.setLevel(logging.DEBUG)

    run_id = str(uuid.uuid4())
    run_short = run_id.split('-')[0]

    # Decide path with probing and fallbacks
    def _mk(base_dir: str) -> str:
        os.makedirs(base_dir, exist_ok=True)
        ts = datetime.now().strftime('%Y-%m-%d_%H%M%S')
        return os.path.join(base_dir, f'app_{ts}_{run_short}.log')

    candidates = []
    try:
        is_dir = os.path.isdir(path_or_dir) or not os.path.splitext(path_or_dir)[1]
    except Exception:
        is_dir = True
    if is_dir:
        candidates.append(_mk(path_or_dir))
    else:
        try:
            os.makedirs(os.path.dirname(path_or_dir), exist_ok=True)
            candidates.append(path_or_dir)
        except Exception:
            pass
    # LocalAppData fallback
    candidates.append(_mk(os.path.join(os.environ.get('LOCALAPPDATA', os.path.expanduser('~')),
'AudioTransformer', 'logs')))
    # Temp dir fallback
    candidates.append(_mk(os.path.join(tempfile.gettempdir(), 'AudioTransformer', 'logs')))

    log_path = None
    for p in candidates:
        try:
            os.makedirs(os.path.dirname(p), exist_ok=True)
            with open(p, 'a', encoding='utf-8') as f:
                f.write('')
            log_path = p
            break
        except Exception:
            continue
    if not log_path:
        # as absolute last resort, don't crash; use temp without writing test
        log_path = os.path.join(tempfile.gettempdir(), f'app_{run_short}.log')

    # Context
    filt = ContextFilter(run_id)
    root.addFilter(filt)

    file_fmt = JsonFormatter() if json_logs else logging.Formatter('%(asctime)s [%(levelname)s] %
(name)s: %(message)s', '%H:%M:%S')
    stream_fmt = JsonFormatter() if json_logs else file_fmt

    try:
        fh = RotatingFileHandler(log_path, maxBytes=5 * 1024 * 1024, backupCount=3,
encoding='utf-8')
        fh.setLevel(logging.DEBUG)
        fh.setFormatter(file_fmt)
        root.addHandler(fh)
    except Exception:
        # Try temp file directly
        try:
            temp_path = os.path.join(tempfile.gettempdir(), f'app_{run_short}.log')
            fh = RotatingFileHandler(temp_path, maxBytes=5 * 1024 * 1024, backupCount=3,
encoding='utf-8')
            fh.setLevel(logging.DEBUG)
            fh.setFormatter(file_fmt)
            root.addHandler(fh)
            log_path = temp_path
        except Exception:
            pass

    sh = logging.StreamHandler()
    sh.setLevel(logging.INFO)
    sh.setFormatter(stream_fmt)
```

```
    root.addHandler(sh)

    os.environ['APP_LOG_PATH'] = log_path
    logging.getLogger("app").info("logging_initialized", extra={"log_path": log_path, "run_id":
run_id})
```

Б.6. logging_new.py

Файл: src/utils/logging_new.py (331 строк)

```
"""
Structured logging configuration for AudioAnalyzer.

Provides:
- JSON-formatted logs for production
- Colorized console output for development
- Context-aware logging with processing metadata
- File rotation and external log aggregation support

Usage:
------
>>> from utils.logging_setup import get_logger
>>> logger = get_logger("audio.processor")
>>> logger.info("processing_started", file_count=10, method="fwht")
"""
from __future__ import annotations

import logging
import sys
from datetime import datetime
from pathlib import Path
from typing import Any, Dict, Optional, Union

try:
    import structlog
    from structlog.types import Processor
    HAS_STRUCTLOG = True
except ImportError:
    HAS_STRUCTLOG = False

# Default log format
DEFAULT_FORMAT = "%(asctime)s - %(name)s - %(levelname)s - %(message)s"
DEFAULT_DATE_FORMAT = "%Y-%m-%d %H:%M:%S"


def setup_logging(
    level: str = "INFO",
    log_file: Optional[Union[str, Path]] = None,
    json_format: bool = False,
    include_context: bool = True,
) -> None:
    """Configure application-wide logging.

    Parameters
    ----------
    level : str
        Log level (DEBUG, INFO, WARNING, ERROR, CRITICAL)
    log_file : Optional[Union[str, Path]]
        Path to log file. If None, logs only to console.
    json_format : bool
        If True, use JSON format for structured logging.
    include_context : bool
        If True, include context information in logs.
    """
    log_level = getattr(logging, level.upper(), logging.INFO)

    if HAS_STRUCTLOG:
        _setup_structlog(log_level, log_file, json_format, include_context)
    else:
        _setup_standard_logging(log_level, log_file)


def _setup_structlog(
    level: int,
    log_file: Optional[Union[str, Path]],
    json_format: bool,
    include_context: bool,
```

```
) -> None:
    """Set up structlog with processors."""
    import structlog
    from structlog.dev import ConsoleRenderer
    from structlog.processors import JSONRenderer

    # Shared processors for all loggers
    shared_processors: list[Processor] = [
        structlog.contextvars.merge_contextvars,
        structlog.stdlib.add_log_level,
        structlog.stdlib.add_logger_name,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.StackInfoRenderer(),
    ]

    if include_context:
        shared_processors.append(
            structlog.processors.UnicodeDecoder()
        )

    # Configure structlog
    structlog.configure(
        processors=shared_processors + [
            structlog.stdlib.ProcessorFormatter.wrap_for_formatter,
        ],
        logger_factory=structlog.stdlib.LoggerFactory(),
        wrapper_class=structlog.stdlib.BoundLogger,
        cache_logger_on_first_use=True,
    )

    # Configure standard logging handlers
    handlers = []


    # Console handler
    console_handler = logging.StreamHandler(sys.stdout)
    if json_format:
        formatter = structlog.stdlib.ProcessorFormatter(
            processor=structlog.processors.JSONRenderer(),
            foreign_pre_chain=shared_processors,
        )
    else:
        formatter = structlog.stdlib.ProcessorFormatter(
            processor=structlog.dev.ConsoleRenderer(colors=True),
            foreign_pre_chain=shared_processors,
        )
    console_handler.setFormatter(formatter)
    handlers.append(console_handler)

    # File handler (if specified)
    if log_file:
        file_path = Path(log_file)
        file_path.parent.mkdir(parents=True, exist_ok=True)

        file_handler = logging.FileHandler(file_path, encoding="utf-8")
        file_formatter = structlog.stdlib.ProcessorFormatter(
            processor=structlog.processors.JSONRenderer(),
            foreign_pre_chain=shared_processors,
        )
        file_handler.setFormatter(file_formatter)
        handlers.append(file_handler)

    # Configure root logger
    root_logger = logging.getLogger()
    root_logger.setLevel(level)

    # Remove existing handlers
    for handler in root_logger.handlers[:]:
        root_logger.removeHandler(handler)

    # Add new handlers
    for handler in handlers:
        root_logger.addHandler(handler)


def _setup_standard_logging(
    level: int,
    log_file: Optional[Union[str, Path]],
) -> None:
    """Set up standard Python logging as fallback."""
```

```
    import logging.config

    handlers = ["console"]


    config = {
        "version": 1,
        "disable_existing_loggers": False,
        "formatters": {
            "standard": {
                "format": DEFAULT_FORMAT,
                "datefmt": DEFAULT_DATE_FORMAT,
            },
            "detailed": {
                "format": "%(asctime)s - %(name)s - %(levelname)s - %(filename)s:%(lineno)d - %
(message)s",
                "datefmt": DEFAULT_DATE_FORMAT,
            },
        },
        "handlers": {
            "console": {
                "class": "logging.StreamHandler",
                "stream": sys.stdout,
                "formatter": "standard",
                "level": level,
            },
        },
        "loggers": {
            "": {
                "handlers": handlers,
                "level": level,
                "propagate": True,
            },
            "audio": {
                "handlers": handlers,
                "level": level,
                "propagate": False,
            },
            "processing": {
                "handlers": handlers,
                "level": level,
                "propagate": False,
            },
            "ui_new": {
                "handlers": handlers,
                "level": level,
                "propagate": False,
            },
        },
    }

    if log_file:
        file_path = Path(log_file)
        file_path.parent.mkdir(parents=True, exist_ok=True)

        config["handlers"]["file"] = {
            "class": "logging.handlers.RotatingFileHandler",
            "filename": str(file_path),
            "maxBytes": 10 * 1024 * 1024,  # 10 MB
            "backupCount": 5,
            "formatter": "detailed",
            "encoding": "utf-8",
        }
        handlers.append("file")

    logging.config.dictConfig(config)


def get_logger(name: str) -> "BoundLogger":
    """Get a logger instance with the given name.

    Parameters
    ----------
    name : str
        Logger name (typically module name)

    Returns
    -------
    BoundLogger
        A structlog or standard logger instance
```

```
    """
    if HAS_STRUCTLOG:
        return structlog.get_logger(name)
    else:
        return logging.getLogger(name)


class LogContext:
    """Context manager for adding temporary logging context.

    Example:
    -------
    >>> with LogContext(file="test.wav", method="fwht"):
    ...     logger.info("processing_started")
    """

    def __init__(self, **kwargs: Any):
        """Initialize context with key-value pairs."""
        self.context = kwargs
        self._token: Optional[Any] = None

    def __enter__(self) -> "LogContext":
        """Enter context and bind variables."""
        if HAS_STRUCTLOG:
            import structlog
            self._token = structlog.contextvars.bind_contextvars(**self.context)
        return self

    def __exit__(self, *args: Any) -> None:
        """Exit context and unbind variables."""
        if HAS_STRUCTLOG and self._token is not None:
            import structlog
            structlog.contextvars.unbind_contextvars(*self.context.keys())


def log_performance(
    operation: str,
    duration_ms: float,
    **kwargs: Any,
) -> None:
    """Log a performance metric.

    Parameters
    ----------
    operation : str
        Name of the operation
    duration_ms : float
        Duration in milliseconds
    **kwargs : Any
        Additional context
    """
    logger = get_logger("performance")
    logger.info(
        operation,
        duration_ms=duration_ms,
        **kwargs,
    )


def log_error_with_context(
    error: Exception,
    operation: str,
    **kwargs: Any,
) -> None:
    """Log an error with additional context.

    Parameters
    ----------
    error : Exception
        The exception that occurred
    operation : str
        Name of the operation that failed
    **kwargs : Any
        Additional context
    """
    logger = get_logger("errors")
    logger.error(
        f"{operation}_failed",
        error_type=type(error).__name__,
        error_message=str(error),
```

```
        **kwargs,
    )


# Initialize default logging on import
_default_initialized = False


def ensure_logging_initialized() -> None:
    """Ensure logging is initialized with defaults."""
    global _default_initialized
    if not _default_initialized:
        setup_logging(level="INFO")
        _default_initialized = True


# Export public API
__all__ = [
    "setup_logging",
    "get_logger",
    "LogContext",
    "log_performance",
    "log_error_with_context",
    "ensure_logging_initialized",
    "HAS_STRUCTLOG",
]
```

Б.7. __init__.py

Файл: src/processing/__init__.py (211 строк)

```
"""
processing: пакет вычислительной части и кодеков.

Назначение:
- Предоставляет унифицированный API для обработки аудио
- Экспортирует все методы трансформаций
- Экспортирует метрики качества

Структура модуля:
=================
processing/
├── __init__.py          # Этот файл - публичный API
├── audio_ops.py         # Standard MP3 + переэкспорт трансформаций
├── codecs.py            # FFmpeg взаимодействия
├── metrics.py           # Метрики качества
├── utils.py             # Вспомогательные функции
├── api.py               # Устаревший фасад совместимости
└── transforms/          # Реализации трансформаций
    ├── __init__.py      # Экспорт всех трансформаций
    ├── base.py          # Базовый класс и утилиты OLA
    ├── fft.py           # FFT
    ├── dct.py           # DCT
    ├── dwt.py           # DWT (Хаар)
    ├── fwht.py          # FWHT
    ├── huffman.py       # Huffman-like (μ-law)
    └── rosenbrock.py    # Rosenbrock-like

Доступные методы обработки:
===========================
1. Standard MP3 - прямое кодирование WAV→MP3 через ffmpeg
2. FFT - Быстрое преобразование Фурье
3. DCT - Дискретное косинусное преобразование
4. DWT - Вейвлет Хаара
5. FWHT - Преобразование Уолша-Адамара
6. Huffman-like - μ-law компандирование
7. Rosenbrock-like - Нелинейное сглаживание

Использование:
==============
>>> from processing import (
...     # Методы обработки
...     standard_convert_to_mp3,
...     fft_transform_and_mp3,
...     fwht_transform_and_mp3,
...
...     # Метрики
```

```
...     compute_snr_db,
...     compute_metrics_batch,
... )


Быстрый старт:
--------------
>>> # Стандартное MP3
>>> out, t = standard_convert_to_mp3("audio.wav", "output/")
>>>
>>> # FFT с отбором по энергии
>>> out, t = fft_transform_and_mp3("audio.wav", "output/",
...     select_mode="energy", keep_energy_ratio=0.8)
>>>
>>> # FWHT
>>> out, t = fwht_transform_and_mp3("audio.wav", "output/")

Метрики качества:
=================
- SNR (dB): Signal-to-Noise Ratio
- RMSE: Root Mean Square Error
- SI-SDR (dB): Scale-Invariant SDR
- LSD (dB): Log-Spectral Distance
- Spectral Convergence: мера сходства спектров
- Spectral Centroid Δ: разница центроидов
- Cosine Similarity: косинусное сходство спектров

>>> from processing import compute_metrics_batch
>>> results = compute_metrics_batch("original.wav", [
...     ("MP3", "output.mp3", 1.5),
...     ("FWHT", "output_fwht.mp3", 2.3),
... ])
"""
from __future__ import annotations

# ============================================================================
# ОСНОВНЫЕ МЕТОДЫ ОБРАБОТКИ
# ============================================================================

from .audio_ops import (
    # Стандартный MP3
    standard_convert_to_mp3,

    # Трансформации (переэкспорт)
    FFTTransform,
    DCTTransform,
    DWTTransform,
    FWHTTransform,
    HuffmanLikeTransform,
    RosenbrockLikeTransform,

    # Функции-обёртки
    fft_transform_and_mp3,
    dct_transform_and_mp3,
    wavelet_transform_and_mp3,
    fwht_transform_and_mp3,
    huffman_like_transform_and_mp3,
    rosenbrock_like_transform_and_mp3,

    # Пакетный расчёт
    _compute_metrics_batch,
    compare_results,
)


# ============================================================================
# МЕТРИКИ КАЧЕСТВА
# ============================================================================

from .metrics import (
    compute_snr_db,
    compute_rmse,
    compute_si_sdr_db,
    compute_lsd_db,
    compute_spectral_convergence,
    compute_spectral_centroid_diff_hz,
    compute_spectral_cosine_similarity,
    compute_metrics_batch,
)
```

```
# ==============================================================================
# УТИЛИТЫ
# ==============================================================================

from .utils import (
    is_power_of_two,
    normalize_ratio,
    parse_int,
    parse_float,
)


# ==============================================================================
# КОНСТАНТЫ ИЗ TRANSFORMS
# ==============================================================================

from .transforms import (
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
    VALID_SELECT_MODES,
    TRANSFORM_CLASSES,
    TRANSFORM_FUNCTIONS,
    get_transform,
    get_transform_function,
)


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    # Стандартный MP3
    "standard_convert_to_mp3",

    # Классы трансформаций
    "FFTTransform",
    "DCTTransform",
    "DWTTransform",
    "FWHTTransform",
    "HuffmanLikeTransform",
    "RosenbrockLikeTransform",

    # Функции-обёртки
    "fft_transform_and_mp3",
    "dct_transform_and_mp3",
    "wavelet_transform_and_mp3",
    "fwht_transform_and_mp3",
    "huffman_like_transform_and_mp3",
    "rosenbrock_like_transform_and_mp3",

    # Метрики
    "compute_snr_db",
    "compute_rmse",
    "compute_si_sdr_db",
    "compute_lsd_db",
    "compute_spectral_convergence",
    "compute_spectral_centroid_diff_hz",
    "compute_spectral_cosine_similarity",
    "compute_metrics_batch",

    # Пакетный расчёт
    "compare_results",
    "_compute_metrics_batch",

    # Утилиты
    "is_power_of_two",
    "normalize_ratio",
    "parse_int",
    "parse_float",

    # Константы
    "SELECT_MODE_NONE",
    "SELECT_MODE_ENERGY",
    "SELECT_MODE_LOWPASS",
    "VALID_SELECT_MODES",

    # Утилиты выбора трансформации
```

```
    "TRANSFORM_CLASSES",
    "TRANSFORM_FUNCTIONS",
    "get_transform",
    "get_transform_function",
]
```

Б.8. utils.py

Файл: src/processing/utils.py (107 строк)

```
"""
Общие утилиты для модуля processing.

Назначение:
- Разделяемые функции, используемые в нескольких модулях.
- Избежание дублирования кода.

Содержимое:
- is_power_of_two: проверка числа на степень двойки
- normalize_ratio: нормализация параметра в диапазон [0.0, 1.0]
"""
from __future__ import annotations

import logging
from typing import Optional

logger = logging.getLogger("audio.processing.utils")


def is_power_of_two(n: int) -> bool:
    """Проверка: n — степень двойки (>0).

    Параметры:
    - n: проверяемое число

    Возвращает: True если n является степенью двойки, False иначе.
    """
    return (n & (n - 1) == 0) and n > 0


def normalize_ratio(value: float, default: float = 1.0) -> float:
    """Нормализация параметра в диапазон [0.0, 1.0].

    Параметры:
    - value: входное значение
    - default: значение по умолчанию при ошибке

    Возвращает: нормализованное значение в диапазоне [0.0, 1.0].
    """
    try:
        return float(max(0.0, min(1.0, float(value))))
    except (TypeError, ValueError) as e:
        logger.debug("normalize_ratio_error: %s, using default %s", e, default)
        return default


def parse_int(value: any, default: int = 0, min_val: Optional[int] = None, max_val: Optional[int]
= None) -> int:
    """Безопасный парсинг целого числа с ограничениями.

    Параметры:
    - value: входное значение
    - default: значение по умолчанию при ошибке
    - min_val: минимальное допустимое значение (опционально)
    - max_val: максимальное допустимое значение (опционально)

    Возвращает: распарсенное целое число.
    """
    try:
        result = int(value)
    except (TypeError, ValueError) as e:
        logger.debug("parse_int_error: %s, using default %s", e, default)
        result = default

    if min_val is not None:
        result = max(min_val, result)
    if max_val is not None:
```

```
        result = min(max_val, result)

    return result


def parse_float(value: any, default: float = 0.0, min_val: Optional[float] = None, max_val:
Optional[float] = None) -> float:
    """Безопасный парсинг числа с плавающей точкой с ограничениями.

    Параметры:
    - value: входное значение
    - default: значение по умолчанию при ошибке
    - min_val: минимальное допустимое значение (опционально)
    - max_val: максимальное допустимое значение (опционально)

    Возвращает: распарсенное число.
    """
    try:
        result = float(value)
    except (TypeError, ValueError) as e:
        logger.debug("parse_float_error: %s, using default %s", e, default)
        result = default

    if min_val is not None:
        result = max(min_val, result)
    if max_val is not None:
        result = min(max_val, result)

    return result


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================


__all__ = [
    "is_power_of_two",
    "normalize_ratio",
    "parse_int",
    "parse_float",
]
```

Б.9. api.py

Файл: src/processing/api.py (71 строк)

```
"""
УСТАРЕВШИЙ фасад совместимости.

Назначение:
- Сохранить обратную совместимость публичного API ранних версий
- Переэкспортировать актуальные функции из processing.audio_ops

Рекомендация:
-------------
Для нового кода используйте прямые импорты:

>>> from processing import (
...     fwht_transform_and_mp3,
...     fft_transform_and_mp3,
...     standard_convert_to_mp3,
... )

или:

>>> from processing.audio_ops import fwht_transform_and_mp3
>>> from processing.transforms import FWHTTransform

Используемые библиотеки:
------------------------
- typing: подсказки типов
"""
from __future__ import annotations

from typing import Callable, Optional, Tuple, List, Dict
```

```
# Переэкспорт актуальных функций (сохраняем старые точки входа для внешнего кода)
from .audio_ops import (
    # Основные методы
    fwht_transform_and_mp3,
    fft_transform_and_mp3,
    dct_transform_and_mp3,
    wavelet_transform_and_mp3,
    huffman_like_transform_and_mp3,
    rosenbrock_like_transform_and_mp3,
    standard_convert_to_mp3,

    # Сравнение результатов
    compare_results,

    # Метрики (для удобства)
    compute_snr_db,
    compute_rmse,
    compute_si_sdr_db,
    compute_lsd_db,
)


# ============================================================================
# ЭКСПОРТ ИМЁН
# ============================================================================

__all__ = [
    "fwht_transform_and_mp3",
    "fft_transform_and_mp3",
    "dct_transform_and_mp3",
    "wavelet_transform_and_mp3",
    "huffman_like_transform_and_mp3",
    "rosenbrock_like_transform_and_mp3",
    "standard_convert_to_mp3",
    "compare_results",
    "compute_snr_db",
    "compute_rmse",
    "compute_si_sdr_db",
    "compute_lsd_db",
]
```

Б.10. audio_ops.py

Файл: src/processing/audio_ops.py (305 строк)

```
"""
Аудио-пайплайны обработки.

Назначение:
- Стандартное MP3 кодирование через ffmpeg
- Переэкспорт трансформаций из transforms/ для обратной совместимости
- Пакетный расчёт метрик качества

Обзор методов обработки:
========================

1. Стандартный MP3 - прямое кодирование WAV в MP3 через ffmpeg.
   Это базовый метод для сравнения.

2. FFT (Быстрое преобразование Фурье) - классический метод частотного анализа.
   Разлагает сигнал на синусоидальные компоненты.
   - Преимущества: хорошая частотная локализация, широкий спектр применения
   - Недостатки: требует комплексных вычислений

3. FWHT (Быстрое преобразование Уолша-Адамара) - использует только сложения/вычитания.
   Разлагает сигнал на функции Уолша (прямоугольные волны).
   - Преимущества: быстрые вычисления, хорош для бинарных сигналов

4. DCT (Дискретное косинусное преобразование) - используется в JPEG, MP3.
   - Преимущества: хорошее энергетическое сжатие

5. DWT (Дискретное вейвлет-преобразование, Хаар) - многоуровневое разложение.
   - Преимущества: хорошее временное и частотное разрешение

6. Huffman-like (μ-law компандирование) - нелинейное сжатие динамического диапазона.
   Сжимает громкие звуки меньше чем тихие.
```

```
7. Rosenbrock-like - эвристическое нелинейное преобразование для сглаживания.

Структура модуля:
=================
- audio_ops.py: стандартный MP3 + переэкспорт
- transforms/: реализации всех трансформаций
  - base.py: базовый класс и утилиты OLA
  - fft.py: FFT трансформация
  - dct.py: DCT трансформация
  - dwt.py: DWT (Хаар) трансформация
  - fwht.py: FWHT трансформация
  - huffman.py: Huffman-like трансформация
  - rosenbrock.py: Rosenbrock-like трансформация
- codecs.py: FFmpeg взаимодействия
- metrics.py: метрики качества
- utils.py: вспомогательные функции
- api.py: устаревший фасад совместимости


Использование:
==============
>>> from processing.audio_ops import (
...     standard_convert_to_mp3,
...     fft_transform_and_mp3,
...     fwht_transform_and_mp3,
... )
>>>
>>> # Стандартное MP3
>>> out_path, time_sec = standard_convert_to_mp3("audio.wav", "output/")
>>>
>>> # FFT с отбором по энергии
>>> out_path, time_sec = fft_transform_and_mp3(
...     "audio.wav", "output/",
...     select_mode="energy",
...     keep_energy_ratio=0.8
... )

Внешние библиотеки:
-------------------
- numpy: численные операции
- logging: диагностические сообщения
"""
from __future__ import annotations

import logging
from typing import Dict, List, Tuple

# Импорт из codecs для Standard MP3
from .codecs import (
    ensure_ffmpeg_available,
    load_wav_mono,
    decode_audio_to_mono,
    standard_convert_to_mp3 as _standard_convert_to_mp3,
    get_audio_meta,
)

# Импорт метрик
from .metrics import compute_metrics_batch as _compute_metrics_batch_internal

# Переэкспорт всех трансформаций из transforms/
from .transforms import (
    # Классы трансформаций
    FFTTransform,
    DCTTransform,
    DWTTransform,
    FWHTTransform,
    HuffmanLikeTransform,
    RosenbrockLikeTransform,


    # Функции-обёртки
    fft_transform_and_mp3,
    dct_transform_and_mp3,
    wavelet_transform_and_mp3,
    fwht_transform_and_mp3,
    huffman_like_transform_and_mp3,
    rosenbrock_like_transform_and_mp3,

    # Утилиты (для совместимости)
    load_audio_safe,
```

```
    create_ola_window,
    finalize_ola,
)

# Переэкспорт метрик для обратной совместимости
from .metrics import (
    compute_snr_db,
    compute_rmse,
    compute_si_sdr_db,
    compute_lsd_db,
    compute_spectral_convergence,
    compute_spectral_centroid_diff_hz,
    compute_spectral_cosine_similarity,
)

logger = logging.getLogger("audio.processing")


# ==============================================================================
# СТАНДАРТНОЕ ПРЕОБРАЗОВАНИЕ MP3
# ==============================================================================

def standard_convert_to_mp3(
    wav_path: str,
    out_dir: str,
    bitrate: str = "192k"
) -> Tuple[str, float]:
    """Обёртка над codecs.standard_convert_to_mp3 с логированием.

    Прямое кодирование WAV в MP3 через FFmpeg без предварительной обработки.
    Используется как базовый метод для сравнения с другими трансформациями.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному WAV файлу
    out_dir : str
        Директория для сохранения MP3
    bitrate : str
        Битрейт MP3 (например, '192k', '128k', '320k')

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время обработки в секундах)

    Пример:
    -------
    >>> out_path, time_sec = standard_convert_to_mp3("audio.wav", "output/")
    >>> print(f"Сохранено в {out_path} за {time_sec:.2f} сек")
    """
    logger.info("standard_convert_to_mp3 start path=%s bitrate=%s", wav_path, bitrate)
    out, dt = _standard_convert_to_mp3(wav_path, out_dir, bitrate)
    logger.info("standard_convert_to_mp3 done path=%s out=%s dt=%.3f", wav_path, out, dt)
    return out, dt


# ==============================================================================
# ПАКЕТНЫЙ РАСЧЁТ МЕТРИК
# ==============================================================================

def _compute_metrics_batch(
    original_wav: str,
    items: List[Tuple[str, str, float]],
) -> List[Dict]:
    """Посчитать метрики качества для набора результатов.

    Для каждого обработанного файла вычисляет:
    - Размер файла
    - SNR (Signal-to-Noise Ratio)
    - RMSE (Root Mean Square Error)
    - SI-SDR (Scale-Invariant Signal-to-Distortion Ratio)
    - LSD (Log-Spectral Distance)
    - Spectral Convergence
    - Spectral Centroid Difference
    - Spectral Cosine Similarity
    - Общий score

    Параметры:
    ----------
```

```
    original_wav : str
        Путь к исходному WAV (референс)
    items : List[Tuple[str, str, float]]
        Список кортежей (variant, path_to_mp3, time_sec)

    Возвращает:
    -----------
    List[Dict]
        Список словарей с полями размера, метрик, времени и score


    Пример:
    -------
    >>> items = [
    ...     ("Стандартный MP3", "output/audio_std.mp3", 1.2),
    ...     ("FWHT MP3", "output/audio_fwht.mp3", 2.5),
    ... ]
    >>> results = _compute_metrics_batch("audio.wav", items)
    """
    return _compute_metrics_batch_internal(
        original_wav,
        items,
        load_wav_func=load_wav_mono,
        decode_audio_func=decode_audio_to_mono,
        get_meta_func=get_audio_meta,
    )


def compare_results(
    original_wav: str,
    std_mp3: str,
    fwht_mp3: str,
    t_std: float,
    t_fwht: float,
    fft_mp3: str = None,
    t_fft: float = None,
) -> List[Dict]:
    """СОВМЕСТИМОСТЬ: старая сигнатура для сравнения результатов.

    Сохраняет обратную совместимость с кодом, использующим
    старую сигнатуру функции compare_results.

    Параметры:
    ----------
    original_wav : str
        Путь к исходному WAV
    std_mp3 : str
        Путь к стандартному MP3
    fwht_mp3 : str
        Путь к FWHT MP3
    t_std : float
        Время обработки стандартного MP3
    t_fwht : float
        Время обработки FWHT MP3
    fft_mp3 : str, optional
        Путь к FFT MP3
    t_fft : float, optional
        Время обработки FFT MP3

    Возвращает:
    -----------
    List[Dict]
        Список результатов с метриками

    Примечание:
    -----------
    Рекомендуется использовать _compute_metrics_batch напрямую
    для произвольного числа методов.
    """
    items: List[Tuple[str, str, float]] = [
        ("Стандартный MP3", std_mp3, float(t_std)),
        ("FWHT MP3", fwht_mp3, float(t_fwht)),
    ]
    if fft_mp3 is not None:
        items.append(("FFT MP3", fft_mp3, float(t_fft or float("nan"))))
    return _compute_metrics_batch(original_wav, items)


# ============================================================================
# ЭКСПОРТ ИМЁН
```

```
# ==============================================================================

__all__ = [
    # Стандартный MP3
    "standard_convert_to_mp3",

    # Трансформации (переэкспорт из transforms/)
    "FFTTransform",
    "DCTTransform",
    "DWTTransform",
    "FWHTTransform",
    "HuffmanLikeTransform",
    "RosenbrockLikeTransform",

    # Функции-обёртки
    "fft_transform_and_mp3",
    "dct_transform_and_mp3",
    "wavelet_transform_and_mp3",
    "fwht_transform_and_mp3",
    "huffman_like_transform_and_mp3",
    "rosenbrock_like_transform_and_mp3",

    # Метрики (переэкспорт)
    "compute_snr_db",
    "compute_rmse",
    "compute_si_sdr_db",
    "compute_lsd_db",
    "compute_spectral_convergence",
    "compute_spectral_centroid_diff_hz",
    "compute_spectral_cosine_similarity",


    # Пакетный расчёт
    "_compute_metrics_batch",
    "compare_results",
]
```

Б.11. codecs.py

Файл: src/processing/codecs.py (371 строк)

```
"""
Кодеки и I/O для аудио.

Назначение:
- Единая точка для всех операций кодирования/декодирования и пробинга метаданных.
- Поддержка FFmpeg/ffprobe (через pydub и прямые вызовы), soundfile для WAV.

Внешние библиотеки:
- pydub/FFmpeg: универсальное декодирование/кодирование.
- soundfile: точная работа с WAV и определением битовой глубины.
- numpy: преобразование форматов PCM и буферов stdin/stdout для ffmpeg.
- subprocess/os/shutil: вызовы внешних утилит и настройка путей.
"""
from __future__ import annotations

import os
import shutil
import logging
from typing import Dict, Tuple

import numpy as np
import soundfile as sf
from pydub import AudioSegment
import subprocess
import sys

logger = logging.getLogger("audio.codecs")

_FFMPEG_CONFIGURED = False


def configure_ffmpeg_search() -> None:
    """Автонастройка путей ffmpeg/ffprobe для pydub.

    Логика:
    - Сначала берём FFMPEG_BINARY из ENV или which('ffmpeg')
```

```
    - На Windows дополнительно ищем WinGet-путь (gyan.ffmpeg)
    - Для ffprobe: рядом с ffmpeg, затем FFPROBE_BINARY, затем which('ffprobe')
    - Проставляем пути в pydub и ENV для стабильной работы внутри exe
    """
    global _FFMPEG_CONFIGURED
    if _FFMPEG_CONFIGURED:
        return
    # Ищем ffmpeg по приоритетам
    path = os.environ.get("FFMPEG_BINARY") or shutil.which("ffmpeg")
    if not path:
        try:
            base = os.path.join(os.environ.get("LOCALAPPDATA", ""), "Microsoft", "WinGet",
"Packages")
            if os.path.isdir(base):
                for name in os.listdir(base):
                    if name.lower().startswith("gyan.ffmpeg"):
                        cand = os.path.join(base, name, "ffmpeg-8.0-full_build", "bin",
"ffmpeg.exe")
                        if os.path.exists(cand):
                            path = cand
                            break
        except Exception as e:
            logger.debug("WinGet ffmpeg search failed: %s", e)
    if path and os.path.exists(path):
        os.environ["FFMPEG_BINARY"] = path
        AudioSegment.converter = path  # уведомляем pydub о местоположении ffmpeg
    # Ищем ffprobe по соседству с ffmpeg либо по ENV/PATH
    probe = None
    try:
        if path and path.endswith("ffmpeg.exe"):
            cand = path.replace("ffmpeg.exe", "ffprobe.exe")
            if os.path.exists(cand):
                probe = cand
    except Exception as e:
        logger.debug("ffprobe path lookup failed: %s", e)
    if not probe:
        p_env = os.environ.get("FFPROBE_BINARY")
        if p_env and os.path.exists(p_env):
            probe = p_env
    if not probe:
        w = shutil.which("ffprobe")
        if w and os.path.exists(w):
            probe = w
    if probe:
        AudioSegment.ffprobe = probe
        os.environ["FFPROBE_BINARY"] = probe
    _FFMPEG_CONFIGURED = True
    logger.debug("FFmpeg configured: %s; ffprobe: %s", os.environ.get("FFMPEG_BINARY"),
os.environ.get("FFPROBE_BINARY"))


def ensure_ffmpeg_available() -> None:
    """Убедиться, что ffmpeg/ffprobe доступны (сконфигурированы в pydub/ENV).

    Бросает RuntimeError с подсказкой, если обнаружить бинарники не удалось.
    """
    configure_ffmpeg_search()
    try:
        AudioSegment.converter
    except Exception as e:
        raise RuntimeError("FFmpeg недоступен. Установите FFmpeg и добавьте его в PATH.") from e


def load_wav_mono(path: str) -> Tuple[np.ndarray, int]:
    """Загрузить WAV и привести к моно float32.

    Возвращает: (x, sample_rate). Для стерео берётся среднее по каналам.
    """
    data, sr = sf.read(path, always_2d=False)
    if getattr(data, 'ndim', 1) == 2:
        data = data.mean(axis=1)
    x = data.astype(np.float32)
    logger.debug("Loaded WAV mono: %s sr=%d len=%d", path, sr, len(x))
    return x, sr


def _ffmpeg_creationflags() -> int:
    """Флаг для скрытого запуска ffmpeg/ffprobe в Windows (и 0 для других ОС)."""
    if sys.platform.startswith('win'):
        return 0x08000000  # CREATE_NO_WINDOW
```

```
    return 0


def _run_ffmpeg(args: list[str]) -> subprocess.CompletedProcess:
    """Запустить ffmpeg/ffprobe с подавлением окна и собрать stdout/stderr."""
    configure_ffmpeg_search()
    logger.debug("Run ffmpeg: %s", " ".join(args))
    return subprocess.run(
        args,
        stdin=subprocess.DEVNULL,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        creationflags=_ffmpeg_creationflags(),
        check=True,
    )


def _probe_sample_rate(path: str) -> int:
    """Определить частоту дискретизации файла через ffprobe; фоллбек 44100."""
    try:
        cp = _run_ffmpeg([
            AudioSegment.ffprobe if hasattr(AudioSegment, 'ffprobe') else 'ffprobe',
            '-v','error','-select_streams','a:0','-show_entries','stream=sample_rate','-
of','default=nk=1:nw=1',
            path
        ])
        sr = int(cp.stdout.strip().decode('utf-8', errors='ignore') or '0')
        return sr if sr > 0 else 44100
    except Exception as e:
        logger.debug("_probe_sample_rate failed for %s: %s", path, e)
        return 44100


def decode_audio_to_mono(path: str) -> Tuple[np.ndarray, int]:
    """Декодировать аудиофайл через ffmpeg в моно PCM float32 без временных файлов.

    - Частота выбирается через ffprobe; поток читается из stdout (-f s16le).
    - Возвращает (x, sample_rate).
    """
    sr = _probe_sample_rate(path)
    # ffmpeg пишет raw PCM (s16le) в stdout; читаем из CompletedProcess.stdout
    cp = _run_ffmpeg([
        os.environ.get('FFMPEG_BINARY','ffmpeg'),
        '-v','error','-nostdin','-y','-i', path,
        '-ac','1','-ar',str(sr),
        '-f','s16le','-acodec','pcm_s16le','-'
    ])
    data = np.frombuffer(cp.stdout, dtype=np.int16)
    # Нормируем в диапазон [-1,1]
    x = (data.astype(np.float32) / 32768.0)
    logger.debug("Decoded audio: %s sr=%d len=%d", path, sr, len(x))
    return x, sr


def standard_convert_to_mp3(wav_path: str, out_dir: str, bitrate: str = "192k") -> Tuple[str,
float]:
    """WAV → MP3 через ffmpeg CLI. Возвращает (путь, время сек)."""
    import time

    ensure_ffmpeg_available()
    base = os.path.splitext(os.path.basename(wav_path))[0]
    out_mp3 = os.path.join(out_dir, f"{base}_standard.mp3")
    t0 = time.perf_counter()
    _run_ffmpeg([
        os.environ.get('FFMPEG_BINARY','ffmpeg'),
        '-v','error','-nostdin','-y','-i', wav_path,
        '-vn','-b:a', bitrate,
        out_mp3,
    ])
    dt = time.perf_counter() - t0
    logger.info("standard_convert_to_mp3 done out=%s dt=%.3f", out_mp3, dt)
    return out_mp3, dt


def _bitrate_to_qscale(bitrate: str) -> int:
    """Грубое сопоставление битрейта к шкале качества LAME (0..9), ниже — лучше.

    Используется для профиля VBR (-q:a). Возвращает 0..9.
    """
    try:
```

```
        s = str(bitrate).strip().lower()
        if s.endswith('k'):
            kb = int(float(s[:-1]))
        else:
            kb = int(float(s))
    except Exception:
        kb = 192
    # Маппинг по типовым значениям
    if kb >= 320:
        return 0
    if kb >= 256:
        return 1
    if kb >= 224:
        return 2
    if kb >= 192:
        return 2
    if kb >= 160:
        return 4
    if kb >= 128:
        return 5
    if kb >= 96:
        return 6
    return 7


def encode_wav_to_mp3(in_wav: str, out_mp3: str, bitrate: str = "192k", *, profile: str = 'cbr',
vbr_quality: int | None = None) -> float:
    """Кодирует WAV → MP3; возвращает время (сек).
    profile: 'cbr' (по умолчанию) или 'vbr' — для VBR используем -q:a (0..9; 0 — лучше всего).
    """
    import time

    ensure_ffmpeg_available()
    t0 = time.perf_counter()
    args = [
        os.environ.get('FFMPEG_BINARY','ffmpeg'),
        '-v','error','-nostdin','-y','-i', in_wav,
        '-vn',
    ]
    prof = (profile or 'cbr').lower()
    if prof == 'vbr':
        q = _bitrate_to_qscale(bitrate) if vbr_quality is None else int(vbr_quality)
        args += ['-q:a', str(max(0, min(9, q)))]
    else:
        args += ['-b:a', bitrate]
    args.append(out_mp3)
    _run_ffmpeg(args)
    dt = time.perf_counter() - t0
    logger.debug("encode_wav_to_mp3 done out=%s dt=%.3f profile=%s", out_mp3, dt, prof)
    return dt


def encode_pcm_to_mp3(pcm: np.ndarray, sample_rate: int, out_mp3: str, bitrate: str = "192k", *,
profile: str = 'vbr', vbr_quality: int | None = None) -> float:
    """Кодирует моно PCM (float32 в диапазоне [-1,1]) напрямую в MP3 через stdin ffmpeg, без
временных WAV.
    Возвращает время кодирования (сек).
    """
    import time
    import subprocess as _sp

    ensure_ffmpeg_available()
    # Подготовка s16le буфера
    x = np.asarray(pcm, dtype=np.float32)
    x = np.clip(x, -1.0, 1.0)
    buf = (np.round(x * 32767.0).astype(np.int16)).tobytes()

    t0 = time.perf_counter()
    args = [
        os.environ.get('FFMPEG_BINARY','ffmpeg'),
        '-v','error','-y',
        '-f','s16le','-ac','1','-ar', str(int(sample_rate)), '-i','pipe:0',
        '-vn',
    ]
    prof = (profile or 'vbr').lower()
    if prof == 'vbr':
        q = _bitrate_to_qscale(bitrate) if vbr_quality is None else int(vbr_quality)
        args += ['-q:a', str(max(0, min(9, q)))]
    else:
        args += ['-b:a', bitrate]
```

```
    args.append(out_mp3)

    # ВАЖНО: без -nostdin, т.к. читаем из pipe:0
    cp = _sp.run(
        args,
        input=buf,
        stdout=_sp.PIPE,
        stderr=_sp.PIPE,
        creationflags=_ffmpeg_creationflags(),
        check=True,
    )
    dt = time.perf_counter() - t0
    logger.debug("encode_pcm_to_mp3 done out=%s dt=%.3f profile=%s size_in=%d", out_mp3, dt,
prof, len(buf))
    return dt


def _sf_bit_depth_from_subtype(subtype: str) -> int:
    """Вернуть битовую глубину для подтипа soundfile; по умолчанию 16 бит."""
    m = {"PCM_U8": 8, "PCM_16": 16, "PCM_24": 24, "PCM_32": 32, "FLOAT": 32, "DOUBLE": 64}
    return m.get(subtype, 16)


def _probe_via_ffmpeg_i(path: str) -> tuple[float,int,int]:
    """Пробинг метаданных через ffprobe: (duration_sec, sample_rate_hz, channels).
    Используем ffprobe с JSON-выводом — устойчиво внутри собранного exe.
    """
    import subprocess, json
    configure_ffmpeg_search()
    # Ищем ffprobe рядом с ffmpeg или в PATH/ENV, читаем JSON и извлекаем поля
    ffprobe_exe = getattr(AudioSegment, 'ffprobe', None) or 'ffprobe'
    try:
        cp = subprocess.run(
            [ffprobe_exe, '-v', 'error', '-print_format', 'json', '-show_streams', '-
show_format', path],
            stdin=subprocess.DEVNULL,
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            creationflags=_ffmpeg_creationflags(),
            check=True,
        )
        info = json.loads(cp.stdout.decode('utf-8', errors='ignore') or '{}')
    except Exception as e:
        logger.debug("ffprobe failed: %s", e)
        return 0.0, 44100, 2
    # Длительность
    dur_sec = 0.0
    try:
        dur_str = (info.get('format') or {}).get('duration')
        if dur_str is not None:
            dur_sec = float(dur_str)
    except Exception:
        dur_sec = 0.0
    # Потоки аудио: берём первый аудио-стрим
    sr = 44100
    ch = 2
    try:
        for st in info.get('streams') or []:
            if (st.get('codec_type') or '').lower() == 'audio':
                try:
                    sr = int(st.get('sample_rate') or sr)
                except Exception as e:
                    logger.debug("sample_rate parse error: %s", e)
                try:
                    ch = int(st.get('channels') or ch)
                except Exception as e:
                    logger.debug("channels parse error: %s", e)
                break
    except Exception as e:
        logger.debug("audio stream parsing failed: %s", e)
    return float(dur_sec or 0.0), int(sr or 44100), int(ch or 2)


def get_audio_meta(path: str) -> Dict[str, int]:
    """Метаданные: sample_rate_hz, bit_depth_bits, channels, bitrate_bps.
    Для несжатых WAV используем soundfile; для прочих — ffprobe JSON.
    """
    # WAV читаем напрямую, остальное — оцениваем через длительность и размер
    ext = os.path.splitext(path)[1].lower()
    try:
```

```
        if ext == ".wav":
            with sf.SoundFile(path) as f:
                sr = int(f.samplerate)
                ch = int(f.channels)
                try:
                    bd = _sf_bit_depth_from_subtype(f.subtype)
                except Exception as e:
                    logger.debug("bit_depth subtype parse error: %s", e)
                    bd = 16
                meta = {"sample_rate_hz": sr, "bit_depth_bits": bd, "channels": ch,
"bitrate_bps": sr * bd * ch}
                logger.debug("WAV meta: %s %s", path, meta)
                return meta
    except Exception as e:
        logger.debug("WAV meta read failed for %s: %s", path, e)
    # Use ffmpeg -i probing for non-wav (works in frozen exe if ffmpeg is configured)
    dur, sr, ch = _probe_via_ffmpeg_i(path)
    size_b = os.path.getsize(path)
    duration_sec = max(1e-6, dur)
    # битовая глубина неизвестна для сжатых форматов — репортим 16 по умолчанию
    bd = 16
    br_bps = int(8.0 * size_b / duration_sec)
    meta = {"sample_rate_hz": int(sr), "bit_depth_bits": int(bd), "channels": int(ch),
"bitrate_bps": int(br_bps)}
    logger.debug("File meta (ffmpeg -i): %s %s", path, meta)
    return meta
```

Б.12. metrics.py

Файл: src/processing/metrics.py (1103 строк)

```
"""
Метрики качества аудио.

Назначение:
- Все функции вычисления метрик качества сгруппированы здесь.
- Сравнение оригинального и обработанного сигналов.
- Расчёт агрегированного балла.

Метрики:
- SNR (дБ): Signal-to-Noise Ratio — выше лучше
- RMSE: Root Mean Square Error — ниже лучше
- SI-SDR (дБ): Scale-Invariant Signal-to-Distortion Ratio — выше лучше
- LSD (дБ): Log-Spectral Distance — ниже лучше
- Spectral Convergence: ошибка амплитуд спектра — ниже лучше
- Spectral Centroid Δ (Гц): разница центров спектра — ниже лучше
- Cosine Similarity: схожесть спектров (0..1) — выше лучше

Внешние библиотеки: numpy, math, logging, os.
"""
from __future__ import annotations

import logging
import math
import os
from typing import Any, Callable, Dict, List, Optional, Tuple, Union

import numpy as np
from numpy.typing import NDArray

logger = logging.getLogger("audio.metrics")

# Type alias для PCM сигналов
PCMSignal = NDArray[np.float32]


# ==============================================================================
# УТИЛИТЫ
# ==============================================================================

def _resample_linear(x: PCMSignal, sr_from: int, sr_to: int) -> PCMSignal:
    """Линейная интерполяция для приведения дискретизации test→reference.

    Алгоритм:
    1) Если частоты совпадают или сигнал пуст — вернуть x в float32 без копии.
    2) Вычислить новую длину new_len = round(len(x)*sr_to/sr_from).
    3) Построить нормализованные оси xp∈[0,1] и xnew∈[0,1] для старых/новых отсчётов.
```

```
    4) Вызвать np.interp(xnew, xp, x) и привести к float32.

    Параметры:
    - x: входной сигнал PCM float32
    - sr_from: исходная частота дискретизации
    - sr_to: целевая частота дискретизации

    Возвращает: ресемплированный сигнал в float32.
    """
    if sr_from == sr_to or len(x) == 0:
        return x.astype(np.float32, copy=False)
    new_len = max(1, int(round(len(x) * float(sr_to) / float(sr_from))))
    xp = np.linspace(0, 1, len(x), endpoint=True)
    xnew = np.linspace(0, 1, new_len, endpoint=True)
    y = np.interp(xnew, xp, x).astype(np.float32)
    return y


# ==============================================================================
# МЕТРИКИ ВРЕМЕННОЙ ОБЛАСТИ
# ==============================================================================

def compute_snr_db(reference: np.ndarray, test: np.ndarray) -> float:
    """Вычислить SNR в дБ между опорным и тестовым сигналами.

    Алгоритм:
    1) Усечь оба сигнала до общей длины N = min(len(ref), len(test)).
    2) Вычислить вектор ошибки noise = ref - test.
    3) Оценить мощности Psig = mean(ref^2) и Pnoise = mean(noise^2) с защитой eps.
    4) Вернуть 10*log10(Psig/Pnoise) в дБ.

    Параметры:
    - reference: опорный PCM float32 [-1,1]
    - test: тестовый PCM float32 [-1,1]

    Возвращает: SNR в дБ (float); выше — лучше.
    """
    n = min(len(reference), len(test))
    if n == 0:
        return float("nan")
    ref = reference[:n]
    tst = test[:n]
    noise = ref - tst
    ref_power = float(np.mean(ref * ref) + 1e-12)
    noise_power = float(np.mean(noise * noise) + 1e-12)
    snr = 10.0 * math.log10(ref_power / noise_power)
    return snr


def compute_rmse(reference: np.ndarray, test: np.ndarray) -> float:
    """RMSE во временной области на общей части сигналов.

    Алгоритм:
    1) Усечь сигналы до общей длины N.
    2) Посчитать вектор ошибки e = ref - test (в float32).
    3) Вернуть sqrt(mean(e^2)).

    Параметры:
    - reference, test: PCM float32 сигналы

    Возвращает: корень из среднего квадрата ошибки; ниже — лучше.
    """
    n = min(len(reference), len(test))
    if n == 0:
        return float("nan")
    err = reference[:n].astype(np.float32) - test[:n].astype(np.float32)
    return float(np.sqrt(np.mean(err * err)))


def compute_si_sdr_db(reference: np.ndarray, test: np.ndarray) -> float:
    """Вычислить SI-SDR (scale-invariant) в дБ.

    Алгоритм:
    1) Усечь до N и привести к float32.
    2) Оценить масштаб alpha = <s,y>/||s||^2 (проекция y на s).
    3) Целевой компонент y_hat = alpha*s; шум e = y - y_hat.
    4) Вернуть 10*log10(||y_hat||^2 / ||e||^2).

    Параметры:
    - reference: опорный сигнал PCM float32
```

```
    - test: тестовый сигнал PCM float32

    Возвращает: SI-SDR в дБ; инвариантен к масштабу; выше — лучше.
    """
    n = min(len(reference), len(test))
    if n == 0:
        return float("nan")
    s = reference[:n].astype(np.float32)
    y = test[:n].astype(np.float32)
    s_energy = float(np.sum(s * s) + 1e-12)
    alpha = float(np.dot(s, y) / s_energy)
    e_target = alpha * s
    e_noise = y - e_target
    num = float(np.sum(e_target * e_target) + 1e-12)
    den = float(np.sum(e_noise * e_noise) + 1e-12)
    return 10.0 * math.log10(num / den)


# ==============================================================================
# МЕТРИКИ СПЕКТРАЛЬНОЙ ОБЛАСТИ
# ==============================================================================

def compute_lsd_db(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
    n_fft: int = 1024,
    hop: int = 512,
) -> float:
    """Log-Spectral Distance (дБ) — средняя по окнам; ниже — лучше.

    Алгоритм:
    1) Привести test к sr_ref линейной интерполяцией.
    2) Если N>len(x): допаддить и посчитать одно окно; иначе — бежать окнами по H.
    3) Для каждой рамки: применить окно, rFFT, взять лог-амплитуды Sa,Sb.
    4) Посчитать RMSE(Sa,Sb) и усреднить по окнам (игнорируя NaN/Inf).

    Параметры:
    - reference: опорный сигнал PCM float32
    - test: тестовый сигнал PCM float32
    - sr_ref: частота дискретизации опорного сигнала
    - sr_test: частота дискретизации тестового сигнала
    - n_fft: размер окна FFT (по умолчанию 1024)
    - hop: шаг окна (по умолчанию 512)

    Возвращает: среднее LSD в дБ; ниже — лучше.
    """
    t = _resample_linear(test, sr_test, sr_ref)
    n = min(len(reference), len(t))
    if n <= 0:
        return float("nan")

    ref = reference[:n].astype(np.float32)
    tst = t[:n].astype(np.float32)
    N = int(n_fft)
    H = int(hop) if hop else N // 2
    if H <= 0:
        H = max(1, N // 2)

    if N > n:
        # Короткий сигнал: одно окно с дополнением
        win = np.hanning(N).astype(np.float32) + 1e-12
        a = np.zeros(N, dtype=np.float32)
        a[:n] = ref * win[:n]
        b = np.zeros(N, dtype=np.float32)
        b[:n] = tst * win[:n]
        A = np.fft.rfft(a)
        B = np.fft.rfft(b)
        Sa = 10.0 * np.log10(np.abs(A) ** 2 + 1e-12)
        Sb = 10.0 * np.log10(np.abs(B) ** 2 + 1e-12)
        d = float(np.sqrt(np.mean((Sa - Sb) ** 2)))
        return d if np.isfinite(d) else float("nan")


    win = np.hanning(N).astype(np.float32) + 1e-12
    frames = max(1, 1 + (n - N) // H)
    lsd_vals = []

    for i in range(frames):
```

```
        s = i * H
        e = s + N
        if e > n:
            e = n
            s = max(0, e - N)
        a = ref[s:e]
        b = tst[s:e]
        if len(a) < N:
            pad = N - len(a)
            a = np.pad(a, (0, pad))
            b = np.pad(b, (0, pad))
        a = a * win
        b = b * win
        A = np.fft.rfft(a)
        B = np.fft.rfft(b)
        Sa = 10.0 * np.log10(np.abs(A) ** 2 + 1e-12)
        Sb = 10.0 * np.log10(np.abs(B) ** 2 + 1e-12)
        d = np.sqrt(np.mean((Sa - Sb) ** 2))
        if np.isfinite(d):
            lsd_vals.append(float(d))

    if not lsd_vals:
        return float("nan")
    return float(np.mean(lsd_vals))


def compute_spectral_convergence(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
    n_fft: int = 1024,
    hop: int = 512,
) -> float:
    """Spectral Convergence: среднее по окнам |||X|-|Y|||_2 / (||X||_2 + eps).

    Параметры:
    - reference, test: PCM float32 сигналы ([-1,1])
    - sr_ref, sr_test: частоты дискретизации; test приводится к sr_ref
    - n_fft: размер окна rFFT
    - hop: шаг между окнами

    Возвращает: среднюю спектральную сходимость; ниже — лучше.
    """
    t = _resample_linear(test, sr_test, sr_ref)
    n = min(len(reference), len(t))
    if n <= 0:
        return float("nan")

    ref = reference[:n].astype(np.float32)
    tst = t[:n].astype(np.float32)
    N = int(n_fft)
    H = int(hop) if hop else N // 2
    if H <= 0:
        H = max(1, N // 2)

    win = np.hanning(N).astype(np.float32)
    vals = []
    i = 0

    while i + N <= n or (i < n and len(vals) == 0):
        s = i
        e = min(n, i + N)
        a = ref[s:e]
        b = tst[s:e]
        if len(a) < N:
            pad = N - len(a)
            a = np.pad(a, (0, pad))
            b = np.pad(b, (0, pad))
        A = np.fft.rfft(a * win)
        B = np.fft.rfft(b * win)
        magA = np.abs(A)
        magB = np.abs(B)
        num = np.linalg.norm(magA - magB)
        den = np.linalg.norm(magA) + 1e-12
        v = float(num / den)
        if np.isfinite(v):
            vals.append(v)
        i += H
```

```
    if not vals:
        return float("nan")
    return float(np.mean(vals))


def compute_spectral_centroid_diff_hz(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
    n_fft: int = 1024,
    hop: int = 512,
) -> float:
    """Средняя абсолютная разница спектрального центроида (в Гц).


    Спектральный центроид — это "центр тяжести" спектра:
    centroid = Σ(k * |X_k|) / Σ|X_k|

    Где k — индекс частотного бина. Центроид измеряется в Гц
    и отражает "яркость" звука:
    - Низкий центроид (~500-2000 Гц): басовые, глухие звуки
    - Высокий центроид (~5000-15000 Гц): яркие, шипящие звуки

    ВАЖНО: Для сигналов с широкополосным шумом центроид может быть
    очень высоким (до 10000+ Гц), так как шум содержит высокочастотные
    компоненты. Это НЕ ошибка, а нормальное поведение метрики.

    Для каждого окна вычисляем центроид Σ(k*|X_k|)/Σ|X_k|,
    затем усредняем |centroid_ref - centroid_test|.

    Параметры:
    - reference, test: PCM float32 сигналы
    - sr_ref, sr_test: частоты дискретизации
    - n_fft: размер окна FFT
    - hop: шаг окна

    Возвращает: среднюю разницу центроидов в Гц; ниже — лучше.
    """
    t = _resample_linear(test, sr_test, sr_ref)
    n = min(len(reference), len(t))
    if n <= 0:
        return float("nan")

    ref = reference[:n].astype(np.float32)
    tst = t[:n].astype(np.float32)
    N = int(n_fft)
    H = int(hop) if hop else N // 2
    if H <= 0:
        H = max(1, N // 2)

    win = np.hanning(N).astype(np.float32)
    df = sr_ref / float(N)
    vals = []
    i = 0

    while i + N <= n or (i < n and len(vals) == 0):
        s = i
        e = min(n, i + N)
        a = ref[s:e]
        b = tst[s:e]
        if len(a) < N:
            pad = N - len(a)
            a = np.pad(a, (0, pad))
            b = np.pad(b, (0, pad))
        A = np.abs(np.fft.rfft(a * win))
        B = np.abs(np.fft.rfft(b * win))
        k = np.arange(len(A), dtype=np.float32)
        ca = float(np.sum(k * A) / (np.sum(A) + 1e-12)) * df
        cb = float(np.sum(k * B) / (np.sum(B) + 1e-12)) * df
        d = abs(ca - cb)
        if np.isfinite(d):
            vals.append(d)
        i += H

    if not vals:
        return float("nan")
    return float(np.mean(vals))
```

```
def compute_spectral_cosine_similarity(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
    n_fft: int = 1024,
    hop: int = 512,
) -> float:
    """Средняя косинусная близость спектров (0..1).

    На каждом окне считаем cos_sim = <|X|, |Y|> / (||X||·||Y||) и усредняем.

    Параметры:
    - reference, test: PCM float32 сигналы
    - sr_ref, sr_test: частоты дискретизации
    - n_fft: размер окна FFT
    - hop: шаг окна

    Возвращает: среднюю косинусную схожесть (0..1); выше — лучше.
    """
    t = _resample_linear(test, sr_test, sr_ref)
    n = min(len(reference), len(t))
    if n <= 0:
        return float("nan")

    ref = reference[:n].astype(np.float32)
    tst = t[:n].astype(np.float32)
    N = int(n_fft)
    H = int(hop) if hop else N // 2
    if H <= 0:
        H = max(1, N // 2)

    win = np.hanning(N).astype(np.float32)
    vals = []
    i = 0


    while i + N <= n or (i < n and len(vals) == 0):
        s = i
        e = min(n, i + N)
        a = ref[s:e]
        b = tst[s:e]
        if len(a) < N:
            pad = N - len(a)
            a = np.pad(a, (0, pad))
            b = np.pad(b, (0, pad))
        A = np.abs(np.fft.rfft(a * win)).astype(np.float32)
        B = np.abs(np.fft.rfft(b * win)).astype(np.float32)
        num = float(np.dot(A, B))
        den = float(np.linalg.norm(A) * np.linalg.norm(B) + 1e-12)
        cs = num / den
        if np.isfinite(cs):
            vals.append(cs)
        i += H

    if not vals:
        return float("nan")
    return float(np.mean(vals))


# ==============================================================================
# ПАКЕТНЫЙ РАСЧЁТ МЕТРИК
# ==============================================================================

def compute_metrics_batch(
    original_wav: str,
    items: List[Tuple[str, str, float]],
    load_wav_func: Callable[[str], Tuple[np.ndarray, int]],
    decode_audio_func: Callable[[str], Tuple[np.ndarray, int]],
    get_meta_func: Callable[[str], Dict[str, int]],
) -> List[Dict]:
    """Посчитать метрики качества для набора результатов.

    Алгоритм:
    1) Пробинг метаданных исходника и загрузка опорного PCM (ref, sr_ref).
    2) Для каждого (variant, path, time):
       2.1) Пробинг метаданных файла (sr, channels, bitrate, size).
       2.2) Декод PCM; расчёт всех метрик.
       2.3) Формирование словаря результата.
    3) Нормировка метрик (min-max) и расчёт агрегированного score.
```

```
    4) Вернуть список результатов.

    Параметры:
    - original_wav: путь к исходному WAV (референс)
    - items: список кортежей (variant, path_to_mp3, time_sec)
    - load_wav_func: функция загрузки WAV (из codecs)
    - decode_audio_func: функция декодирования аудио (из codecs)
    - get_meta_func: функция получения метаданных (из codecs)

    Возвращает: список словарей с полями размера, метрик, времени и score.
    """
    # Метаданные исходника
    orig = get_meta_func(original_wav)
    ref, sr_ref = load_wav_func(original_wav)

    def metrics_for(path: str) -> Tuple[Dict, float, float, float, float, float, float, float]:
        """Вычислить все метрики для одного файла."""
        meta = get_meta_func(path)
        sig, sr = decode_audio_func(path)
        lsd = compute_lsd_db(ref, sig, sr_ref, sr)
        snr = compute_snr_db(ref, sig)
        sc = compute_spectral_convergence(ref, sig, sr_ref, sr)
        rmse = compute_rmse(ref, sig)
        si_sdr = compute_si_sdr_db(ref, sig)
        sc_diff = compute_spectral_centroid_diff_hz(ref, sig, sr_ref, sr)
        cos_sim = compute_spectral_cosine_similarity(ref, sig, sr_ref, sr)
        return meta, float(lsd), float(snr), float(sc), float(rmse), float(si_sdr),
float(sc_diff), float(cos_sim)

    results: List[Dict] = []
    for variant, path, time_s in items:
        meta, lsd, snr, sc, rmse, sisdr, scdiff, cossim = metrics_for(path)

        logger.info(
            "metrics_computed",
            extra={
                "variant": variant,
                "path": path,
                "lsd_db": float(lsd),
                "snr_db": float(snr),
                "spec_conv": float(sc),
                "rmse": float(rmse),
                "si_sdr_db": float(sisdr),
                "spec_centroid_diff_hz": float(scdiff),
                "spec_cosine": float(cossim),
            }
        )

        out = {
            "variant": variant,
            "path": path,
            "size_bytes": os.path.getsize(path),
            "sample_rate_hz": meta["sample_rate_hz"],
            "bit_depth_bits": meta["bit_depth_bits"],
            "bitrate_bps": meta["bitrate_bps"],
            "time_sec": float(time_s),
            "lsd_db": float(lsd),
            "snr_db": float(snr),
            "spec_conv": float(sc),
            "rmse": float(rmse),
            "si_sdr_db": float(sisdr),
            "spec_centroid_diff_hz": float(scdiff),
            "spec_cosine": float(cossim),
            "orig_sample_rate_hz": orig["sample_rate_hz"],
            "orig_bit_depth_bits": orig["bit_depth_bits"],
            "orig_bitrate_bps": orig["bitrate_bps"],
        }
        out["delta_sr"] = out["sample_rate_hz"] - out["orig_sample_rate_hz"]
        out["delta_bd"] = out["bit_depth_bits"] - out["orig_bit_depth_bits"]
        out["delta_br_bps"] = out["bitrate_bps"] - out["orig_bitrate_bps"]
        results.append(out)

    # Агрегированный балл (min-max нормировка)
    def _minmax(vals: List[float]) -> Tuple[Optional[float], Optional[float]]:
        vals_f = [v for v in vals if v == v]  # filter NaN
        if not vals_f:
            return None, None
        return min(vals_f), max(vals_f)

    eps = 1e-12
```

```
    lsd_min, lsd_max = _minmax([r["lsd_db"] for r in results])
    sc_min, sc_max = _minmax([r["spec_conv"] for r in results])
    t_min, t_max = _minmax([r["time_sec"] for r in results])
    snr_min, snr_max = _minmax([r["snr_db"] for r in results])
    rmse_min, rmse_max = _minmax([r.get("rmse") for r in results])
    sisdr_min, sisdr_max = _minmax([r.get("si_sdr_db") for r in results])
    scdiff_min, scdiff_max = _minmax([r.get("spec_centroid_diff_hz") for r in results])
    cos_min, cos_max = _minmax([r.get("spec_cosine") for r in results])

    for r in results:
        # Нормировка: все приводим к "выше-лучше" (1.0 = лучший результат)
        # Для метрик "ниже-лучше" инвертируем: (max - value) / (max - min)
        # Для метрик "выше-лучше" оставляем: (value - min) / (max - min)

        # Метрики "ниже-лучше" (инвертируем):
        lsd_n = 0.0 if lsd_min is None else (lsd_max - r["lsd_db"]) / ((lsd_max - lsd_min) + eps)
        sc_n = 0.0 if sc_min is None else (sc_max - r["spec_conv"]) / ((sc_max - sc_min) + eps)
        rmse_n = 0.0 if rmse_min is None else (rmse_max - r["rmse"]) / ((rmse_max - rmse_min) +
eps)
        scdiff_n = 0.0 if scdiff_min is None else (scdiff_max - r["spec_centroid_diff_hz"]) /
((scdiff_max - scdiff_min) + eps)
        t_n = 0.0 if t_min is None else (t_max - r["time_sec"]) / ((t_max - t_min) + eps)

        # Метрики "выше-лучше" (оставляем как есть):
        snr_n = 0.0 if snr_min is None else (r["snr_db"] - snr_min) / ((snr_max - snr_min) + eps)
        sisdr_n = 0.0 if sisdr_min is None else (r["si_sdr_db"] - sisdr_min) / ((sisdr_max -
sisdr_min) + eps)
        cos_n = 0.0 if cos_min is None else (r["spec_cosine"] - cos_min) / ((cos_max - cos_min) +
eps)

        # Взвешенная сумма (все компоненты приведены к "выше-лучше")
        # Чем выше score, тем лучше метод
        r["score"] = float(
            0.20 * lsd_n +       # LSD: ниже лучше → инвертировано
            0.15 * sc_n +        # Spectral Conv: ниже лучше → инвертировано
            0.20 * snr_n +       # SNR: выше лучше
            0.15 * rmse_n +      # RMSE: ниже лучше → инвертировано
            0.15 * sisdr_n +     # SI-SDR: выше лучше
            0.05 * scdiff_n +    # Centroid Δ: ниже лучше → инвертировано
            0.05 * cos_n +       # Cosine: выше лучше
            0.05 * t_n           # Time: ниже лучше → инвертировано
        )

    return results


# ==============================================================================
# МЕТРИКИ STOI (Short-Time Objective Intelligibility Index)
# ==============================================================================

def compute_stoi(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
    n_fft: int = 512,
    hop: int = 256,
    n_bands: int = 15,
) -> float:
    """Вычислить STOI (Short-Time Objective Intelligibility Index).

    STOI измеряет разборчивость речи путём сравнения кратковременных
    спектральных огибающих опорного и тестового сигналов.

    Диапазон значений: 0.0 - 1.0 (выше лучше).
    - 1.0: идеальное совпадение
    - 0.0: полностью неразборчиво
    - > 0.75: хорошая разборчивость
    - > 0.9: отличная разборчивость

    Алгоритм:
    1) Приведение частот дискретизации
    2) Разбиение на октавные полосы
    3) Вычисление кратковременных огибающих
    4) Корреляция огибающих в каждой полосе
    5) Усреднение корреляций

    Параметры:
    ----------
    reference : np.ndarray
```

```
    Опорный сигнал (чистый)
test : np.ndarray
    Тестовый сигнал (обработанный)
sr_ref : int
    Частота дискретизации опорного сигнала
sr_test : int
    Частота дискретизации тестового сигнала
n_fft : int
    Размер окна FFT (по умолчанию 512)
hop : int
    Шаг окна (по умолчанию 256)
n_bands : int
    Число октавных полос (по умолчанию 15)

Возвращает:
-----------
float
    STOI значение от 0.0 до 1.0

Примечание:
-----------
Это упрощённая реализация. Для полной точности
рекомендуется использовать библиотеку pystoi.
"""
# Ресемплирование
test_resampled = _resample_linear(test, sr_test, sr_ref)
n = min(len(reference), len(test_resampled))

if n < n_fft:
    return float("nan")

ref = reference[:n].astype(np.float64)
tst = test_resampled[:n].astype(np.float64)

# Параметры
N = int(n_fft)
H = int(hop)
n_frames = max(1, 1 + (n - N) // H)

# Создаём окно
win = np.hanning(N)

# Центральные частоты октавных полос (от 150 Гц до ~8 кГц)
f_center = 150.0 * (2.0 ** (np.arange(n_bands) / 3.0))  # 1/3 октавы
f_center = f_center[f_center < sr_ref / 2].astype(np.float64)
n_bands_actual = len(f_center)

if n_bands_actual == 0:
    return float("nan")

# Ширина полос (1/3 октавы)
bandwidth = f_center * (2.0 ** (1.0 / 6.0) - 2.0 ** (-1.0 / 6.0))

# Частотные индексы для каждой полосы
freq_bins = np.fft.rfftfreq(N, 1.0 / sr_ref)

# Вычисляем огибающие для каждого фрейма
env_ref = np.zeros((n_frames, n_bands_actual), dtype=np.float64)
env_tst = np.zeros((n_frames, n_bands_actual), dtype=np.float64)

for frame_idx in range(n_frames):
    start = frame_idx * H
    end = start + N

    if end > n:
        # Дополнение нулями для последнего фрейма
        ref_frame = np.zeros(N)
        tst_frame = np.zeros(N)
        ref_frame[:n - start] = ref[start:]
        tst_frame[:n - start] = tst[start:]
    else:
        ref_frame = ref[start:end]
        tst_frame = tst[start:end]

    # Применяем окно и FFT
    ref_fft = np.abs(np.fft.rfft(ref_frame * win))
    tst_fft = np.abs(np.fft.rfft(tst_frame * win))

    # Вычисляем энергию в каждой полосе
    for band_idx in range(n_bands_actual):
```

```
            f_low = f_center[band_idx] - bandwidth[band_idx] / 2
            f_high = f_center[band_idx] + bandwidth[band_idx] / 2

            # Находим бины в пределах полосы
            mask = (freq_bins >= f_low) & (freq_bins <= f_high)

            if np.any(mask):
                env_ref[frame_idx, band_idx] = np.sqrt(np.mean(ref_fft[mask] ** 2))
                env_tst[frame_idx, band_idx] = np.sqrt(np.mean(tst_fft[mask] ** 2))

    # Нормировка огибающих
    eps = 1e-12
    env_ref = env_ref / (np.max(env_ref, axis=0, keepdims=True) + eps)
    env_tst = env_tst / (np.max(env_tst, axis=0, keepdims=True) + eps)

    # Вычисляем корреляцию огибающих для каждой полосы
    correlations = []

    for band_idx in range(n_bands_actual):
        ref_band = env_ref[:, band_idx]
        tst_band = env_tst[:, band_idx]


        # Центрируем
        ref_centered = ref_band - np.mean(ref_band)
        tst_centered = tst_band - np.mean(tst_band)

        # Корреляция Пирсона
        num = np.sum(ref_centered * tst_centered)
        den = np.sqrt(np.sum(ref_centered ** 2) * np.sum(tst_centered ** 2)) + eps

        corr = num / den
        correlations.append(corr)

    # Усредняем корреляции
    # Примечание: стандартный STOI преобразует [-1,1] в [0,1] через (corr+1)/2
    # Мы сохраняем эту логику для совместимости с диапазоном STOI [0,1]
    mean_corr = np.mean(correlations)
    # Преобразуем из [-1, 1] в [0, 1] - это корректно для STOI
    return float((mean_corr + 1.0) / 2.0)


def compute_stoi_simplified(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
) -> float:
    """Упрощённая версия STOI для быстрой оценки.

    Использует меньшее число полос и фреймов для ускорения.

    Параметры:
    ----------
    reference : np.ndarray
        Опорный сигнал
    test : np.ndarray
        Тестовый сигнал
    sr_ref : int
        Частота дискретизации опорного
    sr_test : int
        Частота дискретизации тестового

    Возвращает:
    -----------
    float
        STOI значение от 0.0 до 1.0
    """
    return compute_stoi(reference, test, sr_ref, sr_test, n_fft=256, hop=128, n_bands=8)


# ============================================================================
# МЕТРИКИ PESQ (Perceptual Evaluation of Speech Quality)
# ============================================================================

def compute_pesq_approx(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
```

```
) -> float:
    """Приближённая оценка PESQ без внешних библиотек.

    PESQ (Perceptual Evaluation of Speech Quality) — стандарт ITU-T P.862
    для оценки качества речи. Диапазон: -0.5 до 4.5 (выше лучше).

    ПРЕДУПРЕЖДЕНИЕ: Это приближённая реализация!
    Для точных результатов используйте официальную реализацию ITU-T
    или библиотеку pesq (pip install pesq).

    Приближение основано на:
    - Взвешенном спектральном расстоянии
    - Модели маскирования
    - Компенсации задержки

    Параметры:
    ----------
    reference : np.ndarray
        Опорный сигнал (чистый)
    test : np.ndarray
        Тестовый сигнал (обработанный)
    sr_ref : int
        Частота дискретизации опорного
    sr_test : int
        Частота дискретизации тестового

    Возвращает:
    -----------
    float
        Приближённое значение PESQ (-0.5 до 4.5)
    """
    # Ресемплирование к 16 кГц (стандарт для PESQ)
    target_sr = 16000

    if sr_ref != target_sr:
        ref = _resample_linear(reference, sr_ref, target_sr)
        sr_ref = target_sr
    else:
        ref = reference.astype(np.float64)

    if sr_test != target_sr:
        tst = _resample_linear(test, sr_test, target_sr)
    else:
        tst = test.astype(np.float64)

    n = min(len(ref), len(tst))
    if n < 1024:
        return float("nan")

    ref = ref[:n]
    tst = tst[:n]

    # Параметры
    n_fft = 512
    hop = 256
    n_frames = max(1, 1 + (n - n_fft) // hop)

    # Психоакустическое взвешивание (A-взвешивание упрощённое)
    freqs = np.fft.rfftfreq(n_fft, 1.0 / sr_ref)

    # Упрощённое A-взвешивание
    f = np.abs(freqs)
    # Обрабатываем f=0 отдельно - DC компонент должен иметь вес ~0
    a_weight = np.zeros_like(f)
    nonzero_mask = f > 0
    if np.any(nonzero_mask):
        f_nonzero = f[nonzero_mask]
        with np.errstate(divide='ignore', invalid='ignore'):
            a_weight_nonzero = (12194**2 * f_nonzero**4) / (
                (f_nonzero**2 + 20.6**2) *
                np.sqrt((f_nonzero**2 + 107.7**2) * (f_nonzero**2 + 737.9**2)) *
                (f_nonzero**2 + 12194**2)
            )
            a_weight[nonzero_mask] = np.nan_to_num(a_weight_nonzero, nan=0.0, posinf=0.0,
neginf=0.0)
    # Нормализация по максимуму (исключая DC)
    max_val = np.max(a_weight[nonzero_mask]) if np.any(nonzero_mask) else 1.0
    a_weight = a_weight / (max_val + 1e-12)

    # Вычисляем спектры
```

```
    win = np.hanning(n_fft)

    distortion_vals = []

    for frame_idx in range(n_frames):
        start = frame_idx * hop
        end = start + n_fft

        if end > n:
            continue

        ref_frame = ref[start:end] * win
        tst_frame = tst[start:end] * win


        # FFT
        ref_fft = np.fft.rfft(ref_frame)
        tst_fft = np.fft.rfft(tst_frame)

        # Амплитуды
        ref_mag = np.abs(ref_fft)
        tst_mag = np.abs(tst_fft)

        # Взвешенное спектральное расстояние
        diff = np.log10((ref_mag + 1e-12) / (tst_mag + 1e-12) + 1e-12)
        weighted_diff = np.abs(diff) * a_weight

        # Усредняем
        frame_dist = np.mean(weighted_diff ** 2)
        distortion_vals.append(frame_dist)

    if not distortion_vals:
        return float("nan")

    # Среднее искажение
    mean_distortion = np.sqrt(np.mean(distortion_vals))

    # Маппинг на шкалу PESQ (эмпирический)
    # PESQ = 4.5 - k * distortion
    # k подобран эмпирически
    k = 3.0
    pesq_approx = 4.5 - k * mean_distortion

    # Ограничение диапазона
    return float(max(-0.5, min(4.5, pesq_approx)))


def compute_pesq_mos(
    reference: np.ndarray,
    test: np.ndarray,
    sr_ref: int,
    sr_test: int,
) -> float:
    """Вычислить PESQ и конвертировать в MOS (Mean Opinion Score).

    MOS — шкала от 1 до 5:
    - 5: Отлично
    - 4: Хорошо
    - 3: Удовлетворительно
    - 2: Плохо
    - 1: Неприемлемо

    Параметры:
    ----------
    reference : np.ndarray
        Опорный сигнал
    test : np.ndarray
        Тестовый сигнал
    sr_ref : int
        Частота дискретизации опорного
    sr_test : int
        Частота дискретизации тестового

    Возвращает:
    -----------
    float
        MOS значение от 1.0 до 5.0
    """
    pesq = compute_pesq_approx(reference, test, sr_ref, sr_test)
```

```
    if np.isnan(pesq):
        return float("nan")

    # Функция отображения PESQ -> MOS (по ITU-T P.862.1)
    # MOS-LQO = 1 + (4.87 - 1) / (1 + exp(-1.3669 * (PESQ - 0.7197)))
    mos = 1.0 + 3.87 / (1.0 + np.exp(-1.3669 * (pesq - 0.7197)))

    return float(max(1.0, min(5.0, mos)))


# ==============================================================================
# ДОПОЛНИТЕЛЬНЫЕ МЕТРИКИ
# ==============================================================================

def compute_spectral_flatness(
    signal: np.ndarray,
    sr: int,
    n_fft: int = 2048,
    hop: int = 512,
) -> float:
    """Вычислить спектральную плоскостность (Wiener entropy).

    Спектральная плоскостность измеряет, насколько спектр сигнала
    похож на белый шум (плоский спектр).

    Диапазон: 0.0 - 1.0
    - Близко к 0: тональный сигнал (музыка, речь)
    - Близко к 1: шумоподобный сигнал

    Параметры:
    ----------
    signal : np.ndarray
        Входной сигнал
    sr : int
        Частота дискретизации
    n_fft : int
        Размер окна FFT
    hop : int
        Шаг окна

    Возвращает:
    -----------
    float
        Средняя спектральная плоскостность
    """
    n = len(signal)
    if n < n_fft:
        return float("nan")

    win = np.hanning(n_fft)
    n_frames = max(1, 1 + (n - n_fft) // hop)

    flatness_vals = []

    for frame_idx in range(n_frames):
        start = frame_idx * hop
        end = start + n_fft

        if end > n:
            break

        frame = signal[start:end] * win
        spectrum = np.abs(np.fft.rfft(frame)) + 1e-12

        # Спектральная плоскостность = geometric_mean / arithmetic_mean
        geometric_mean = np.exp(np.mean(np.log(spectrum)))
        arithmetic_mean = np.mean(spectrum)

        flatness = geometric_mean / (arithmetic_mean + 1e-12)
        flatness_vals.append(flatness)

    if not flatness_vals:
        return float("nan")

    return float(np.mean(flatness_vals))


def compute_dynamic_range(
    signal: np.ndarray,
    percentile_low: float = 5.0,
```

```
    percentile_high: float = 95.0,
) -> float:
    """Вычислить динамический диапазон сигнала в дБ.

    Динамический диапазон — разница между громкими и тихими
    участками сигнала.

    Параметры:
    ----------
    signal : np.ndarray
        Входной сигнал
    percentile_low : float
        Нижний процентиль (тихие участки)
    percentile_high : float
        Верхний процентиль (громкие участки)

    Возвращает:
    -----------
    float
        Динамический диапазон в дБ
    """
    if len(signal) == 0:
        return float("nan")

    # Вычисляем RMS по коротким окнам
    window_size = min(1024, len(signal) // 10)
    if window_size < 2:
        window_size = 2

    rms_values = []
    for i in range(0, len(signal) - window_size, window_size // 2):
        window = signal[i:i + window_size]
        rms = np.sqrt(np.mean(window ** 2))
        rms_values.append(rms)

    if not rms_values:
        return float("nan")

    rms_array = np.array(rms_values)

    # Процентили
    low_val = np.percentile(rms_array, percentile_low) + 1e-12
    high_val = np.percentile(rms_array, percentile_high) + 1e-12

    # Динамический диапазон в дБ
    dynamic_range = 20.0 * np.log10(high_val / low_val)

    return float(dynamic_range)


def compute_crest_factor(signal: np.ndarray) -> float:
    """Вычислить пик-фактор (crest factor) сигнала.

    Пик-фактор — отношение пикового значения к RMS.
    Характеризует "пиковость" сигнала.

    Диапазон: >= 1.0
    - 1.0: синусоида
    - ~3-4: типичная речь/музыка
    - > 10: импульсные сигналы

    Параметры:
    ----------
    signal : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    float
        Пик-фактор
    """
    if len(signal) == 0:
        return float("nan")

    peak = np.max(np.abs(signal))
    rms = np.sqrt(np.mean(signal ** 2)) + 1e-12

    return float(peak / rms)
```

```
# ============================================================================
# ЭКСПОРТ ИМЁН
# ============================================================================

__all__ = [
    # Метрики временной области
    "compute_snr_db",
    "compute_rmse",
    "compute_si_sdr_db",
    # Метрики спектральной области
    "compute_lsd_db",
    "compute_spectral_convergence",
    "compute_spectral_centroid_diff_hz",
    "compute_spectral_cosine_similarity",
    # STOI
    "compute_stoi",
    "compute_stoi_simplified",
    # PESQ
    "compute_pesq_approx",
    "compute_pesq_mos",
    # Дополнительные метрики
    "compute_spectral_flatness",
    "compute_dynamic_range",
    "compute_crest_factor",
    # Пакетный расчёт
    "compute_metrics_batch",
]
```

Б.13. parallel_ola.py

Файл: src/processing/parallel_ola.py (464 строк)

```
"""
Многопоточная OLA (Overlap-Add) обработка.

Назначение:
- Параллельная обработка блоков сигнала
- Ускорение обработки на многоядерных процессорах
- Сохранение обратной совместимости с однопоточным режимом

Алгоритм многопоточной OLA:
===========================
1. Сигнал разбивается на блоки с 50% перекрытием
2. Блоки группируются в чанки по num_threads штук
3. Каждый чанк обрабатывается параллельно в отдельном потоке
4. Результаты собираются и складываются (overlap-add)

Особенности:
-----------
- Thread-safe накопление результатов через блокировки
- Динамическое распределение нагрузки между потоками
- Сохранение порядка блоков для корректного OLA

Использование:
-------------
from processing.parallel_ola import parallel_ola_process

def transform_block(block: np.ndarray) -> np.ndarray:
    # Ваше преобразование
    return np.fft.irfft(np.fft.rfft(block))

result = parallel_ola_process(
    signal=x,
    transform_func=transform_block,
    block_size=2048,
    num_workers=4
)
"""
from __future__ import annotations

import logging
import threading
import time
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass
from typing import Callable, Dict, List, Optional, Tuple, Any
```

```
import numpy as np

logger = logging.getLogger("audio.processing.parallel_ola")


# ============================================================================
# КОНФИГУРАЦИЯ
# ============================================================================

# Минимальное число блоков для параллельной обработки
MIN_BLOCKS_FOR_PARALLEL = 8

# Размер чанка блоков для одного воркера
BLOCKS_PER_CHUNK = 16

# Число воркеров по умолчанию (None = автоопределение)
DEFAULT_NUM_WORKERS = None


# ============================================================================
# РЕЗУЛЬТАТ ОБРАБОТКИ БЛОКА
# ============================================================================

@dataclass
class BlockResult:
    """Результат обработки одного блока.

    Атрибуты:
    ----------
    frame_index : int
        Индекс фрейма (блока)
    start_pos : int
        Начальная позиция в сигнале
    data : np.ndarray
        Обработанные данные блока
    weights : np.ndarray
        Веса окна для OLA
    processing_time : float
        Время обработки блока в секундах
    """
    frame_index: int
    start_pos: int
    data: np.ndarray
    weights: np.ndarray
    processing_time: float = 0.0


# ============================================================================
# ПАРАЛЛЕЛЬНАЯ OLA ОБРАБОТКА
# ============================================================================

def parallel_ola_process(
    signal: np.ndarray,
    transform_func: Callable[[np.ndarray], np.ndarray],
    block_size: int = 2048,
    num_workers: Optional[int] = DEFAULT_NUM_WORKERS,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> np.ndarray:
    """Параллельная OLA обработка сигнала.

    Разбивает сигнал на блоки, обрабатывает параллельно и
    собирает результаты методом overlap-add.

    Параметры:
    ----------
    signal : np.ndarray
        Входной сигнал (моно, float32)
    transform_func : Callable
        Функция преобразования блока (block -> processed_block)
    block_size : int
        Размер блока (степень двойки)
    num_workers : int, optional
        Число рабочих потоков (None = авто)
    progress_cb : Callable, optional
        Колбэк прогресса (fraction, message)

    Возвращает:
    -----------
    np.ndarray
```

```
        Обработанный сигнал

    Пример:
    -------
    >>> def my_transform(block):
    ...     return np.fft.irfft(np.fft.rfft(block))
    >>> result = parallel_ola_process(x, my_transform, block_size=2048)
    """
    n = len(signal)

    # Определяем параметры OLA
    hop = block_size // 2  # 50% перекрытие
    frames = max(1, int(np.ceil(max(0, n - block_size) / hop)) + 1)
    total_len = (frames - 1) * hop + block_size

    # Дополнение сигнала
    pad = total_len - n
    x_padded = np.pad(signal, (0, pad), mode="constant")

    # Создаём окно
    win = np.sqrt(np.hanning(block_size) + 1e-12).astype(np.float32)

    # Определяем, нужно ли параллелить
    use_parallel = frames >= MIN_BLOCKS_FOR_PARALLEL and num_workers != 1

    if use_parallel and num_workers is None:
        num_workers = min(4, max(1, (frames // BLOCKS_PER_CHUNK)))

    if progress_cb:
        progress_cb(0.0, f"OLA: подготовка ({frames} блоков)")

    if use_parallel and num_workers and num_workers > 1:
        result = _process_parallel(
            x_padded, win, hop, frames, block_size, total_len, n,
            transform_func, num_workers, progress_cb
        )
    else:
        result = _process_sequential(
            x_padded, win, hop, frames, block_size, total_len, n,
            transform_func, progress_cb
        )

    return result


def _process_sequential(
    x_padded: np.ndarray,
    win: np.ndarray,
    hop: int,
    frames: int,
    block_size: int,
    total_len: int,
    original_len: int,
    transform_func: Callable,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> np.ndarray:
    """Последовательная OLA обработка (оригинальный алгоритм)."""
    y_accum = np.zeros(total_len, dtype=np.float32)
    w_accum = np.zeros(total_len, dtype=np.float32)

    for fi in range(frames):
        i0 = fi * hop
        block = (x_padded[i0:i0 + block_size] * win).astype(np.float32)

        try:
            processed = transform_func(block)
        except Exception as e:
            logger.warning(f"Transform failed for frame {fi}: {e}")
            processed = block  # Fallback: вернуть исходный блок

        # Проверка размера
        if len(processed) != block_size:
            if len(processed) < block_size:
                processed = np.pad(processed, (0, block_size - len(processed)))
            else:
                processed = processed[:block_size]

        # Накопление
        y_accum[i0:i0 + block_size] += processed * win
        w_accum[i0:i0 + block_size] += win * win
```

```
        if progress_cb and fi % 50 == 0:
            progress_cb(0.1 + 0.85 * fi / frames, f"OLA: блок {fi+1}/{frames}")

    # Финализация
    result = np.divide(y_accum, np.maximum(w_accum, 1e-8))[:original_len]

    # Защита от клиппинга
    peak = np.max(np.abs(result)) + 1e-9
    if peak > 1.0:
        result = result / peak

    if progress_cb:
        progress_cb(1.0, "OLA: завершено")

    return result.astype(np.float32)


def _process_parallel(
    x_padded: np.ndarray,
    win: np.ndarray,
    hop: int,
    frames: int,
    block_size: int,
    total_len: int,
    original_len: int,
    transform_func: Callable,
    num_workers: int,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> np.ndarray:
    """Параллельная OLA обработка с ThreadPoolExecutor."""

    # Буферы для накопления (thread-safe через блокировку)
    y_accum = np.zeros(total_len, dtype=np.float32)
    w_accum = np.zeros(total_len, dtype=np.float32)
    accum_lock = threading.Lock()

    # Счётчик прогресса
    processed_frames = [0]
    progress_lock = threading.Lock()

    def process_block(frame_index: int) -> BlockResult:
        """Обработка одного блока."""
        t_start = time.perf_counter()
        i0 = frame_index * hop


        block = (x_padded[i0:i0 + block_size] * win).astype(np.float32)

        try:
            processed = transform_func(block)
        except Exception as e:
            logger.warning(f"Transform failed for frame {frame_index}: {e}")
            processed = block

        # Проверка размера
        if len(processed) != block_size:
            if len(processed) < block_size:
                processed = np.pad(processed, (0, block_size - len(processed)))
            else:
                processed = processed[:block_size]

        elapsed = time.perf_counter() - t_start

        return BlockResult(
            frame_index=frame_index,
            start_pos=i0,
            data=processed * win,
            weights=win * win,
            processing_time=elapsed
        )

    def accumulate_result(result: BlockResult) -> None:
        """Thread-safe накопление результата."""
        with accum_lock:
            y_accum[result.start_pos:result.start_pos + block_size] += result.data
            w_accum[result.start_pos:result.start_pos + block_size] += result.weights

        # Обновление прогресса
        with progress_lock:
```

```
            processed_frames[0] += 1
            if progress_cb and processed_frames[0] % 50 == 0:
                progress_cb(
                    0.1 + 0.85 * processed_frames[0] / frames,
                    f"OLA: блок {processed_frames[0]}/{frames}"
                )

    if progress_cb:
        progress_cb(0.05, f"OLA: запуск {num_workers} воркеров")

    # Параллельная обработка
    with ThreadPoolExecutor(max_workers=num_workers) as executor:
        futures = {}

        for fi in range(frames):
            future = executor.submit(process_block, fi)
            futures[future] = fi

        for future in as_completed(futures):
            try:
                result = future.result()
                accumulate_result(result)
            except Exception as e:
                fi = futures[future]
                logger.error(f"Frame {fi} failed: {e}")

    # Финализация
    result = np.divide(y_accum, np.maximum(w_accum, 1e-8))[:original_len]

    # Защита от клиппинга
    peak = np.max(np.abs(result)) + 1e-9
    if peak > 1.0:
        result = result / peak

    if progress_cb:
        progress_cb(1.0, "OLA: завершено")

    return result.astype(np.float32)


# ============================================================================
# BATCH PARALLEL PROCESSING
# ============================================================================

def batch_parallel_ola(
    signal: np.ndarray,
    transform_funcs: Dict[str, Callable[[np.ndarray], np.ndarray]],
    block_size: int = 2048,
    num_workers: Optional[int] = None,
    progress_cb: Optional[Callable[[float, str, str], None]] = None,
) -> Dict[str, np.ndarray]:
    """Параллельная обработка сигнала несколькими методами одновременно.

    Полезно для сравнения разных преобразований на одном сигнале.

    Параметры:
    ----------
    signal : np.ndarray
        Входной сигнал
    transform_funcs : Dict[str, Callable]
        Словарь {имя_метода: функция_преобразования}
    block_size : int
        Размер блока
    num_workers : int, optional
        Число воркеров для каждого метода
    progress_cb : Callable, optional
        Колбэк прогресса (fraction, message, method_name)

    Возвращает:
    -----------
    Dict[str, np.ndarray]
        Словарь {имя_метода: обработанный_сигнал}

    Пример:
    -------
    >>> funcs = {
    ...     'fft': lambda b: np.fft.irfft(np.fft.rfft(b)),
    ...     'dct': lambda b: idct(dct(b)),
    ... }
    >>> results = batch_parallel_ola(x, funcs)
```

```
    """
    results = {}
    results_lock = threading.Lock()

    def process_method(name: str, func: Callable) -> Tuple[str, np.ndarray]:
        """Обработка одним методом."""

        def wrapped_progress(frac: float, msg: str):
            if progress_cb:
                progress_cb(frac, msg, name)

        result = parallel_ola_process(
            signal, func, block_size, num_workers, wrapped_progress
        )
        return name, result

    # Определяем число воркеров
    if num_workers is None:
        num_workers = min(2, max(1, len(transform_funcs)))

    with ThreadPoolExecutor(max_workers=len(transform_funcs)) as executor:
        futures = {}

        for name, func in transform_funcs.items():
            future = executor.submit(process_method, name, func)
            futures[future] = name

        for future in as_completed(futures):
            try:
                name, result = future.result()
                with results_lock:
                    results[name] = result
                logger.info(f"Method {name} completed")
            except Exception as e:
                name = futures[future]
                logger.error(f"Method {name} failed: {e}")


    return results


# ============================================================================
# ADAPTIVE PARALLELISM
# ============================================================================

def get_optimal_workers(
    signal_length: int,
    block_size: int = 2048,
    max_workers: int = 8,
) -> int:
    """Определить оптимальное число воркеров для сигнала.

    Учитывает:
    - Длину сигнала (число блоков)
    - Максимальное доступное число потоков
    - Overhead от многопоточности

    Параметры:
    ----------
    signal_length : int
        Длина сигнала в отсчётах
    block_size : int
        Размер блока
    max_workers : int
        Максимальное число воркеров

    Возвращает:
    -----------
    int
        Оптимальное число воркеров
    """
    hop = block_size // 2
    frames = max(1, int(np.ceil(max(0, signal_length - block_size) / hop)) + 1)

    # Мало блоков - не имеет смысла параллелить
    if frames < MIN_BLOCKS_FOR_PARALLEL:
        return 1

    # Определяем по числу блоков
    # Каждый воркер должен обрабатывать минимум BLOCKS_PER_CHUNK блоков
```

```
    optimal = max(1, frames // BLOCKS_PER_CHUNK)

    # Ограничиваем максимумом
    optimal = min(optimal, max_workers)

    return optimal


# ==============================================================================
# ЭКСПОРТ
# ==============================================================================

__all__ = [
    "parallel_ola_process",
    "batch_parallel_ola",
    "get_optimal_workers",
    "BlockResult",
    "MIN_BLOCKS_FOR_PARALLEL",
    "BLOCKS_PER_CHUNK",
    "DEFAULT_NUM_WORKERS",
]
```

Б.14. __init__.py

Файл: src/processing/transforms/__init__.py (321 строк)

```
"""
transforms: Модуль трансформаций аудиосигналов.

Назначение:
- Единая точка импорта всех методов обработки
- Консистентный интерфейс для всех трансформаций
- Подробная документация и аннотации типов

Структура модуля:
=================
- base.py: Базовый класс и общие утилиты OLA
- fft.py: Быстрое преобразование Фурье (FFT)
- dct.py: Дискретное косинусное преобразование (DCT)
- dwt.py: Дискретное вейвлет-преобразование (Хаар)
- fwht.py: Быстрое преобразование Уолша-Адамара (FWHT)
- huffman.py: μ-law компандирование с квантованием
- rosenbrock.py: Нелинейное сглаживающее преобразование

Доступные методы обработки:
===========================
1. Standard MP3 - прямое кодирование (в audio_ops.py)
2. FFT - Быстрое преобразование Фурье
3. DCT - Дискретное косинусное преобразование
4. DWT - Вейвлет Хаара
5. FWHT - Преобразование Уолша-Адамара
6. Huffman-like - μ-law компандирование
7. Rosenbrock-like - Нелинейное сглаживание

Использование:
==============
>>> from processing.transforms import FFTTransform, DCTTransform
>>>
>>> # Через классы
>>> fft = FFTTransform(block_size=2048, select_mode="energy", keep_energy_ratio=0.8)
>>> out_path, time_sec = fft.process("audio.wav", "output/")
>>>
>>> # Через функции-обёртки
>>> from processing.transforms import fft_transform_and_mp3
>>> out_path, time_sec = fft_transform_and_mp3("audio.wav", "output/")

Базовый класс:
==============
>>> from processing.transforms import BaseTransform
>>> help(BaseTransform)

Общие утилиты:
==============
>>> from processing.transforms import (
...     load_audio_safe,
...     create_ola_window,
```

```
...     finalize_ola,
...     SELECT_MODE_NONE,
...     SELECT_MODE_ENERGY,
...     SELECT_MODE_LOWPASS,
... )

Режимы отбора коэффициентов:
============================
- 'none': Без отбора (идеальная реконструкция)
- 'energy': Сохранение доли энергии (сжатие)
- 'lowpass': Сохранение низких частот (фильтрация)
"""
from __future__ import annotations

# ==============================================================================
# БАЗОВЫЙ МОДУЛЬ
# ==============================================================================

from .base import (
    # Базовый класс
    BaseTransform,

    # Константы
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
    VALID_SELECT_MODES,

    # Утилиты
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    get_output_path,
    prepare_ola_buffers,
    select_coefficients_energy,
    select_coefficients_lowpass,
)


# ==============================================================================
# ТРАНСФОРМАЦИИ
# ==============================================================================

# FFT: Быстрое преобразование Фурье
from .fft import (
    FFTTransform,
    fft_transform_and_mp3,
    fft_forward,
    fft_inverse,
    apply_fft_coefficient_selection,
)

# DCT: Дискретное косинусное преобразование
from .dct import (
    DCTTransform,
    dct_transform_and_mp3,
    dct2,
    idct3,
    apply_dct_coefficient_selection,
)

# DWT: Вейвлет Хаара
from .dwt import (
    DWTTransform,
    wavelet_transform_and_mp3,
    haar_dwt_1level,
    haar_idwt_1level,
    dwt_decompose,
    dwt_reconstruct,
    flatten_dwt_coefficients,
    unflatten_dwt_coefficients,
    apply_dwt_coefficient_selection,
)

# FWHT: Преобразование Уолша-Адамара
from .fwht import (
    FWHTTransform,
    fwht_transform_and_mp3,
    fwht_ola,
    fwht,
```

```
    ifwht,
    fwht_ortho,
    ifwht_ortho,
    apply_fwht_coefficient_selection,
)

# Huffman-like: μ-law компандирование
from .huffman import (
    HuffmanLikeTransform,
    huffman_like_transform_and_mp3,
    mulaw_compress,
    mulaw_expand,
    quantize_uniform,
    huffman_like_process,
)

# Rosenbrock-like: Нелинейное сглаживание
from .rosenbrock import (
    RosenbrockLikeTransform,
    rosenbrock_like_transform_and_mp3,
    rosenbrock_nonlinear,
    normalize_peak,
    rosenbrock_process,
)


# ============================================================================
# СПИСОК ВСЕХ ДОСТУПНЫХ ТРАНСФОРМАЦИЙ
# ============================================================================

# Словарь {имя: класс} для динамического выбора метода
TRANSFORM_CLASSES = {
    "fft": FFTTransform,
    "dct": DCTTransform,
    "dwt": DWTTransform,
    "fwht": FWHTTransform,
    "huffman": HuffmanLikeTransform,
    "rosenbrock": RosenbrockLikeTransform,
}

# Словарь {имя: функция} для обратной совместимости
TRANSFORM_FUNCTIONS = {
    "fft": fft_transform_and_mp3,
    "dct": dct_transform_and_mp3,
    "dwt": wavelet_transform_and_mp3,
    "fwht": fwht_transform_and_mp3,
    "huffman": huffman_like_transform_and_mp3,
    "rosenbrock": rosenbrock_like_transform_and_mp3,
}


def get_transform(name: str):
    """Получить класс трансформации по имени.

    Параметры:
    ----------
    name : str
        Имя метода: 'fft', 'dct', 'dwt', 'fwht', 'huffman', 'rosenbrock'

    Возвращает:
    -----------
    type
        Класс трансформации (наследник BaseTransform)

    Исключения:
    -----------
    KeyError
        Если имя не найдено

    Пример:
    -------
    >>> TransformClass = get_transform("fft")
    >>> transform = TransformClass(block_size=2048)
    >>> out_path, time_sec = transform.process("audio.wav", "output/")
    """
    if name.lower() not in TRANSFORM_CLASSES:
        raise KeyError(
            f"Неизвестная трансформация: {name}. "
            f"Доступные: {list(TRANSFORM_CLASSES.keys())}"
        )
```

```
    return TRANSFORM_CLASSES[name.lower()]


def get_transform_function(name: str):
    """Получить функцию-обёртку трансформации по имени.

    Параметры:
    ----------
    name : str
        Имя метода: 'fft', 'dct', 'dwt', 'fwht', 'huffman', 'rosenbrock'

    Возвращает:
    -----------
    callable
        Функция трансформации (wav_path, out_dir, **params) -> (path, time)

    Пример:
    -------
    >>> transform_func = get_transform_function("fwht")
    >>> out_path, time_sec = transform_func("audio.wav", "output/")
    """
    if name.lower() not in TRANSFORM_FUNCTIONS:
        raise KeyError(
            f"Неизвестная трансформация: {name}. "
            f"Доступные: {list(TRANSFORM_FUNCTIONS.keys())}"
        )
    return TRANSFORM_FUNCTIONS[name.lower()]


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    # Базовый класс и константы
    "BaseTransform",
    "SELECT_MODE_NONE",
    "SELECT_MODE_ENERGY",
    "SELECT_MODE_LOWPASS",
    "VALID_SELECT_MODES",

    # Утилиты
    "load_audio_safe",
    "create_ola_window",
    "finalize_ola",
    "get_output_path",
    "prepare_ola_buffers",
    "select_coefficients_energy",
    "select_coefficients_lowpass",

    # Классы трансформаций
    "FFTTransform",
    "DCTTransform",
    "DWTTransform",
    "FWHTTransform",
    "HuffmanLikeTransform",
    "RosenbrockLikeTransform",

    # Функции-обёртки
    "fft_transform_and_mp3",
    "dct_transform_and_mp3",
    "wavelet_transform_and_mp3",
    "fwht_transform_and_mp3",
    "huffman_like_transform_and_mp3",
    "rosenbrock_like_transform_and_mp3",

    # Низкоуровневые функции FFT
    "fft_forward",
    "fft_inverse",
    "apply_fft_coefficient_selection",

    # Низкоуровневые функции DCT
    "dct2",
    "idct3",
    "apply_dct_coefficient_selection",

    # Низкоуровневые функции DWT
    "haar_dwt_1level",
    "haar_idwt_1level",
    "dwt_decompose",
```

```
    "dwt_reconstruct",
    "flatten_dwt_coefficients",
    "unflatten_dwt_coefficients",
    "apply_dwt_coefficient_selection",

    # Низкоуровневые функции FWHT
    "fwht",
    "ifwht",
    "fwht_ortho",
    "ifwht_ortho",
    "fwht_ola",
    "apply_fwht_coefficient_selection",

    # Низкоуровневые функции Huffman
    "mulaw_compress",
    "mulaw_expand",
    "quantize_uniform",
    "huffman_like_process",

    # Низкоуровневые функции Rosenbrock
    "rosenbrock_nonlinear",
    "normalize_peak",
    "rosenbrock_process",

    # Утилиты выбора трансформации
    "TRANSFORM_CLASSES",
    "TRANSFORM_FUNCTIONS",
    "get_transform",
    "get_transform_function",
]
```

Б.15. base.py

Файл: src/processing/transforms/base.py (478 строк)

```
"""
Базовые утилиты и абстракции для модуля трансформаций.

Назначение:
- Общие функции для всех методов обработки (OLA, окна, нормировка)
- Абстрактный базовый класс для единообразного интерфейса трансформаций
- Константы и типы данных

Метод OLA (Overlap-Add):
========================
OLA — стандартный метод обработки сигналов произвольной длины через
перекрывающиеся блоки. При 50% перекрытии и sqrt-Hann окне достигается
идеальная реконструкция (сумма квадратов окон = 1).

Алгоритм:
1. Сигнал разбивается на блоки длины N с перекрытием N/2
2. Каждый блок умножается на окно анализа (sqrt-Hann)
3. Применяется прямое преобразование
4. Применяется обратное преобразование
5. Результат умножается на окно синтеза (то же sqrt-Hann)
6. Блоки складываются с накоплением весов

Математическое обоснование:
- Окно Ханна: w[n] = 0.5 * (1 - cos(2*pi*n/N))
- sqrt-Hann: h[n] = sqrt(w[n])
- При 50% перекрытии: h^2[n] + h^2[n+N/2] = 1

Внешние библиотеки:
- numpy: численные операции
- logging: диагностические сообщения
- abc: абстрактные базовые классы
"""
from __future__ import annotations

import logging
import os
import time
from abc import ABC, abstractmethod
from typing import Callable, Dict, List, Optional, Tuple, Any

import numpy as np
```

```
from ..utils import is_power_of_two, normalize_ratio
from ..codecs import (
    ensure_ffmpeg_available,
    decode_audio_to_mono,
    load_wav_mono,
    encode_pcm_to_mp3,
)


logger = logging.getLogger("audio.processing.transforms")


# ============================================================================
# КОНСТАНТЫ
# ============================================================================

# Режимы отбора коэффициентов
SELECT_MODE_NONE = "none"       # Без отбора (идеальная реконструкция)
SELECT_MODE_ENERGY = "energy"   # По энергии (сжатие)
SELECT_MODE_LOWPASS = "lowpass" # По частоте (фильтрация)

VALID_SELECT_MODES = (SELECT_MODE_NONE, SELECT_MODE_ENERGY, SELECT_MODE_LOWPASS)


# ============================================================================
# УНИВЕРСАЛЬНЫЕ ФУНКЦИИ-УТИЛИТЫ
# ============================================================================

def load_audio_safe(wav_path: str) -> Tuple[np.ndarray, int]:
    """Безопасная загрузка аудио с fallback на soundfile.

    Пытается декодировать через ffmpeg (предпочтительно для exe),
    при ошибке падает на soundfile для локального окружения.

    Параметры:
    ----------
    wav_path : str
        Путь к аудиофайлу (WAV, MP3, FLAC и др.)

    Возвращает:
    -----------
    Tuple[np.ndarray, int]
        (pcm_data, sample_rate) — моно сигнал и частота дискретизации

    Пример:
    -------
    >>> x, sr = load_audio_safe("audio.wav")
    >>> print(f"Длительность: {len(x)/sr:.2f} сек")
    """
    try:
        return decode_audio_to_mono(wav_path)
    except Exception as e:
        logger.debug("decode_audio_to_mono failed (%s), falling back to load_wav_mono", e)
        return load_wav_mono(wav_path)


def create_ola_window(block_size: int) -> np.ndarray:
    """Создать sqrt-Hann окно для OLA с 50% перекрытием.


    Окно sqrt-Hann обеспечивает идеальную реконструкцию при 50% перекрытии:
    h^2[n] + h^2[n+N/2] = 1 для всех n в пределах блока.

    Параметры:
    ----------
    block_size : int
        Длина блока (должна быть степенью двойки)

    Возвращает:
    -----------
    np.ndarray
        Массив длины block_size с окном sqrt-Hann (float32)

    Примечание:
    -----------
    Добавление 1e-12 к окну Ханна предотвращает деление на ноль
    при вычислении квадратного корня.
    """
    return np.sqrt(np.hanning(block_size) + 1e-12).astype(np.float32)
```

```
def finalize_ola(
    y_accum: np.ndarray,
    w_accum: np.ndarray,
    original_len: int
) -> np.ndarray:
    """Завершить OLA: нормировка окна и обрезка до исходной длины.

    Выполняет нормализацию накопленного сигнала по накопленным весам окна,
    затем обрезает до исходной длины и защищает от клиппинга.

    Параметры:
    ----------
    y_accum : np.ndarray
        Накопленный сигнал после сложения всех блоков
    w_accum : np.ndarray
        Накопленные квадраты окна (веса нормализации)
    original_len : int
        Исходная длина сигнала до дополнения

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал длины original_len (float32)

    Примечание:
    -----------
    - Деление на max(w_accum, 1e-8) защищает от деления на ноль
    - Нормировка по пику защищает от клиппинга при кодировании
    """
    y = np.divide(y_accum, np.maximum(w_accum, 1e-8))[:original_len]
    peak = float(np.max(np.abs(y)) + 1e-9)
    if peak > 1.0:
        y = y / peak
    return y


def get_output_path(wav_path: str, out_dir: str, suffix: str) -> str:
    """Сформировать путь к выходному MP3 файлу.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному аудиофайлу
    out_dir : str
        Директория для сохранения результата
    suffix : str
        Суффикс имени (например, 'fft', 'fwht', 'dct')

    Возвращает:
    -----------
    str
        Полный путь к выходному файлу: {out_dir}/{basename}_{suffix}.mp3
    """
    base = os.path.splitext(os.path.basename(wav_path))[0]
    return os.path.join(out_dir, f"{base}_{suffix}.mp3")


def prepare_ola_buffers(
    signal_length: int,
    block_size: int
) -> Tuple[int, int, int, np.ndarray, np.ndarray]:
    """Подготовить буферы для OLA обработки.

    Вычисляет параметры OLA (число фреймов, длина дополнения) и
    создаёт нулевые буферы для накопления сигнала.

    Параметры:
    ----------
    signal_length : int
        Длина исходного сигнала
    block_size : int
        Размер блока (степень двойки)

    Возвращает:
    -----------
    Tuple[int, int, int, np.ndarray, np.ndarray]
        (frames, hop, total_len, y_accum, w_accum)
        - frames: число фреймов
        - hop: шаг между фреймами (N/2 для 50% перекрытия)
```

```
        - total_len: полная длина с дополнением
        - y_accum: буфер накопления сигнала
        - w_accum: буфер накопления весов
    """
    hop = max(1, block_size // 2)  # 50% перекрытие
    frames = max(1, int(np.ceil(max(0, signal_length - block_size) / hop)) + 1)
    total_len = (frames - 1) * hop + block_size
    pad = total_len - signal_length

    y_accum = np.zeros(total_len, dtype=np.float32)
    w_accum = np.zeros(total_len, dtype=np.float32)

    return frames, hop, total_len, y_accum, w_accum


def select_coefficients_energy(
    coeffs: np.ndarray,
    keep_ratio: float
) -> np.ndarray:
    """Отбор коэффициентов по энергии.

    Сохраняет минимальное число наибольших по модулю коэффициентов,
    которые содержат заданную долю энергии сигнала.

    Параметры:
    ----------
    coeffs : np.ndarray
        Массив коэффициентов преобразования
    keep_ratio : float
        Доля энергии для сохранения (0.0-1.0)

    Возвращает:
    -----------
    np.ndarray
        Массив с обнулёнными коэффициентами

    Примечание:
    -----------
    DC-компонента (коэффициент 0) всегда сохраняется,
    так как содержит среднее значение сигнала.
    """
    if keep_ratio >= 1.0:
        return coeffs

    magsq = coeffs * coeffs
    order = np.argsort(magsq)[::-1]  # По убыванию энергии
    cumsum = np.cumsum(magsq[order])
    total_e = cumsum[-1] + 1e-12
    need = keep_ratio * total_e
    keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1

    keep_idx = order[:keep_n]
    mask = np.zeros_like(coeffs, dtype=bool)
    mask[keep_idx] = True
    mask[0] = True  # DC-компонента всегда сохраняется

    return np.where(mask, coeffs, 0.0)


def select_coefficients_lowpass(
    coeffs: np.ndarray,
    keep_ratio: float
) -> np.ndarray:
    """Отбор коэффициентов по частоте (lowpass).

    Сохраняет первые коэффициенты в порядке Уолша/частоты,
    аналогично lowpass-фильтрации в частотной области.

    Параметры:
    ----------
    coeffs : np.ndarray
        Массив коэффициентов преобразования
    keep_ratio : float
        Доля коэффициентов для сохранения (0.0-1.0)

    Возвращает:
    -----------
    np.ndarray
        Массив с обнулёнными высокочастотными коэффициентами
    """
```

```
    if keep_ratio >= 1.0:
        return coeffs

    k_lp = max(1, int(keep_ratio * len(coeffs)))
    mask = np.zeros_like(coeffs, dtype=bool)
    mask[:k_lp] = True

    return np.where(mask, coeffs, 0.0)


# ==============================================================================
# АБСТРАКТНЫЙ БАЗОВЫЙ КЛАСС
# ==============================================================================

class BaseTransform(ABC):
    """Абстрактный базовый класс для всех методов трансформации.

    Определяет единый интерфейс для всех методов обработки аудио:
    - Стандартные параметры (block_size, bitrate, select_mode)
    - Прогресс-колбэки для UI
    - Общий паттерн OLA обработки

    Атрибуты класса:
    ----------------
    NAME : str
        Имя метода для логирования и имён файлов
    DESCRIPTION : str
        Краткое описание метода
    FILE_SUFFIX : str
        Суффикс для выходного файла

    Методы:
    -------
    transform_block : абстрактный
        Применить преобразование к одному блоку
    process : абстрактный
        Полный пайплайн обработки файла

    Пример реализации:
    ------------------
    class FFTTransform(BaseTransform):
        NAME = "FFT"
        DESCRIPTION = "Быстрое преобразование Фурье"
        FILE_SUFFIX = "fft"

        def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
            return np.fft.irfft(np.fft.rfft(block))
    """

    # Атрибуты класса (переопределяются в подклассах)
    NAME: str = "BaseTransform"
    DESCRIPTION: str = "Базовый класс трансформации"
    FILE_SUFFIX: str = "base"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
    ):
        """Инициализация трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока для OLA (степень двойки, обычно 1024-4096)
        bitrate : str
            Битрейт MP3 (например, '192k', '128k')
        select_mode : str
            Режим отбора: 'none', 'energy', 'lowpass'
        keep_energy_ratio : float
            Доля энергии для сохранения (режим 'energy')
        sequency_keep_ratio : float
            Доля низких частот для сохранения (режим 'lowpass')
        """
        self.block_size = int(block_size)
        self.bitrate = str(bitrate)
        self.select_mode = (select_mode or SELECT_MODE_NONE).lower()
```

```
        self.keep_energy_ratio = normalize_ratio(keep_energy_ratio)
        self.sequency_keep_ratio = normalize_ratio(sequency_keep_ratio)

        # Валидация
        if not is_power_of_two(self.block_size):
            raise ValueError(f"block_size должен быть степенью двойки, получено:
{self.block_size}")

        if self.select_mode not in VALID_SELECT_MODES:
            raise ValueError(f"Недопустимый select_mode: {self.select_mode}. Допустимые:
{VALID_SELECT_MODES}")

    @abstractmethod
    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить преобразование к одному блоку.

        Параметры:
        ----------
        block : np.ndarray
            Входной блок сигнала (умноженный на окно)
        **params : dict
            Дополнительные параметры конкретного метода

        Возвращает:
        -----------
        np.ndarray
            Восстановленный блок (той же длины)
        """
        pass

    @abstractmethod
    def process(
        self,
        wav_path: str,
        out_dir: str,
        progress_cb: Optional[Callable[[float, str], None]] = None,
    ) -> Tuple[str, float]:
        """Полный пайплайн обработки файла.

        Параметры:
        ----------
        wav_path : str
            Путь к исходному аудиофайлу
        out_dir : str
            Директория для сохранения результата
        progress_cb : callable, optional
            Колбэк прогресса: progress_cb(fraction: float, message: str)

        Возвращает:
        -----------
        Tuple[str, float]
            (путь к MP3, время обработки в секундах)
        """
        pass

    def get_output_path(self, wav_path: str, out_dir: str) -> str:
        """Сформировать путь к выходному файлу.

        Параметры:
        ----------
        wav_path : str
            Путь к исходному файлу
        out_dir : str
            Директория для сохранения

        Возвращает:
        -----------
        str
            Путь к выходному файлу с суффиксом метода
        """
        return get_output_path(wav_path, out_dir, self.FILE_SUFFIX)

    def log_start(self, wav_path: str, **extra_params) -> None:
        """Залогировать начало обработки.

        Параметры:
        ----------
        wav_path : str
            Путь к обрабатываемому файлу
        **extra_params : dict
```

```
            Дополнительные параметры для логирования
        """
        params_str = " ".join(f"{k}={v}" for k, v in extra_params.items())
        logger.info(f"{self.NAME}_start path={wav_path} {params_str}")

    def log_done(self, out_path: str, time_sec: float) -> None:
        """Залогировать завершение обработки.

        Параметры:
        ----------
        out_path : str
            Путь к выходному файлу
        time_sec : float
            Время обработки в секундах
        """
        logger.info(f"{self.NAME}_done out={out_path} dt={time_sec:.3f}")


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    # Константы
    "SELECT_MODE_NONE",
    "SELECT_MODE_ENERGY",
    "SELECT_MODE_LOWPASS",
    "VALID_SELECT_MODES",
    # Функции-утилиты
    "load_audio_safe",
    "create_ola_window",
    "finalize_ola",
    "get_output_path",
    "prepare_ola_buffers",
    "select_coefficients_energy",
    "select_coefficients_lowpass",
    # Базовый класс
    "BaseTransform",
]
```

Б.16. fft.py

Файл: src/processing/transforms/fft.py (444 строк)

```
"""
FFT: Быстрое преобразование Фурье.

Назначение:
- Частотный анализ и обработка аудиосигналов
- Отбор коэффициентов по энергии или частоте
- Кодирование восстановленного сигнала в MP3

Теоретические основы:
=====================
Преобразование Фурье разлагает сигнал на синусоидальные компоненты.
FFT (Fast Fourier Transform) — быстрый алгоритм вычисления DFT
за O(N log N) операций вместо O(N^2).

rFFT (Real FFT):
----------------
Для действительных сигналов используется rFFT, который вычисляет
только неотрицательные частоты (N/2+1 комплексных коэффициентов
вместо N). Это экономит память и время вычисления.

Формулы:
--------
X[k] = sum_{n=0}^{N-1} x[n] * exp(-j*2*pi*k*n/N)

x[n] = (1/N) * sum_{k=0}^{N-1} X[k] * exp(j*2*pi*k*n/N)

Отбор коэффициентов:
--------------------
1. По энергии (energy): сохраняем k% энергии сигнала
   - Сортируем коэффициенты по |X[k]|^2
   - Оставляем минимальное число, дающее нужную долю энергии

2. Lowpass: сохраняем низкие частоты
```

```
   - Обнуляем коэффициенты выше заданной частоты среза
   - Аналог ФНЧ в частотной области

Параметры:
----------
block_size : int
    Размер блока FFT (степень двойки). Типичные значения:
    - 1024: высокое временное разрешение, низкое частотное
    - 2048: баланс (по умолчанию)
    - 4096: высокое частотное разрешение, низкое временное

select_mode : str
    Режим отбора коэффициентов:
    - 'none': без отбора (идеальная реконструкция)
    - 'energy': сохранение доли энергии (сжатие)
    - 'lowpass': сохранение низких частот (фильтрация)


keep_energy_ratio : float
    Доля энергии для сохранения (режим 'energy'), 0.0-1.0

sequency_keep_ratio : float
    Доля низких частот (режим 'lowpass'), 0.0-1.0

Примеры использования:
----------------------
>>> # Идеальная реконструкция
>>> fft_transform_and_mp3("audio.wav", "output/")

>>> # Сжатие с сохранением 80% энергии
>>> fft_transform_and_mp3("audio.wav", "output/",
...     select_mode="energy", keep_energy_ratio=0.8)

>>> # Lowpass фильтрация (первые 50% частот)
>>> fft_transform_and_mp3("audio.wav", "output/",
...     select_mode="lowpass", sequency_keep_ratio=0.5)

Внешние библиотеки:
-------------------
- numpy.fft: реализация FFT (оптимизирована на C/Fortran)
- logging: диагностические сообщения
"""
from __future__ import annotations

import time
import logging
from typing import Callable, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    prepare_ola_buffers,
    select_coefficients_energy,
    select_coefficients_lowpass,
    get_output_path,
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3
from ..utils import normalize_ratio

logger = logging.getLogger("audio.processing.transforms.fft")



# ============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ============================================================================

def fft_forward(block: np.ndarray) -> np.ndarray:
    """Прямое rFFT преобразование.

    Вычисляет real FFT для действительного сигнала.
    Возвращает N/2+1 комплексных коэффициентов.
```

```
    Параметры:
    ----------
    block : np.ndarray
        Входной блок сигнала (действительные значения)

    Возвращает:
    -----------
    np.ndarray
        Комплексные коэффициенты FFT (N/2+1 элементов)
    """
    return np.fft.rfft(block)


def fft_inverse(coeffs: np.ndarray, n: int) -> np.ndarray:
    """Обратное irFFT преобразование.

    Восстанавливает действительный сигнал из коэффициентов FFT.

    Параметры:
    ----------
    coeffs : np.ndarray
        Комплексные коэффициенты FFT (N/2+1 элементов)
    n : int
        Длина восстанавливаемого сигнала

    Возвращает:
    -----------
    np.ndarray
        Восстановленный действительный сигнал длины n
    """
    return np.fft.irfft(coeffs, n=n).astype(np.float32)


def apply_fft_coefficient_selection(
    coeffs: np.ndarray,
    select_mode: str,
    keep_energy_ratio: float,
    sequency_keep_ratio: float
) -> np.ndarray:
    """Применить отбор коэффициентов FFT.

    Параметры:
    ----------
    coeffs : np.ndarray
        Комплексные коэффициенты FFT
    select_mode : str
        Режим отбора: 'none', 'energy', 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения (режим 'energy')
    sequency_keep_ratio : float
        Доля низких частот (режим 'lowpass')

    Возвращает:
    -----------
    np.ndarray
        Коэффициенты после отбора (с обнулёнными элементами)
    """
    if select_mode == SELECT_MODE_NONE:
        return coeffs

    if select_mode == SELECT_MODE_ENERGY:
        # Для комплексных чисел энергия = |X|^2
        magsq = coeffs.real * coeffs.real + coeffs.imag * coeffs.imag
        order = np.argsort(magsq)[::-1]
        cumsum = np.cumsum(magsq[order])
        total_e = cumsum[-1] + 1e-12
        need = keep_energy_ratio * total_e
        keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1

        keep_idx = order[:keep_n]
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[keep_idx] = True
        mask[0] = True  # DC-компонента всегда сохраняется

        return np.where(mask, coeffs, 0.0+0.0j)

    elif select_mode == SELECT_MODE_LOWPASS:
        k_lp = max(1, int(sequency_keep_ratio * len(coeffs)))
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[:k_lp] = True
```

```
        return np.where(mask, coeffs, 0.0+0.0j)

    return coeffs


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
# ==============================================================================


class FFTTransform(BaseTransform):
    """FFT-трансформация с OLA и отбором коэффициентов.

    Выполняет блочную обработку с перекрытием-сложением (OLA)
    и опциональным отбором коэффициентов по энергии или частоте.

    Атрибуты:
    ---------
    NAME : str
        Имя метода для логирования
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс выходного файла ('fft')

    Пример:
    -------
    >>> transform = FFTTransform(
    ...     block_size=2048,
    ...     select_mode="energy",
    ...     keep_energy_ratio=0.8
    ... )
    >>> out_path, time_sec = transform.process("audio.wav", "output/")
    """

    NAME = "FFT"
    DESCRIPTION = "Быстрое преобразование Фурье"
    FILE_SUFFIX = "fft"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
    ):
        """Инициализация FFT-трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока FFT (степень двойки)
        bitrate : str
            Битрейт MP3
        select_mode : str
            Режим отбора: 'none', 'energy', 'lowpass'
        keep_energy_ratio : float
            Доля энергии для сохранения (режим 'energy')
        sequency_keep_ratio : float
            Доля низких частот (режим 'lowpass')
        """
        super().__init__(
            block_size=block_size,
            bitrate=bitrate,
            select_mode=select_mode,
            keep_energy_ratio=keep_energy_ratio,
            sequency_keep_ratio=sequency_keep_ratio,
        )

    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить FFT к одному блоку.

        Параметры:
        ----------
        block : np.ndarray
            Входной блок (умноженный на окно)

        Возвращает:
        -----------
```

```
    np.ndarray
        Восстановленный блок
    """
    # Прямое FFT
    coeffs = fft_forward(block)

    # Отбор коэффициентов
    coeffs = apply_fft_coefficient_selection(
        coeffs,
        self.select_mode,
        self.keep_energy_ratio,
        self.sequency_keep_ratio,
    )

    # Обратное FFT
    return fft_inverse(coeffs, len(block))

def process(
    self,
    wav_path: str,
    out_dir: str,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Полный пайплайн FFT обработки.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному аудиофайлу
    out_dir : str
        Директория для сохранения MP3
    progress_cb : callable, optional
        Колбэк прогресса progress_cb(frac, msg)

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время обработки в секундах)
    """
    t0_total = time.perf_counter()
    self.log_start(
        wav_path,
        block_size=self.block_size,
        mode=self.select_mode,
        keep_energy=self.keep_energy_ratio,
        seq_keep=self.sequency_keep_ratio,
    )
    ensure_ffmpeg_available()

    if progress_cb:
        progress_cb(0.0, "FFT: декодирование входа")

    # Загрузка аудио
    x, sr = load_audio_safe(wav_path)
    n = len(x)
    N = self.block_size

    # Подготовка OLA
    win = create_ola_window(N)
    frames, hop, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

    # Дополнение сигнала
    pad = total_len - n
    x_padded = np.pad(x, (0, pad), mode="constant")

    # Обработка блоков
    for fi in range(frames):
        i0 = fi * hop
        blk = x_padded[i0 : i0 + N]
        xb = blk * win

        # FFT трансформация
        rec = self.transform_block(xb) * win

        # OLA накопление
        y_accum[i0 : i0 + N] += rec
        w_accum[i0 : i0 + N] += win * win

        if progress_cb:
            progress_cb(
```

```
                    min(0.95, 0.1 + 0.8 * (fi + 1) / frames),
                    f"FFT: блок {fi+1}/{frames}"
                )

        # Финализация OLA
        y = finalize_ola(y_accum, w_accum, n)
        out_mp3 = self.get_output_path(wav_path, out_dir)

        # Кодирование в MP3
        if progress_cb:
            progress_cb(0.97, "FFT: кодирование MP3")
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")

        total_dt = time.perf_counter() - t0_total
        if progress_cb:
            progress_cb(1.0, "FFT: готово")

        self.log_done(out_mp3, total_dt)
        return out_mp3, total_dt


# ============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ============================================================================

def fft_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 2048,
    bitrate: str = "192k",
    select_mode: str = "none",
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Блочная rFFT/iFFT с OLA и отбором коэффициентов → MP3.

    Функция-обёртка над FFTTransform для обратной совместимости.

    Параметры:
    ----------
    wav_path : str
        Входной WAV/аудиофайл
    out_dir : str
        Каталог для сохранения MP3
    block_size : int
        Длина блока (2^n), типичные значения 1024-4096
    bitrate : str
        Битрейт MP3 (например, '192k')
    select_mode : str
        'none' | 'energy' | 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения (mode='energy'), 0..1
    sequency_keep_ratio : float
        Доля низких частот (mode='lowpass'), 0..1
    progress_cb : callable
        Колбэк прогресса progress_cb(frac, msg)

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, общее время в секундах)

    Пример:
    -------
    >>> out_path, time_sec = fft_transform_and_mp3(
    ...     "audio.wav", "output/",
    ...     select_mode="energy",
    ...     keep_energy_ratio=0.8
    ... )
    """
    transform = FFTTransform(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )
    return transform.process(wav_path, out_dir, progress_cb)
```

```
# ============================================================================
# ЭКСПОРТ ИМЁН
# ============================================================================

__all__ = [
    "FFTTransform",
    "fft_transform_and_mp3",
    "fft_forward",
    "fft_inverse",
    "apply_fft_coefficient_selection",
]
```

Б.17. dct.py

Файл: src/processing/transforms/dct.py (501 строк)

```
"""
DCT: Дискретное косинусное преобразование.

Назначение:
- Частотный анализ с использованием только косинусоид
- Энергетическое сжатие сигнала
- Кодирование восстановленного сигнала в MP3

Теоретические основы:
=====================
DCT (Discrete Cosine Transform) разлагает сигнал на косинусоиды.
В отличие от FFT, DCT работает только с действительными числами
и лучше подходит для сжатия, так как концентрирует энергию
в малом числе коэффициентов.

DCT-II (используемая в JPEG, MP3):
----------------------------------
X[k] = sum_{n=0}^{N-1} x[n] * cos(pi*k*(2n+1)/(2N))

Это ортонормированное преобразование с хорошими свойствами
энергетической концентрации.

Реализация без SciPy:
---------------------
DCT-II вычисляется через rFFT чётного отражения сигнала:
1. Создаём массив 2N: y = [x, x_reversed]
2. Вычисляем rFFT(y)
3. Берём первые N коэффициентов с фазовым сдвигом
4. Нормируем для ортонормированности

IDCT-III:
---------
Обратное преобразование, парное к DCT-II.
Вычисляется аналогично через irFFT.

Отбор коэффициентов:
--------------------
Аналогично FFT:
- По энергии: сохраняем k% энергии
- Lowpass: сохраняем низкочастотные компоненты

Параметры:
----------
block_size : int
    Размер блока DCT (степень двойки)

select_mode : str
    Режим отбора: 'none', 'energy', 'lowpass'

keep_energy_ratio : float
    Доля энергии для сохранения (режим 'energy')

sequency_keep_ratio : float
    Доля низких частот (режим 'lowpass')

Примеры:
--------
>>> # Идеальная реконструкция
>>> dct_transform_and_mp3("audio.wav", "output/")
```

```
>>> # Сжатие с сохранением 70% энергии
>>> dct_transform_and_mp3("audio.wav", "output/",
...     select_mode="energy", keep_energy_ratio=0.7)

Внешние библиотеки:
-------------------
- numpy.fft: для вычисления через rFFT
- logging: диагностические сообщения
"""
from __future__ import annotations

import time
import logging
from typing import Callable, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    prepare_ola_buffers,
    get_output_path,
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3
from ..utils import normalize_ratio

logger = logging.getLogger("audio.processing.transforms.dct")


# ==============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ==============================================================================

def dct2(x: np.ndarray) -> np.ndarray:
    """DCT-II (ортонормированная) без SciPy через rFFT чётного отражения.

    Это стандартная DCT, используемая в JPEG и MP3.
    Реализация через rFFT позволяет использовать оптимизированный numpy.

    Параметры:
    ----------
    x : np.ndarray
        Входной массив (действительные значения)

    Возвращает:
    -----------
    np.ndarray
        Коэффициенты DCT-II (float32)

    Алгоритм:
    ---------
    1. Создаём отражённый массив y = [x, x[::-1]]
    2. Вычисляем rFFT(y)
    3. Умножаем на фазовый множитель exp(-j*pi*k/(2N))
    4. Нормируем для ортонормированности

    Примечание:
    -----------
    Коэффициент X[0] (DC) масштабируется отдельно,
    чтобы обеспечить ортонормированность.
    """
    N = int(x.shape[0])
    xr = x.astype(np.float64, copy=False)

    # Чётное отражение
    y = np.empty(2 * N, dtype=np.float64)
    y[:N] = xr
    y[N:] = xr[::-1]

    # FFT отражённого сигнала
    Y = np.fft.rfft(y)

    # Фазовый множитель для DCT-II
    k = np.arange(N, dtype=np.float64)
```

```
    W = np.exp(-1j * np.pi * k / (2.0 * N))

    # Берём первые N коэффициентов и применяем фазу
    C = (Y[:N] * W).real

    # Ортонормировка
    C *= np.sqrt(2.0 / N)
    C[0] /= np.sqrt(2.0)  # DC-компонента

    return C.astype(np.float32)



def idct3(X: np.ndarray) -> np.ndarray:
    """IDCT-III (ортонормированная), парная к dct2.

    Восстанавливает сигнал из коэффициентов DCT-II.

    Параметры:
    ----------
    X : np.ndarray
        Коэффициенты DCT-II

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал (float32)

    Алгоритм:
    ---------
    1. Масштабируем DC-коэффициент
    2. Умножаем на обратный фазовый множитель
    3. Дополняем до N+1 элементов
    4. Вычисляем irFFT
    5. Нормируем
    """
    N = int(X.shape[0])
    Xd = X.astype(np.float64, copy=False).copy()

    # Обратное масштабирование DC
    Xd[0] /= np.sqrt(2.0)

    # Фазовый множитель для IDCT-III
    k = np.arange(N, dtype=np.float64)
    W = np.exp(1j * np.pi * k / (2.0 * N))

    # Комплексный вектор для irFFT
    Z = Xd * W
    H = np.zeros(N + 1, dtype=np.complex128)
    H[:N] = Z
    H[N] = 0.0

    # Обратное FFT
    y = np.fft.irfft(H, n=2 * N)

    # Нормировка
    x = y[:N] * np.sqrt(2.0 / N)

    return x.astype(np.float32)


def apply_dct_coefficient_selection(
    coeffs: np.ndarray,
    select_mode: str,
    keep_energy_ratio: float,
    sequency_keep_ratio: float
) -> np.ndarray:
    """Применить отбор коэффициентов DCT.

    Параметры:
    ----------
    coeffs : np.ndarray
        Коэффициенты DCT (действительные)
    select_mode : str
        Режим отбора: 'none', 'energy', 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения (режим 'energy')
    sequency_keep_ratio : float
        Доля низких частот (режим 'lowpass')
```

```
    Возвращает:
    -----------
    np.ndarray
        Коэффициенты после отбора
    """
    if select_mode == SELECT_MODE_NONE:
        return coeffs

    if select_mode == SELECT_MODE_ENERGY and keep_energy_ratio < 1.0:
        # Энергия = коэффициент^2 (для действительных чисел)
        magsq = coeffs * coeffs
        order = np.argsort(magsq)[::-1]
        cumsum = np.cumsum(magsq[order])
        total_e = cumsum[-1] + 1e-12
        need = keep_energy_ratio * total_e
        keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1

        keep_idx = order[:keep_n]
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[keep_idx] = True
        mask[0] = True  # DC-компонента всегда сохраняется

        return np.where(mask, coeffs, 0.0)

    elif select_mode == SELECT_MODE_LOWPASS and sequency_keep_ratio < 1.0:
        k_lp = max(1, int(sequency_keep_ratio * len(coeffs)))
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[:k_lp] = True
        return np.where(mask, coeffs, 0.0)

    return coeffs


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
# ==============================================================================

class DCTTransform(BaseTransform):
    """DCT-трансформация с OLA и отбором коэффициентов.

    Выполняет блочную обработку DCT-II/IDCT-III с перекрытием-сложением.

    Атрибуты:
    ---------
    NAME : str
        Имя метода для логирования
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс выходного файла ('dct')

    Пример:
    -------
    >>> transform = DCTTransform(
    ...     block_size=2048,
    ...     select_mode="energy",
    ...     keep_energy_ratio=0.8
    ... )
    >>> out_path, time_sec = transform.process("audio.wav", "output/")
    """

    NAME = "DCT"
    DESCRIPTION = "Дискретное косинусное преобразование"
    FILE_SUFFIX = "dct"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
    ):
        """Инициализация DCT-трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока DCT (степень двойки)
```

```
        bitrate : str
            Битрейт MP3
        select_mode : str
            Режим отбора: 'none', 'energy', 'lowpass'
        keep_energy_ratio : float
            Доля энергии для сохранения
        sequency_keep_ratio : float
            Доля низких частот
        """
        super().__init__(
            block_size=block_size,
            bitrate=bitrate,
            select_mode=select_mode,
            keep_energy_ratio=keep_energy_ratio,
            sequency_keep_ratio=sequency_keep_ratio,
        )

    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить DCT к одному блоку.

        Параметры:
        ----------
        block : np.ndarray
            Входной блок (умноженный на окно)

        Возвращает:
        -----------
        np.ndarray
            Восстановленный блок
        """
        # Прямое DCT
        C = dct2(block)

        # Отбор коэффициентов
        C = apply_dct_coefficient_selection(
            C,
            self.select_mode,
            self.keep_energy_ratio,
            self.sequency_keep_ratio,
        )

        # Обратное DCT
        return idct3(C)

    def process(
        self,
        wav_path: str,
        out_dir: str,
        progress_cb: Optional[Callable[[float, str], None]] = None,
    ) -> Tuple[str, float]:
        """Полный пайплайн DCT обработки.

        Параметры:
        ----------
        wav_path : str
            Путь к исходному аудиофайлу
        out_dir : str
            Директория для сохранения MP3
        progress_cb : callable, optional
            Колбэк прогресса

        Возвращает:
        -----------
        Tuple[str, float]
            (путь к MP3, время обработки в секундах)
        """
        t0 = time.perf_counter()
        self.log_start(
            wav_path,
            block_size=self.block_size,
            mode=self.select_mode,
            keep_energy=self.keep_energy_ratio,
            seq_keep=self.sequency_keep_ratio,
        )
        ensure_ffmpeg_available()

        if progress_cb:
            progress_cb(0.0, "DCT: декодирование входа")

        # Загрузка аудио
```

```
        x, sr = load_audio_safe(wav_path)
        n = len(x)
        N = self.block_size

        # Подготовка OLA
        win = create_ola_window(N)
        frames, hop, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

        # Дополнение сигнала
        pad = total_len - n
        x_padded = np.pad(x, (0, pad), mode="constant")

        # Обработка блоков
        for fi in range(frames):
            i0 = fi * hop
            blk = x_padded[i0 : i0 + N]
            xb = blk * win

            # Проверка: нужно ли преобразование?
            identity = (
                self.select_mode == SELECT_MODE_NONE
                and self.keep_energy_ratio >= 1.0
                and self.sequency_keep_ratio >= 1.0
            )

            if identity:
                # Идеальная реконструкция без DCT
                rec = (xb * win).astype(np.float32)
            else:
                # DCT трансформация
                rec = self.transform_block(xb) * win

            # OLA накопление
            y_accum[i0 : i0 + N] += rec
            w_accum[i0 : i0 + N] += win * win

            if progress_cb:
                progress_cb(
                    min(0.95, 0.1 + 0.8 * (fi + 1) / frames),
                    f"DCT: блок {fi+1}/{frames}"
                )

        # Финализация OLA
        y = finalize_ola(y_accum, w_accum, n)
        out_mp3 = self.get_output_path(wav_path, out_dir)

        # Кодирование в MP3
        if progress_cb:
            progress_cb(0.97, "DCT: кодирование MP3")
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")

        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, "DCT: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


# ============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ============================================================================

def dct_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 2048,
    bitrate: str = "192k",
    select_mode: str = "none",
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Блочная DCT-II/IDCT-III с OLA и отбором коэффициентов → MP3.

    Функция-обёртка над DCTTransform для обратной совместимости.

    Параметры:
    ----------
```

```
    wav_path : str
        Входной WAV/аудиофайл
    out_dir : str
        Каталог для сохранения MP3
    block_size : int
        Длина блока (степень двойки)
    bitrate : str
        Битрейт MP3
    select_mode : str
        'none' | 'energy' | 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения, 0..1
    sequency_keep_ratio : float
        Доля низких частот, 0..1
    progress_cb : callable
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    transform = DCTTransform(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    "DCTTransform",
    "dct_transform_and_mp3",
    "dct2",
    "idct3",
    "apply_dct_coefficient_selection",
]
```

Б.18. dwt.py

Файл: src/processing/transforms/dwt.py (613 строк)

```
"""
DWT: Дискретное вейвлет-преобразование (Хаар).

Назначение:
- Многоуровневое частотно-временное разложение сигнала
- Разделение на аппроксимацию (низкие частоты) и детали (высокие)
- Кодирование восстановленного сигнала в MP3

Теоретические основы:
=====================
DWT (Discrete Wavelet Transform) — многоуровневое разложение сигнала,
которое предоставляет информацию как о частотном содержании, так и
о его локализации во времени.

Вейвлет Хаара:
-------------
Простейший вейвлет, использует ступенчатые функции.
На каждом уровне сигнал разделяется на:
- Аппроксимацию (A): сглаженная версия сигнала
- Детали (D): высокочастотные компоненты

Формулы (одноуровневое DWT):
----------------------------
A[k] = (x[2k] + x[2k+1]) / sqrt(2)  — аппроксимация
D[k] = (x[2k] - x[2k+1]) / sqrt(2)  — детали

Обратное преобразование:
x[2k]   = (A[k] + D[k]) / sqrt(2)
x[2k+1] = (A[k] - D[k]) / sqrt(2)
```

```
Многоуровневое DWT:
-------------------
К аппроксимации применяется следующее разложение:
Level 1: A0 → A1, D1
Level 2: A1 → A2, D2
...
Level L: AL-1 → AL, DL

Результат: [AL, DL, DL-1, ..., D2, D1]
где AL — грубая аппроксимация, Di — детали на уровне i.

Отбор коэффициентов:
--------------------
1. По энергии: сохраняем наибольшие по |коэффициент| элементы
2. Lowpass: сохраняем аппроксимацию и первые уровни деталей

Параметры:
----------
block_size : int
    Размер блока (степень двойки)

levels : int
    Число уровней декомпозиции (1-log2(block_size))
    Типичные значения: 4-8

select_mode : str
    Режим отбора: 'none', 'energy', 'lowpass'

Примеры:
--------
>>> # Идеальная реконструкция
>>> wavelet_transform_and_mp3("audio.wav", "output/")

>>> # 6 уровней с сохранением 80% энергии
>>> wavelet_transform_and_mp3("audio.wav", "output/",
...     levels=6, select_mode="energy", keep_energy_ratio=0.8)

Внешние библиотеки:
-------------------
- numpy: численные операции
- logging: диагностические сообщения
"""
from __future__ import annotations

import time
import logging
from typing import Callable, List, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    prepare_ola_buffers,
    get_output_path,
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3
from ..utils import normalize_ratio

logger = logging.getLogger("audio.processing.transforms.dwt")


# ============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ============================================================================

def haar_dwt_1level(x: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """Одноуровневое вейвлет-преобразование Хаара.

    Разделяет сигнал на аппроксимацию (низкие частоты)
    и детали (высокие частоты).

    Параметры:
    ----------
```

```
    x : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    Tuple[np.ndarray, np.ndarray]
        (A, D) — аппроксимация и детали

    Примечание:
    -----------
    При нечётной длине сигнал дополняется нулём.
    """
    n = len(x)
    if n % 2 == 1:
        x = np.pad(x, (0, 1))
        n += 1

    # Аппроксимация: среднее соседних элементов
    a = (x[0:n:2] + x[1:n:2]) / np.sqrt(2.0)

    # Детали: разность соседних элементов
    d = (x[0:n:2] - x[1:n:2]) / np.sqrt(2.0)

    return a.astype(np.float32), d.astype(np.float32)


def haar_idwt_1level(a: np.ndarray, d: np.ndarray, orig_len: int) -> np.ndarray:
    """Обратное одноуровневое преобразование Хаара.

    Восстанавливает сигнал из аппроксимации и деталей.

    Параметры:
    ----------
    a : np.ndarray
        Коэффициенты аппроксимации
    d : np.ndarray
        Коэффициенты деталей
    orig_len : int
        Исходная длина сигнала (для обрезки)

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    n = a.shape[0] + d.shape[0]
    x = np.empty(n, dtype=np.float32)

    # Чётные элементы
    x[0:n:2] = (a + d) / np.sqrt(2.0)
    # Нечётные элементы
    x[1:n:2] = (a - d) / np.sqrt(2.0)

    return x[:orig_len]


def dwt_decompose(x: np.ndarray, levels: int) -> List[np.ndarray]:
    """Многоуровневое DWT разложение.

    Применяет каскадное разложение Хаара.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал
    levels : int
        Число уровней декомпозиции

    Возвращает:
    -----------
    List[np.ndarray]
        Список [AL, DL, DL-1, ..., D1]
        AL — финальная аппроксимация
        Di — детали на уровне i
    """
    coeffs = []
    a = x.astype(np.float32)

    for _ in range(levels):
        a, d = haar_dwt_1level(a)
```

```
        coeffs.append(d)  # Сохраняем детали

    coeffs.append(a)  # Финальная аппроксимация

    return coeffs


def dwt_reconstruct(coeffs: List[np.ndarray], orig_len: int) -> np.ndarray:
    """Многоуровневое DWT восстановление.

    Собирает сигнал из коэффициентов DWT.

    Параметры:
    ----------
    coeffs : List[np.ndarray]
        Список коэффициентов [AL, DL, DL-1, ..., D1]
    orig_len : int
        Исходная длина сигнала

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    # Начинаем с аппроксимации
    a = coeffs[-1].astype(np.float32)

    # Восстанавливаем по уровням
    for i in range(len(coeffs) - 2, -1, -1):
        d = coeffs[i].astype(np.float32)

        # Выравниваем длины при необходимости
        min_len = min(len(a), len(d))
        if len(a) != len(d):
            d = np.pad(d, (0, len(a) - len(d))) if len(d) < len(a) else d[:len(a)]

        a = haar_idwt_1level(a, d, len(a) * 2)

    return a[:orig_len]


def flatten_dwt_coefficients(coeffs: List[np.ndarray]) -> np.ndarray:
    """Собрать коэффициенты DWT в единый вектор.

    Порядок: [AL | DL | DL-1 | ... | D1]

    Параметры:
    ----------
    coeffs : List[np.ndarray]
        Список коэффициентов DWT

    Возвращает:
    -----------
    np.ndarray
        Единый вектор коэффициентов
    """
    # Порядок: аппроксимация, затем детали от последнего к первому уровню
    return np.concatenate(coeffs[::-1])


def unflatten_dwt_coefficients(
    flat: np.ndarray,
    orig_len: int,
    levels: int
) -> List[np.ndarray]:
    """Разбить вектор обратно в список коэффициентов DWT.

    Параметры:
    ----------
    flat : np.ndarray
        Единый вектор коэффициентов
    orig_len : int
        Исходная длина сигнала
    levels : int
        Число уровней декомпозиции

    Возвращает:
    -----------
    List[np.ndarray]
        Список коэффициентов [AL, DL, DL-1, ..., D1]
```

```
    """
    coeffs = []
    ptr = 0

    # Длина аппроксимации
    a_len = int(np.ceil(orig_len / (2 ** levels)))
    # Используем доступные данные, не теряем их
    actual_a_len = min(a_len, max(0, len(flat) - ptr))
    if actual_a_len > 0:
        coeffs.append(flat[ptr:ptr + actual_a_len].astype(np.float32))
        # Если меньше ожидаемого, дополняем нулями
        if actual_a_len < a_len:
            coeffs[0] = np.pad(coeffs[0], (0, a_len - actual_a_len), mode='constant')
    else:
        coeffs.append(np.zeros(a_len, dtype=np.float32))
    ptr += a_len

    # Детали от последнего уровня к первому
    for level in range(levels):
        d_len = int(np.ceil(orig_len / (2 ** (levels - level))))
        # Используем доступные данные
        actual_d_len = min(d_len, max(0, len(flat) - ptr))
        if actual_d_len > 0:
            detail = flat[ptr:ptr + actual_d_len].astype(np.float32)
            # Если меньше ожидаемого, дополняем нулями
            if actual_d_len < d_len:
                detail = np.pad(detail, (0, d_len - actual_d_len), mode='constant')
            coeffs.append(detail)
        else:
            coeffs.append(np.zeros(d_len, dtype=np.float32))
        ptr += d_len

    return coeffs


def apply_dwt_coefficient_selection(
    coeffs: np.ndarray,
    select_mode: str,
    keep_energy_ratio: float,
    sequency_keep_ratio: float
) -> np.ndarray:
    """Применить отбор коэффициентов DWT.

    Параметры:
    ----------
    coeffs : np.ndarray
        Вектор коэффициентов DWT
    select_mode : str
        Режим отбора: 'none', 'energy', 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения
    sequency_keep_ratio : float
        Доля низкочастотных компонент

    Возвращает:
    -----------
    np.ndarray
        Коэффициенты после отбора
    """
    if select_mode == SELECT_MODE_NONE:
        return coeffs

    if select_mode == SELECT_MODE_ENERGY and keep_energy_ratio < 1.0:
        magsq = coeffs * coeffs
        order = np.argsort(magsq)[::-1]
        cumsum = np.cumsum(magsq[order])
        total_e = cumsum[-1] + 1e-12
        need = keep_energy_ratio * total_e
        keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1

        mask = np.zeros_like(coeffs, dtype=bool)
        mask[order[:keep_n]] = True
        mask[0] = True  # DC-компонента

        return np.where(mask, coeffs, 0.0)

    elif select_mode == SELECT_MODE_LOWPASS and sequency_keep_ratio < 1.0:
        k_lp = max(1, int(sequency_keep_ratio * len(coeffs)))
        mask = np.zeros_like(coeffs, dtype=bool)
```

```
        mask[:k_lp] = True
        return np.where(mask, coeffs, 0.0)

    return coeffs


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
# ==============================================================================

class DWTTransform(BaseTransform):
    """DWT (Хаар) трансформация с OLA и многоуровневым разложением.

    Атрибуты:
    ---------
    NAME : str
        Имя метода
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс файла ('dwt')

    Дополнительные параметры:
    -------------------------
    levels : int
        Число уровней декомпозиции
    """

    NAME = "DWT"
    DESCRIPTION = "Дискретное вейвлет-преобразование (Хаар)"
    FILE_SUFFIX = "dwt"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
        levels: int = 4,
    ):
        """Инициализация DWT-трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока (степень двойки)
        bitrate : str
            Битрейт MP3
        select_mode : str
            Режим отбора: 'none', 'energy', 'lowpass'
        keep_energy_ratio : float
            Доля энергии для сохранения
        sequency_keep_ratio : float
            Доля низких частот
        levels : int
            Число уровней декомпозиции (1-12)
        """
        super().__init__(
            block_size=block_size,
            bitrate=bitrate,
            select_mode=select_mode,
            keep_energy_ratio=keep_energy_ratio,
            sequency_keep_ratio=sequency_keep_ratio,
        )

        # Парсинг levels с защитой
        try:
            self.levels = int(max(1, int(levels)))
        except (TypeError, ValueError) as e:
            logger.debug("levels_parse_error: %s, using default 4", e)
            self.levels = 4

    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить DWT к одному блоку.

        Параметры:
        ----------
        block : np.ndarray
            Входной блок
```

```
        Возвращает:
        -----------
        np.ndarray
            Восстановленный блок
        """
        N = len(block)

        # Многоуровневое разложение
        coeffs = dwt_decompose(block, self.levels)

        # Собираем в единый вектор
        flat = flatten_dwt_coefficients(coeffs)

        # Отбор коэффициентов
        flat = apply_dwt_coefficient_selection(
            flat,
            self.select_mode,
            self.keep_energy_ratio,
            self.sequency_keep_ratio,
        )


        # Восстановление
        coeffs_back = unflatten_dwt_coefficients(flat, N, self.levels)
        rec = dwt_reconstruct(coeffs_back, N)

        return rec

    def process(
        self,
        wav_path: str,
        out_dir: str,
        progress_cb: Optional[Callable[[float, str], None]] = None,
    ) -> Tuple[str, float]:
        """Полный пайплайн DWT обработки.

        Параметры:
        ----------
        wav_path : str
            Путь к исходному файлу
        out_dir : str
            Директория для сохранения
        progress_cb : callable, optional
            Колбэк прогресса

        Возвращает:
        -----------
        Tuple[str, float]
            (путь к MP3, время в секундах)
        """
        t0 = time.perf_counter()
        self.log_start(
            wav_path,
            block_size=self.block_size,
            mode=self.select_mode,
            keep_energy=self.keep_energy_ratio,
            seq_keep=self.sequency_keep_ratio,
            levels=self.levels,
        )
        ensure_ffmpeg_available()

        if progress_cb:
            progress_cb(0.0, "DWT: декодирование входа")

        # Загрузка аудио
        x, sr = load_audio_safe(wav_path)
        n = len(x)
        N = self.block_size

        # Подготовка OLA
        win = create_ola_window(N)
        frames, hop, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

        # Дополнение сигнала
        pad = total_len - n
        x_padded = np.pad(x, (0, pad), mode="constant")

        # Обработка блоков
        for fi in range(frames):
```

```
            i0 = fi * hop
            blk = (x_padded[i0 : i0 + N] * win).astype(np.float32)

            # DWT трансформация
            rec = self.transform_block(blk) * win

            # OLA накопление
            y_accum[i0 : i0 + N] += rec
            w_accum[i0 : i0 + N] += win * win

            if progress_cb:
                progress_cb(
                    min(0.95, 0.1 + 0.8 * (fi + 1) / frames),
                    f"DWT: блок {fi+1}/{frames}"
                )

        # Финализация OLA
        y = finalize_ola(y_accum, w_accum, n)
        out_mp3 = self.get_output_path(wav_path, out_dir)

        # Кодирование в MP3
        if progress_cb:
            progress_cb(0.97, "DWT: кодирование MP3")
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")

        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, "DWT: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


# ============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ============================================================================

def wavelet_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 2048,
    bitrate: str = "192k",
    select_mode: str = "none",
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
    levels: int = 4,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Многоуровневое DWT (Хаар) с OLA → MP3.

    Функция-обёртка над DWTTransform для обратной совместимости.

    Параметры:
    ----------
    wav_path : str
        Входной WAV/аудиофайл
    out_dir : str
        Каталог для сохранения MP3
    block_size : int
        Длина блока (степень двойки)
    bitrate : str
        Битрейт MP3
    select_mode : str
        'none' | 'energy' | 'lowpass'
    keep_energy_ratio : float
        Доля энергии, 0..1
    sequency_keep_ratio : float
        Доля низких частот, 0..1
    levels : int
        Число уровней декомпозиции
    progress_cb : callable
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    transform = DWTTransform(
```

```
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
        levels=levels,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    "DWTTransform",
    "wavelet_transform_and_mp3",
    "haar_dwt_1level",
    "haar_idwt_1level",
    "dwt_decompose",
    "dwt_reconstruct",
    "flatten_dwt_coefficients",
    "unflatten_dwt_coefficients",
    "apply_dwt_coefficient_selection",
]
```

Б.19. fwht.py

Файл: src/processing/transforms/fwht.py (652 строк)

```
"""
FWHT: Быстрое преобразование Уолша-Адамара.

Назначение:
- Частотный анализ с использованием прямоугольных волн (функций Уолша)
- Эффективное вычисление (только сложения/вычитания)
- Кодирование восстановленного сигнала в MP3

Теоретические основы:
=====================
FWHT (Fast Walsh-Hadamard Transform) — аналог FFT для функций Уолша.
Функции Уолша — это прямоугольные волны, принимающие значения ±1.

Преимущества FWHT:
------------------
1. Вычислительная простота: только сложения/вычитания, нет умножений
2. O(N log N) операций (как у FFT)
3. Хорошо подходит для бинарных/ступенчатых сигналов
4. Ортонормированное преобразование (энергия сохраняется)

Функции Уолша:
--------------
WAL(k, t) — функция Уолша порядка k:
- Принимает значения +1 или -1
- k — номер функции в последовательности Уолша (sequency)
- Sequency — число пересечений нуля (аналог частоты для синусоид)

Матрица Адамара:
----------------
H = [1  1]   H_2n = [H_n  H_n ]
    [1 -1]          [H_n -H_n ]

Преобразование: y = H_n @ x / sqrt(n)

Алгоритм бабочки:
-----------------
FWHT вычисляется через структуру "бабочка", аналогичную FFT:
На каждом уровне h = 1, 2, 4, ..., N/2:
    a = x[i]
    b = x[i+h]
    x[i]   = a + b
    x[i+h] = a - b

Отбор коэффициентов:
--------------------
1. По энергии: сохраняем k% энергии (аналогично FFT)
2. Lowpass (sequency): сохраняем низкие sequency коэффициенты
```

```
3. Top-k: сохраняем k наибольших по модулю коэффициентов

Производительность:
-------------------
Теоретически FWHT должен быть быстрее FFT (нет умножений),
но numpy FFT оптимизирован на уровне C/Fortran.
Для достижения сопоставимой скорости используется векторизация.

Параметры:
----------
block_size : int
    Размер блока (степень двойки)

select_mode : str
    Режим отбора: 'none', 'energy', 'lowpass'

keep_energy_ratio : float
    Доля энергии для сохранения

sequency_keep_ratio : float
    Доля низких sequency коэффициентов

Примеры:
--------
>>> # Идеальная реконструкция
>>> fwht_transform_and_mp3("audio.wav", "output/")

>>> # Сжатие с сохранением 80% энергии
>>> fwht_transform_and_mp3("audio.wav", "output/",
...     select_mode="energy", keep_energy_ratio=0.8)

Внешние библиотеки:
-------------------
- numpy: векторизованные операции
- math: нормировка sqrt(N)
- logging: диагностические сообщения
"""
from __future__ import annotations

import math
import time
import logging
from typing import Callable, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    prepare_ola_buffers,
    get_output_path,
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3
from ..utils import normalize_ratio, is_power_of_two

logger = logging.getLogger("audio.processing.transforms.fwht")


# ============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ============================================================================

def fwht(x: np.ndarray) -> np.ndarray:
    """Векторизованное быстрое преобразование Уолша–Адамара (FWHT).

    Алгоритм использует бабочку Уолша-Адамара с O(N log N) операциями.
    В отличие от FFT, FWHT использует только сложения и вычитания.

    Параметры:
    ----------
    x : np.ndarray
        Входной массив (длина должна быть степенью двойки)

    Возвращает:
    -----------
```

```
    np.ndarray
        Коэффициенты Уолша-Адамара (без нормировки)

    Алгоритм:
    ---------
    На каждом уровне h = 1, 2, 4, ..., N/2:
        y = reshape(y, (N//(2h), 2, h))
        a = y[:, 0, :]  # первые элементы пар
        b = y[:, 1, :]  # вторые элементы пар
        y[:, 0, :] = a + b  # суммы
        y[:, 1, :] = a - b  # разности

    Примечание:
    -----------
    Векторизованная реализация в ~10-100 раз быстрее
    циклической версии на Python.
    """
    n = x.shape[0]
    if not is_power_of_two(n):
        raise ValueError(f"Длина FWHT должна быть степенью двойки, получено: {n}")

    y = np.asarray(x, dtype=np.float64).copy()


    # Векторизованный алгоритм бабочек
    h = 1
    while h < n:
        # Reshape для векторизации: (n // (2*h), 2, h)
        y = y.reshape(-1, 2, h)
        a = y[:, 0, :].copy()  # первые элементы каждой пары
        b = y[:, 1, :].copy()  # вторые элементы каждой пары
        y[:, 0, :] = a + b     # суммы (верхняя половина бабочки)
        y[:, 1, :] = a - b     # разности (нижняя половина бабочки)
        y = y.reshape(-1)      # обратно в плоский массив
        h *= 2

    return y.astype(np.float32)


def ifwht(x: np.ndarray) -> np.ndarray:
    """Обратное FWHT в небезопасной нормировке: повторный FWHT и деление на N.

    Параметры:
    ----------
    x : np.ndarray
        Коэффициенты FWHT

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    n = x.shape[0]
    return fwht(x) / n


def fwht_ortho(x: np.ndarray) -> np.ndarray:
    """Ортонормированное FWHT (деление на sqrt(N)).

    При ортонормированном преобразовании:
    - Энергия сохраняется: ||y||^2 = ||x||^2
    - Обратное преобразование идентично прямому

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    np.ndarray
        Ортонормированные коэффициенты FWHT
    """
    n = x.shape[0]
    return fwht(x) / math.sqrt(n)


def ifwht_ortho(x: np.ndarray) -> np.ndarray:
    """Обратное ортонормированное FWHT.
```

```
    Для ортонормированного преобразования обратное
    идентично прямому (self-inverse property).

    Параметры:
    ----------
    x : np.ndarray
        Коэффициенты FWHT

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    n = x.shape[0]
    return fwht(x) / math.sqrt(n)


def apply_fwht_coefficient_selection(
    coeffs: np.ndarray,
    select_mode: str,
    keep_energy_ratio: float,
    sequency_keep_ratio: float,
    keep_ratio: Optional[float] = None
) -> np.ndarray:
    """Применить отбор коэффициентов FWHT.

    Параметры:
    ----------
    coeffs : np.ndarray
        Коэффициенты FWHT
    select_mode : str
        Режим отбора: 'none', 'energy', 'lowpass'
    keep_energy_ratio : float
        Доля энергии для сохранения
    sequency_keep_ratio : float
        Доля низких sequency коэффициентов
    keep_ratio : float, optional
        Доля top-k коэффициентов (устаревший параметр)

    Возвращает:
    -----------
    np.ndarray
        Коэффициенты после отбора
    """
    # Устаревший режим top-k
    if keep_ratio is not None and 0.0 < keep_ratio < 1.0:
        N = len(coeffs)
        k = max(1, int(keep_ratio * N))
        thresh = np.partition(np.abs(coeffs), -k)[-k]
        return coeffs * (np.abs(coeffs) >= thresh)

    if select_mode == SELECT_MODE_NONE:
        return coeffs

    if select_mode == SELECT_MODE_ENERGY and keep_energy_ratio < 1.0:
        magsq = coeffs * coeffs
        order = np.argsort(magsq)[::-1]
        cumsum = np.cumsum(magsq[order])
        total_e = cumsum[-1] + 1e-12
        need = keep_energy_ratio * total_e
        keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1

        keep_idx = order[:keep_n]
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[keep_idx] = True
        mask[0] = True  # DC-компонента

        return np.where(mask, coeffs, 0.0)

    elif select_mode == SELECT_MODE_LOWPASS and sequency_keep_ratio < 1.0:
        k_lp = max(1, int(sequency_keep_ratio * len(coeffs)))
        mask = np.zeros_like(coeffs, dtype=bool)
        mask[:k_lp] = True
        return np.where(mask, coeffs, 0.0)

    return coeffs


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
```

```
# ==============================================================================

class FWHTTransform(BaseTransform):
    """FWHT-трансформация с OLA и отбором коэффициентов.

    Атрибуты:
    ---------
    NAME : str
        Имя метода
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс файла ('fwht')


    Пример:
    -------
    >>> transform = FWHTTransform(
    ...     block_size=2048,
    ...     select_mode="energy",
    ...     keep_energy_ratio=0.8
    ... )
    >>> out_path, time_sec = transform.process("audio.wav", "output/")
    """

    NAME = "FWHT"
    DESCRIPTION = "Быстрое преобразование Уолша-Адамара"
    FILE_SUFFIX = "fwht"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
        keep_ratio: float = 0.0,
    ):
        """Инициализация FWHT-трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока (степень двойки)
        bitrate : str
            Битрейт MP3
        select_mode : str
            Режим отбора: 'none', 'energy', 'lowpass'
        keep_energy_ratio : float
            Доля энергии для сохранения
        sequency_keep_ratio : float
            Доля низких sequency коэффициентов
        keep_ratio : float
            Доля top-k (устаревший параметр)
        """
        super().__init__(
            block_size=block_size,
            bitrate=bitrate,
            select_mode=select_mode,
            keep_energy_ratio=keep_energy_ratio,
            sequency_keep_ratio=sequency_keep_ratio,
        )

        # Устаревший параметр keep_ratio
        try:
            self.keep_ratio = float(max(0.0, min(1.0, keep_ratio or 0.0)))
        except (TypeError, ValueError) as e:
            logger.debug("keep_ratio_parse_error: %s, using default 0.0", e)
            self.keep_ratio = 0.0

    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить FWHT к одному блоку.

        Параметры:
        ----------
        block : np.ndarray
            Входной блок

        Возвращает:
        -----------
```

```
    np.ndarray
        Восстановленный блок
    """
    # Прямое FWHT
    coeffs = fwht_ortho(block)

    # Отбор коэффициентов
    coeffs = apply_fwht_coefficient_selection(
        coeffs,
        self.select_mode,
        self.keep_energy_ratio,
        self.sequency_keep_ratio,
        self.keep_ratio,
    )

    # Обратное FWHT
    return ifwht_ortho(coeffs)

def process(
    self,
    wav_path: str,
    out_dir: str,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Полный пайплайн FWHT обработки.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному файлу
    out_dir : str
        Директория для сохранения
    progress_cb : callable, optional
        Колбэк прогресса


    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    t_total0 = time.perf_counter()
    self.log_start(
        wav_path,
        block_size=self.block_size,
        mode=self.select_mode,
        keep_ratio=self.keep_ratio,
        keep_energy=self.keep_energy_ratio,
        seq_keep=self.sequency_keep_ratio,
    )
    ensure_ffmpeg_available()

    if progress_cb:
        progress_cb(0.0, "FWHT: загрузка WAV")
        progress_cb(0.02, "FWHT: декодирование входа")

    # Загрузка аудио
    t_dec0 = time.perf_counter()
    x, sr = load_audio_safe(wav_path)
    t_dec = time.perf_counter() - t_dec0

    n = len(x)
    N = self.block_size

    # Подготовка OLA
    win = create_ola_window(N)
    frames, hop, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

    # Дополнение сигнала
    pad = total_len - n
    x_padded = np.pad(x, (0, pad), mode="constant")

    # Предварительные вычисления для режимов отбора
    use_topk = self.keep_ratio is not None and 0.0 < self.keep_ratio < 1.0

    # Обработка блоков
    t_proc0 = time.perf_counter()
    for fi in range(frames):
        i = fi * hop
        blk = x_padded[i : i + N]
```

```
            blk_w = blk * win

            # FWHT трансформация
            rec = self.transform_block(blk_w) * win

            # OLA накопление
            y_accum[i : i + N] += rec
            w_accum[i : i + N] += win * win

            if progress_cb:
                progress_cb(
                    min(0.95, 0.1 + 0.8 * (fi + 1) / frames),
                    f"FWHT: блок {fi+1}/{frames}"
                )

        # Финализация OLA
        y = finalize_ola(y_accum, w_accum, n)
        t_proc = time.perf_counter() - t_proc0

        out_mp3 = self.get_output_path(wav_path, out_dir)

        # Кодирование в MP3
        if progress_cb:
            progress_cb(0.97, "FWHT: кодирование MP3")

        t_enc0 = time.perf_counter()
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")
        t_enc = time.perf_counter() - t_enc0

        if progress_cb:
            progress_cb(1.0, "FWHT: готово")

        total_dt = time.perf_counter() - t_total0
        logger.info(
            f"fwht_transform_and_mp3 done out={out_mp3} dt={total_dt:.3f} "
            f"t_decode={t_dec:.3f} t_process={t_proc:.3f} t_encode={t_enc:.3f}"
        )

        return out_mp3, total_dt


# ==============================================================================
# ФУНКЦИЯ OLA (для использования без MP3 кодирования)
# ==============================================================================

def fwht_ola(
    x: np.ndarray,
    *,
    block_size: int = 2048,
    window: Optional[np.ndarray] = None,
    select_mode: str = SELECT_MODE_NONE,
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> np.ndarray:
    """Блочная FWHT-обработка с OLA без кодирования в MP3.

    Полезна для промежуточной обработки или анализа.

    Параметры:
    ----------
    x : np.ndarray
        Входной PCM сигнал (float32, [-1, 1])
    block_size : int
        Размер блока (степень двойки)
    window : np.ndarray, optional
        Окно анализа (по умолчанию sqrt-Hann)
    select_mode : str
        Режим отбора: 'none', 'energy', 'lowpass'
    keep_energy_ratio : float
        Доля энергии
    sequency_keep_ratio : float
        Доля низких sequency
    progress_cb : callable
        Колбэк прогресса

    Возвращает:
    -----------
    np.ndarray
        Обработанный сигнал той же длины
```

```
    """
    N = int(block_size)
    if not is_power_of_two(N):
        raise ValueError("block_size должен быть степенью двойки")

    hop = N // 2  # 50% перекрытие

    # Окно
    if window is None:
        window = create_ola_window(N)
    elif len(window) != N:
        raise ValueError("длина окна должна совпадать с block_size")

    n = len(x)
    frames, _, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

    # Дополнение
    pad = total_len - n
    x_padded = np.pad(x, (0, pad), mode="constant")

    for fi in range(frames):
        i = fi * hop
        blk = x_padded[i : i + N]

        # FWHT трансформация
        blk_w = blk * window
        coeffs = fwht_ortho(blk_w)

        # Отбор
        coeffs = apply_fwht_coefficient_selection(
            coeffs, select_mode, keep_energy_ratio, sequency_keep_ratio
        )

        rec = ifwht_ortho(coeffs) * window

        # OLA
        y_accum[i : i + N] += rec
        w_accum[i : i + N] += window * window

        if progress_cb:
            progress_cb(min(0.95, (fi + 1) / frames), f"FWHT: блок {fi+1}/{frames}")

    # Нормировка
    y = np.divide(y_accum, np.maximum(w_accum, 1e-8))[:n]

    # Защита от клиппинга
    peak = float(np.max(np.abs(y)) + 1e-9)
    if peak > 1.0:
        y = y / peak

    if progress_cb:
        progress_cb(1.0, "FWHT: готово")

    logger.debug("fwht_ola done n=%d N=%d frames=%d", n, N, frames)
    return y


# ============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ============================================================================

def fwht_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    block_size: int = 2048,
    keep_ratio: float = 0.0,
    bitrate: str = "192k",
    progress_cb: Optional[Callable[[float, str], None]] = None,
    keep_energy_ratio: float = 1.0,
    select_mode: str = "none",
    sequency_keep_ratio: float = 1.0,
) -> Tuple[str, float]:
    """FWHT → MP3 с блочной OLA.

    Функция-обёртка над FWHTTransform для обратной совместимости.


    Параметры:
    ----------
    wav_path : str
```

```
        Путь к исходному файлу
    out_dir : str
        Директория для вывода
    block_size : int
        Размер блока (степень двойки)
    keep_ratio : float
        Доля коэффициентов top-k (устаревший)
    bitrate : str
        Битрейт MP3
    progress_cb : callable
        Колбэк прогресса
    keep_energy_ratio : float
        Доля энергии (mode='energy')
    select_mode : str
        'none' | 'energy' | 'lowpass'
    sequency_keep_ratio : float
        Доля низких sequency (mode='lowpass')

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    transform = FWHTTransform(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
        keep_ratio=keep_ratio,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ============================================================================
# ЭКСПОРТ ИМЁН
# ============================================================================

__all__ = [
    "FWHTTransform",
    "fwht_transform_and_mp3",
    "fwht_ola",
    "fwht",
    "ifwht",
    "fwht_ortho",
    "ifwht_ortho",
    "apply_fwht_coefficient_selection",
]
```

Б.20. huffman.py

Файл: src/processing/transforms/huffman.py (426 строк)

```
"""
Huffman-like: μ-law компандирование с квантованием.

Назначение:
- Нелинейное сжатие динамического диапазона
- Имитация предобработки для энтропийного кодирования
- Кодирование обработанного сигнала в MP3

Теоретические основы:
=====================
Данный метод не является истинным кодированием Хаффмана, но
имитирует подготовительный этап для энтропийного кодирования
через μ-law компандирование.

μ-law компандирование:
----------------------
Нелинейное преобразование, которое сжимает динамический диапазон
сигнала по логарифмическому закону. Широко используется в
телефонии (Северная Америка, Япония).

Формулы:
--------
Прямое преобразование:
y = sign(x) * ln(1 + μ|x|) / ln(1 + μ)
```

```
где μ — параметр сжатия (обычно 255 для телефонии)

Обратное преобразование:
x = sign(y) * ((1 + μ)^|y| - 1) / μ

Свойства:
- Большие значения сжимаются меньше
- Малые значения сжимаются больше
- Улучшается отношение сигнал/шум для тихих звуков

Квантование:
------------
После μ-law преобразования применяется равномерное квантование:
- Сигнал масштабируется в диапазон [0, Q-1]
- Округляется до целого
- Восстанавливается в [-1, 1]

Число уровней Q = 2^bits (bits = 8-16 типично)

Параметры:
----------
mu : float
    Параметр μ-law (1-255, больше = сильнее сжатие)
    - μ = 1: линейное преобразование
    - μ = 255: стандартное телефонное компандирование

bits : int
    Число бит квантования (1-16)
    - 8 бит: 256 уровней (телефония)
    - 16 бит: 65536 уровней (высокое качество)

block_size : int
    Размер блока (для совместимости с OLA, но не используется)

Примеры:
--------
>>> # Стандартное телефонное качество
>>> huffman_like_transform_and_mp3("audio.wav", "output/", mu=255, bits=8)

>>> # Высокое качество
>>> huffman_like_transform_and_mp3("audio.wav", "output/", mu=100, bits=14)

Внешние библиотеки:
-------------------
- numpy: численные операции
- logging: диагностические сообщения
"""
from __future__ import annotations

import time
import logging
from typing import Callable, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    get_output_path,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3

logger = logging.getLogger("audio.processing.transforms.huffman")


# ============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ============================================================================

def mulaw_compress(x: np.ndarray, mu: float = 255.0) -> np.ndarray:
    """Прямое μ-law компандирование.

    Сжимает динамический диапазон по логарифмическому закону.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал в диапазоне [-1, 1]
    mu : float
```

```
        Параметр сжатия (обычно 255)

    Возвращает:
    -----------
    np.ndarray
        Сжатый сигнал в диапазоне [-1, 1]

    Формула:
    --------
    y = sign(x) * ln(1 + μ|x|) / ln(1 + μ)
    """
    return np.sign(x) * (np.log1p(mu * np.abs(x)) / np.log1p(mu))


def mulaw_expand(y: np.ndarray, mu: float = 255.0) -> np.ndarray:
    """Обратное μ-law преобразование.

    Восстанавливает сигнал из сжатого представления.

    Параметры:
    ----------
    y : np.ndarray
        Сжатый сигнал в диапазоне [-1, 1]
    mu : float
        Параметр сжатия (должен совпадать с прямым преобразованием)

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал

    Формула:
    --------
    x = sign(y) * ((1 + μ)^|y| - 1) / μ
    """
    return np.sign(y) * ((1.0 + mu) ** np.abs(y) - 1.0) / mu


def quantize_uniform(x: np.ndarray, bits: int = 8) -> Tuple[np.ndarray, int]:
    """Равномерное квантование.

    Преобразует непрерывный сигнал в дискретный с заданным
    числом уровней.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал в диапазоне [-1, 1]
    bits : int
        Число бит (определяет число уровней Q = 2^bits)

    Возвращает:
    -----------
    Tuple[np.ndarray, int]
        (квантованный сигнал, число уровней Q)

    Примечание:
    -----------
    Квантование вносит шум, который тем меньше,
    чем больше bits.
    """
    Q = max(2, int(2 ** bits))

    # Масштабирование в [0, Q-1]
    xi = np.clip((x + 1.0) * 0.5 * (Q - 1), 0, Q - 1).astype(np.int32)

    # Обратное масштабирование в [-1, 1]
    x_rec = (xi.astype(np.float32) / (Q - 1)) * 2.0 - 1.0

    return x_rec, Q


def huffman_like_process(
    x: np.ndarray,
    mu: float = 255.0,
    bits: int = 8
) -> np.ndarray:
    """Полный цикл Huffman-like обработки.

    Последовательность:
```

```
    1. μ-law компандирование (сжатие динамического диапазона)
    2. Равномерное квантование (внесение шума квантования)
    3. Обратное μ-law (восстановление)

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал
    mu : float
        Параметр μ-law
    bits : int
        Число бит квантования

    Возвращает:
    -----------
    np.ndarray
        Обработанный сигнал
    """
    # Прямое μ-law
    x_mu = mulaw_compress(x, mu)

    # Квантование
    x_q, _ = quantize_uniform(x_mu, bits)

    # Обратное μ-law
    return mulaw_expand(x_q, mu)


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
# ==============================================================================

class HuffmanLikeTransform(BaseTransform):
    """Huffman-like трансформация (μ-law + квантование).

    Имитирует предобработку для энтропийного кодирования
    через нелинейное компандирование.

    Атрибуты:
    ---------
    NAME : str
        Имя метода
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс файла ('huffman')

    Дополнительные параметры:
    -------------------------
    mu : float
        Параметр μ-law компандирования
    bits : int
        Число бит квантования
    """

    NAME = "Huffman"
    DESCRIPTION = "μ-law компандирование с квантованием"
    FILE_SUFFIX = "huffman"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        mu: float = 255.0,
        bits: int = 8,
        select_mode: str = "none",
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
    ):
        """Инициализация Huffman-like трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока (для совместимости, не используется)
        bitrate : str
            Битрейт MP3
        mu : float
            Параметр μ-law (1-255)
        bits : int
```

```
        Число бит квантования (1-16)
    """
    super().__init__(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )

    # Параметр μ
    try:
        self.mu = float(max(1.0, mu))
    except (TypeError, ValueError) as e:
        logger.debug("mu_parse_error: %s, using default 255.0", e)
        self.mu = 255.0

    # Число бит
    try:
        self.bits = int(max(1, min(16, int(bits))))
    except (TypeError, ValueError) as e:
        logger.debug("bits_parse_error: %s, using default 8", e)
        self.bits = 8

def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
    """Применить Huffman-like обработку к блоку.

    Примечание: для этого метода OLA не используется,
    обработка применяется ко всему сигналу целиком.

    Параметры:
    ----------
    block : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    np.ndarray
        Обработанный сигнал
    """
    return huffman_like_process(block, self.mu, self.bits)

def process(
    self,
    wav_path: str,
    out_dir: str,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Полный пайплайн Huffman-like обработки.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному файлу
    out_dir : str
        Директория для сохранения
    progress_cb : callable, optional
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    t0 = time.perf_counter()
    self.log_start(wav_path, mu=self.mu, bits=self.bits)
    ensure_ffmpeg_available()

    if progress_cb:
        progress_cb(0.0, "Huffman: декодирование входа")

    # Загрузка аудио
    x, sr = load_audio_safe(wav_path)

    if progress_cb:
        progress_cb(0.3, "Huffman: μ-law компандирование")

    # Huffman-like обработка
    y = self.transform_block(x)
```

```
        if progress_cb:
            progress_cb(0.7, "Huffman: кодирование MP3")

        out_mp3 = self.get_output_path(wav_path, out_dir)
        encode_pcm_to_mp3(y.astype(np.float32), sr, out_mp3, self.bitrate, profile="vbr")


        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, "Huffman: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


# ==============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ==============================================================================

def huffman_like_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 2048,
    bitrate: str = "192k",
    mu: float = 255.0,
    bits: int = 8,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Huffman-like (μ-law + квантование) → MP3.

    Функция-обёртка над HuffmanLikeTransform для обратной совместимости.

    Параметры:
    ----------
    wav_path : str
        Входной WAV/аудиофайл
    out_dir : str
        Каталог для сохранения MP3
    block_size : int
        Размер блока (не используется)
    bitrate : str
        Битрейт MP3
    mu : float
        Параметр μ-law (1-255)
    bits : int
        Число бит квантования (1-16)
    progress_cb : callable
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)

    Пример:
    -------
    >>> out_path, time_sec = huffman_like_transform_and_mp3(
    ...     "audio.wav", "output/", mu=255, bits=8
    ... )
    """
    transform = HuffmanLikeTransform(
        block_size=block_size,
        bitrate=bitrate,
        mu=mu,
        bits=bits,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ==============================================================================
# ЭКСПОРТ ИМЁН
# ==============================================================================

__all__ = [
    "HuffmanLikeTransform",
    "huffman_like_transform_and_mp3",
    "mulaw_compress",
    "mulaw_expand",
    "quantize_uniform",
```

```
    "huffman_like_process",
]
```

Б.21. rosenbrock.py

Файл: src/processing/transforms/rosenbrock.py (383 строк)

```
"""
Rosenbrock-like: Нелинейное сглаживающее преобразование.

Назначение:
- Эвристическое сглаживание аудиосигнала
- Мягкое сжатие динамики
- Кодирование обработанного сигнала в MP3

Теоретические основы:
====================
Данный метод назван "Rosenbrock-like" по аналогии с функцией Розенброка,
используемой в оптимизации. Однако здесь применяется простое нелинейное
сглаживающее преобразование.

Преобразование:
---------------
y = x / (1 + α(x - β)²)

где:
- x — входной сигнал
- α (alpha) — параметр сглаживания
- β (beta) — параметр сдвига

Свойства преобразования:
------------------------
1. При α = 0: y = x (тождественное преобразование)
2. При α > 0: сигнал сглаживается около точки β
3. Чем больше α, тем сильнее сжатие динамики

Графическая интерпретация:
--------------------------
Преобразование создаёт "седловину" в точке x = β:
- При x ≈ β: y ≈ x (мало изменений)
- При x >> β или x << β: y < x (сжатие)

Это создаёт эффект мягкого лимитера/компрессора.

Параметры:
----------
alpha : float
    Параметр сглаживания (0-10)
    - α = 0: без изменений
    - α = 0.2: мягкое сжатие (по умолчанию)
    - α = 1-10: сильное сжатие

beta : float
    Параметр сдвига (-5...+5)
    - β = 0: симметричное сжатие вокруг нуля
    - β > 0: асимметрия в сторону положительных значений
    - β < 0: асимметрия в сторону отрицательных значений

block_size : int
    Размер блока (для совместимости, не используется)

Примеры:
--------
>>> # Мягкое сглаживание
>>> rosenbrock_like_transform_and_mp3("audio.wav", "output/", alpha=0.2, beta=1.0)

>>> # Сильное сжатие
>>> rosenbrock_like_transform_and_mp3("audio.wav", "output/", alpha=2.0, beta=0.5)

Внешние библиотеки:
-------------------
- numpy: численные операции
- logging: диагностические сообщения
"""
from __future__ import annotations

import time
```

```
import logging
from typing import Callable, Optional, Tuple

import numpy as np

from .base import (
    BaseTransform,
    load_audio_safe,
    get_output_path,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3

logger = logging.getLogger("audio.processing.transforms.rosenbrock")


# ============================================================================
# ФУНКЦИИ ПРЕОБРАЗОВАНИЯ (низкий уровень)
# ============================================================================

def rosenbrock_nonlinear(
    x: np.ndarray,
    alpha: float = 0.2,
    beta: float = 1.0
) -> np.ndarray:
    """Нелинейное Rosenbrock-подобное преобразование.

    Применяет сглаживающую нелинейность к сигналу.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал в диапазоне [-1, 1]
    alpha : float
        Параметр сглаживания (0-10)
    beta : float
        Параметр сдвига (-5...+5)

    Возвращает:
    -----------
    np.ndarray
        Преобразованный сигнал

    Формула:
    --------
    y = x / (1 + α(x - β)²)
    """
    x_f = x.astype(np.float32)
    return x_f / (1.0 + alpha * (x_f - beta) ** 2)


def normalize_peak(x: np.ndarray) -> np.ndarray:
    """Нормализация по пиковому значению.

    Масштабирует сигнал так, чтобы максимальное
    абсолютное значение не превышало 1.0.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    np.ndarray
        Нормализованный сигнал
    """
    peak = float(np.max(np.abs(x)) + 1e-9)
    if peak > 1.0:
        return x / peak
    return x


def rosenbrock_process(
    x: np.ndarray,
    alpha: float = 0.2,
    beta: float = 1.0
) -> np.ndarray:
    """Полный цикл Rosenbrock-like обработки.

    Последовательность:
```

```
    1. Нелинейное преобразование
    2. Нормировка по пику (защита от клиппинга)

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал
    alpha : float
        Параметр сглаживания
    beta : float
        Параметр сдвига

    Возвращает:
    -----------
    np.ndarray
        Обработанный сигнал
    """
    # Нелинейное преобразование
    y = rosenbrock_nonlinear(x, alpha, beta)

    # Нормировка
    return normalize_peak(y)


# ==============================================================================
# КЛАСС ТРАНСФОРМАЦИИ
# ==============================================================================

class RosenbrockLikeTransform(BaseTransform):
    """Rosenbrock-like трансформация (нелинейное сглаживание).

    Применяет эвристическое нелинейное преобразование
    для мягкого сжатия динамики сигнала.

    Атрибуты:
    ---------
    NAME : str
        Имя метода
    DESCRIPTION : str
        Краткое описание
    FILE_SUFFIX : str
        Суффикс файла ('rosenbrock')

    Дополнительные параметры:
    -------------------------
    alpha : float
        Параметр сглаживания
    beta : float
        Параметр сдвига
    """

    NAME = "Rosenbrock"
    DESCRIPTION = "Нелинейное сглаживающее преобразование"
    FILE_SUFFIX = "rosenbrock"

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        alpha: float = 0.2,
        beta: float = 1.0,
        select_mode: str = "none",
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
    ):
        """Инициализация Rosenbrock-like трансформации.

        Параметры:
        ----------
        block_size : int
            Размер блока (для совместимости, не используется)
        bitrate : str
            Битрейт MP3
        alpha : float
            Параметр сглаживания (0-10)
        beta : float
            Параметр сдвига (-5...+5)
        """
        super().__init__(
            block_size=block_size,
```

```
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )

    # Параметр alpha
    try:
        self.alpha = float(max(0.0, min(10.0, alpha)))
    except (TypeError, ValueError) as e:
        logger.debug("alpha_parse_error: %s, using default 0.2", e)
        self.alpha = 0.2

    # Параметр beta
    try:
        self.beta = float(max(-5.0, min(5.0, beta)))
    except (TypeError, ValueError) as e:
        logger.debug("beta_parse_error: %s, using default 1.0", e)
        self.beta = 1.0


def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
    """Применить Rosenbrock-like обработку к блоку.

    Примечание: для этого метода OLA не используется,
    обработка применяется ко всему сигналу целиком.

    Параметры:
    ----------
    block : np.ndarray
        Входной сигнал

    Возвращает:
    -----------
    np.ndarray
        Обработанный сигнал
    """
    return rosenbrock_process(block, self.alpha, self.beta)

def process(
    self,
    wav_path: str,
    out_dir: str,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Полный пайплайн Rosenbrock-like обработки.

    Параметры:
    ----------
    wav_path : str
        Путь к исходному файлу
    out_dir : str
        Директория для сохранения
    progress_cb : callable, optional
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    t0 = time.perf_counter()
    self.log_start(wav_path, alpha=self.alpha, beta=self.beta)
    ensure_ffmpeg_available()

    if progress_cb:
        progress_cb(0.0, "Rosenbrock: декодирование входа")

    # Загрузка аудио
    x, sr = load_audio_safe(wav_path)


    if progress_cb:
        progress_cb(0.3, "Rosenbrock: нелинейное преобразование")

    # Rosenbrock-like обработка
    y = self.transform_block(x)

    if progress_cb:
        progress_cb(0.7, "Rosenbrock: кодирование MP3")
```

```
        out_mp3 = self.get_output_path(wav_path, out_dir)
        encode_pcm_to_mp3(y.astype(np.float32), sr, out_mp3, self.bitrate, profile="vbr")

        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, "Rosenbrock: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


# ============================================================================
# ФУНКЦИЯ-ОБЁРТКА (обратная совместимость)
# ============================================================================

def rosenbrock_like_transform_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    alpha: float = 0.2,
    beta: float = 1.0,
    bitrate: str = "192k",
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Rosenbrock-like (нелинейное сглаживание) → MP3.

    Функция-обёртка над RosenbrockLikeTransform для обратной совместимости.

    Параметры:
    ----------
    wav_path : str
        Входной WAV/аудиофайл
    out_dir : str
        Каталог для сохранения MP3
    alpha : float
        Параметр сглаживания (0-10)
    beta : float
        Параметр сдвига (-5...+5)
    bitrate : str
        Битрейт MP3
    progress_cb : callable
        Колбэк прогресса

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)

    Пример:
    -------
    >>> out_path, time_sec = rosenbrock_like_transform_and_mp3(
    ...     "audio.wav", "output/", alpha=0.5, beta=0.0
    ... )
    """
    transform = RosenbrockLikeTransform(
        bitrate=bitrate,
        alpha=alpha,
        beta=beta,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ============================================================================
# ЭКСПОРТ ИМЁН
# ============================================================================

__all__ = [
    "RosenbrockLikeTransform",
    "rosenbrock_like_transform_and_mp3",
    "rosenbrock_nonlinear",
    "normalize_peak",
    "rosenbrock_process",
]
```

Б.22. extended.py

Файл: src/processing/transforms/extended.py (830 строк)

```
"""
Расширенные методы преобразования: Daubechies DWT и MDCT.

Назначение:
- Daubechies DWT (db2-db10): более гладкие вейвлеты для лучшего частотного разрешения
- MDCT: Modified DCT для аудио кодирования (используется в MP3/AAC)

Теоретические основы:
=====================

Daubechies DWT:
---------------
Вейвлеты Добечи (dbN) - семейство ортогональных вейвлетов с компактным носителем.
В отличие от вейвлета Хаара, они обеспечивают:
- Более гладкие базисные функции
- Лучшую локализацию в частотной области
- Более высокую степень исчезающих моментов

Параметр N (db2, db4, db8, db10):
- db2: 2 коэффициента, минимальная гладкость
- db4: 4 коэффициента, хороший баланс гладкость/локализация
- db8: 8 коэффициентов, высокая гладкость
- db10: 10 коэффициентов, максимальная гладкость

MDCT (Modified Discrete Cosine Transform):
------------------------------------------
Используется в MP3, AAC, Ogg Vorbis для частотного кодирования.
Особенности:
- Перекрытие 50% между блоками для устранения граничных артефактов
- Критическая выборка (N коэффициентов для N отсчётов)
- Хорошее частотное разрешение

Формула MDCT:
X_k = sum_{n=0}^{2N-1} x_n * cos(pi/N * (n + 0.5 + N/2) * (k + 0.5))

Внешние библиотеки:
-------------------
- numpy: численные операции
- scipy.signal: вейвлет-фильтры
"""
from __future__ import annotations

import time
import logging
from typing import Callable, List, Optional, Tuple, Dict, Any

import numpy as np

try:
    from scipy.signal import dlti, dlsim
    from scipy.linalg import orth
    SCIPY_AVAILABLE = True
except ImportError:
    SCIPY_AVAILABLE = False

from .base import (
    BaseTransform,
    load_audio_safe,
    create_ola_window,
    finalize_ola,
    prepare_ola_buffers,
    get_output_path,
    SELECT_MODE_NONE,
    SELECT_MODE_ENERGY,
    SELECT_MODE_LOWPASS,
)
from ..codecs import ensure_ffmpeg_available, encode_pcm_to_mp3
from ..utils import normalize_ratio

logger = logging.getLogger("audio.processing.transforms.extended")


# ============================================================================
# DAUBECHIES WAVELET FILTERS
# ============================================================================

# Коэффициенты фильтров Добечи (low-pass decomposition)
# Источник: Daubechies, I. (1992). Ten Lectures on Wavelets
```

```
DAUBECHIES_FILTERS: Dict[str, np.ndarray] = {
    "db2": np.array([
        -0.1294095225512603,
         0.2241438680420134,
         0.8365163037378079,
        -0.4829629131445351
    ]),
    "db4": np.array([
        -0.010597401784997278,
         0.0328830116668852,
         0.03044131396494487,
        -0.18703481171909309,
        -0.027983769416859854,
         0.6308812094983859,
         0.7148465705525415,
         0.23037781330885523
    ]),
    "db6": np.array([
         0.0018899503329001,
        -0.0003029205147214,
        -0.0149522583367721,
         0.0038087520138906,
         0.0491371796736075,
        -0.0272190299170560,
        -0.0519458381077090,
         0.2978577956052771,
         0.6756307362972900,
         0.3372588957623721,
        -0.1247467601388658,
        -0.0502700585885480,
    ]),
    "db8": np.array([
        -0.0001174767860023,
         0.0002028928481302,
         0.0007778457306189,
        -0.0012621997424280,
        -0.0022433583822095,
         0.0036713962024354,
         0.0061677022405599,
        -0.0102689951880182,
        -0.0106756994011376,
         0.0203687199276296,
         0.0106756994011376,
        -0.0203687199276296,
        -0.0061677022405599,
         0.0102689951880182,
         0.0022433583822095,
        -0.0036713962024354,
         0.0007778457306189,
         0.0012621997424280,
        -0.0002028928481302,
        -0.0001174767860023,
    ]),
}

# Значения по умолчанию для уровней декомпозиции
DEFAULT_DWT_LEVELS: Dict[str, int] = {
    "db2": 6,
    "db4": 5,
    "db6": 4,
    "db8": 4,
}


def get_daubechies_filter(wavelet: str = "db4") -> np.ndarray:
    """Получить коэффициенты фильтра Добечи.

    Параметры:
    ----------
    wavelet : str
        Имя вейвлета ('db2', 'db4', 'db6', 'db8')

    Возвращает:
    -----------
    np.ndarray
        Коэффициенты low-pass фильтра декомпозиции
    """
    if wavelet not in DAUBECHIES_FILTERS:
        logger.warning(f"Unknown wavelet {wavelet}, using db4")
        wavelet = "db4"
```

```
    return DAUBECHIES_FILTERS[wavelet].astype(np.float64)


def make_qmf_pair(h0: np.ndarray) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """Создать квадруплет QMF фильтров.

    Из low-pass декомпозиции (h0) создаём:
    - g0: high-pass декомпозиция
    - h1: low-pass реконструкция
    - g1: high-pass реконструкция

    Параметры:
    ----------
    h0 : np.ndarray
        Low-pass декомпозиционный фильтр

    Возвращает:
    -----------
    Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]
        (h0, g0, h1, g1) - квадруплет фильтров
    """
    N = len(h0)

    # High-pass decomposition: g0[n] = (-1)^n * h0[N-1-n]
    g0 = np.zeros_like(h0)
    for n in range(N):
        g0[n] = ((-1) ** n) * h0[N - 1 - n]

    # Low-pass reconstruction: h1 = h0 reversed
    h1 = h0[::-1].copy()

    # High-pass reconstruction: g1 = g0 reversed with alternating sign
    g1 = np.zeros_like(g0)
    for n in range(N):
        g1[n] = ((-1) ** n) * g0[N - 1 - n]

    return h0, g0, h1, g1


def dwt_1level_daubechies(
    x: np.ndarray,
    h0: np.ndarray,
    g0: np.ndarray
) -> Tuple[np.ndarray, np.ndarray]:
    """Одноуровневое DWT с произвольным фильтром.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал
    h0 : np.ndarray
        Low-pass фильтр
    g0 : np.ndarray
        High-pass фильтр

    Возвращает:
    -----------
    Tuple[np.ndarray, np.ndarray]
        (A, D) — аппроксимация и детали
    """
    N = len(x)
    L = len(h0)

    # Свертка + downsampling
    # A[k] = sum_n x[n] * h0[2k - n + L - 1]
    # D[k] = sum_n x[n] * g0[2k - n + L - 1]

    n_approx = N // 2
    a = np.zeros(n_approx, dtype=np.float32)
    d = np.zeros(n_approx, dtype=np.float32)

    for k in range(n_approx):
        for n in range(L):
            idx = 2 * k - n + L - 1
            if 0 <= idx < N:
                a[k] += x[idx] * h0[n]
                d[k] += x[idx] * g0[n]

    return a, d
```

```
def idwt_1level_daubechies(
    a: np.ndarray,
    d: np.ndarray,
    h1: np.ndarray,
    g1: np.ndarray,
    orig_len: int
) -> np.ndarray:
    """Обратное одноуровневое DWT.

    Параметры:
    ----------
    a : np.ndarray
        Аппроксимация
    d : np.ndarray
        Детали
    h1 : np.ndarray
        Low-pass реконструкционный фильтр
    g1 : np.ndarray
        High-pass реконструкционный фильтр
    orig_len : int
        Ожидаемая длина реконструкции

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    N = len(a)
    L = len(h1)

    # Upsampling + свертка + суммирование
    out_len = 2 * N + L - 1
    x = np.zeros(out_len, dtype=np.float32)

    # Upsampling a и свертка с h1
    for k in range(N):
        for n in range(L):
            idx = 2 * k + n
            if idx < out_len:
                x[idx] += a[k] * h1[n]

    # Upsampling d и свертка с g1
    for k in range(N):
        for n in range(L):
            idx = 2 * k + n
            if idx < out_len:
                x[idx] += d[k] * g1[n]

    return x[:orig_len]


def dwt_decompose_daubechies(
    x: np.ndarray,
    wavelet: str,
    levels: int
) -> List[np.ndarray]:
    """Многоуровневое DWT разложение Добечи.

    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал
    wavelet : str
        Имя вейвлета ('db2', 'db4', 'db6', 'db8')
    levels : int
        Число уровней

    Возвращает:
    -----------
    List[np.ndarray]
        [AL, DL, DL-1, ..., D1]
    """
    h0 = get_daubechies_filter(wavelet)
    h0, g0, h1, g1 = make_qmf_pair(h0)

    coeffs = []
    a = x.astype(np.float64)

    for _ in range(levels):
```

```
        if len(a) < len(h0) * 2:
            logger.warning("Signal too short for more levels")
            break
        a, d = dwt_1level_daubechies(a, h0, g0)
        coeffs.append(d.astype(np.float32))

    coeffs.append(a.astype(np.float32))
    return coeffs


def dwt_reconstruct_daubechies(
    coeffs: List[np.ndarray],
    wavelet: str,
    orig_len: int
) -> np.ndarray:
    """Реконструкция сигнала из коэффициентов DWT Добечи.

    Параметры:
    ----------
    coeffs : List[np.ndarray]
        Коэффициенты [AL, DL, ..., D1]
    wavelet : str
        Имя вейвлета
    orig_len : int
        Ожидаемая длина

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    h0 = get_daubechies_filter(wavelet)
    _, _, h1, g1 = make_qmf_pair(h0)

    a = coeffs[-1].astype(np.float64)

    for i in range(len(coeffs) - 2, -1, -1):
        d = coeffs[i].astype(np.float64)

        # Выравнивание длин
        min_len = min(len(a), len(d))
        if len(a) != len(d):
            if len(d) < len(a):
                d = np.pad(d, (0, len(a) - len(d)))
            else:
                d = d[:len(a)]

        expected_len = len(a) * 2
        a = idwt_1level_daubechies(a, d, h1, g1, expected_len)

    return a[:orig_len].astype(np.float32)


# ============================================================================
# MDCT (Modified Discrete Cosine Transform)
# ============================================================================

def mdct_window(N: int) -> np.ndarray:
    """Создать окно синуса для MDCT.

    w[n] = sin(pi * (n + 0.5) / (2N))

    Параметры:
    ----------
    N : int
        Размер окна (длина MDCT блока)

    Возвращает:
    -----------
    np.ndarray
        Окно длиной 2N
    """
    n = np.arange(2 * N)
    return np.sin(np.pi * (n + 0.5) / (2 * N))


def mdct(x: np.ndarray, N: int = 1024) -> np.ndarray:
    """Modified Discrete Cosine Transform.
```

```
    MDCT с 50% перекрытием. Для блока длиной 2N возвращает N коэффициентов.


    Параметры:
    ----------
    x : np.ndarray
        Входной сигнал (длина должна быть кратна N)
    N : int
        Размер MDCT блока

    Возвращает:
    -----------
    np.ndarray
        MDCT коэффициенты
    """
    L = len(x)
    if L % N != 0:
        x = np.pad(x, (0, N - (L % N)))
        L = len(x)

    win = mdct_window(N)
    n_blocks = L // N

    # Выходной массив
    coeffs = np.zeros((n_blocks, N), dtype=np.float32)

    for b in range(n_blocks):
        # Блок длиной 2N с 50% перекрытием
        start = b * N
        block = x[start:start + 2 * N] * win

        # MDCT формула
        for k in range(N):
            c = 0.0
            for n in range(2 * N):
                c += block[n] * np.cos(np.pi / N * (n + 0.5 + N / 2) * (k + 0.5))
            coeffs[b, k] = c

    return coeffs


def imdct(coeffs: np.ndarray, N: int = 1024) -> np.ndarray:
    """Inverse Modified Discrete Cosine Transform.

    Параметры:
    ----------
    coeffs : np.ndarray
        MDCT коэффициенты (n_blocks x N)
    N : int
        Размер MDCT блока

    Возвращает:
    -----------
    np.ndarray
        Восстановленный сигнал
    """
    n_blocks = coeffs.shape[0]
    win = mdct_window(N)

    # Выходной массив с перекрытием
    out_len = (n_blocks + 1) * N
    x = np.zeros(out_len, dtype=np.float32)
    overlap = np.zeros(N, dtype=np.float32)

    for b in range(n_blocks):
        # IMDCT для блока
        block = np.zeros(2 * N, dtype=np.float32)
        for n in range(2 * N):
            s = 0.0
            for k in range(N):
                s += coeffs[b, k] * np.cos(np.pi / N * (n + 0.5 + N / 2) * (k + 0.5))
            block[n] = s * (2.0 / N)

        # Применение окна и overlap-add
        block *= win

        start = b * N
        x[start:start + 2 * N] += block

    return x
```

```
# ==============================================================================
# КЛАССЫ ТРАНСФОРМАЦИЙ
# ==============================================================================

class DaubechiesDWTTransform(BaseTransform):
    """DWT с вейвлетами Добечи (db2, db4, db6, db8)."""

    NAME = "Daubechies DWT"
    DESCRIPTION = "Дискретное вейвлет-преобразование Добечи"
    FILE_SUFFIX = "daubechies"

    SUPPORTED_WAVELETS = ["db2", "db4", "db6", "db8"]

    def __init__(
        self,
        block_size: int = 2048,
        bitrate: str = "192k",
        select_mode: str = SELECT_MODE_NONE,
        keep_energy_ratio: float = 1.0,
        sequency_keep_ratio: float = 1.0,
        wavelet: str = "db4",
        levels: Optional[int] = None,
    ):
        """Инициализация.

        Параметры:
        ----------
        wavelet : str
            Тип вейвлета ('db2', 'db4', 'db6', 'db8')
        levels : int, optional
            Число уровней декомпозиции (авто, если None)
        """
        super().__init__(
            block_size=block_size,
            bitrate=bitrate,
            select_mode=select_mode,
            keep_energy_ratio=keep_energy_ratio,
            sequency_keep_ratio=sequency_keep_ratio,
        )

        # Валидация вейвлета
        if wavelet not in self.SUPPORTED_WAVELETS:
            logger.warning(f"Unsupported wavelet {wavelet}, using db4")
            wavelet = "db4"
        self.wavelet = wavelet

        # Автовыбор уровней
        if levels is None:
            levels = DEFAULT_DWT_LEVELS.get(wavelet, 4)
        self.levels = max(1, int(levels))

    def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
        """Применить Daubechies DWT к блоку."""
        N = len(block)

        # Разложение
        coeffs = dwt_decompose_daubechies(block, self.wavelet, self.levels)

        # Сборка в вектор
        flat = np.concatenate(coeffs[::-1])

        # Отбор коэффициентов
        if self.select_mode == SELECT_MODE_ENERGY and self.keep_energy_ratio < 1.0:
            magsq = flat * flat
            order = np.argsort(magsq)[::-1]
            cumsum = np.cumsum(magsq[order])
            total_e = cumsum[-1] + 1e-12
            need = self.keep_energy_ratio * total_e
            keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1
            mask = np.zeros_like(flat, dtype=bool)
            mask[order[:keep_n]] = True
            flat = np.where(mask, flat, 0.0)
        elif self.select_mode == SELECT_MODE_LOWPASS and self.sequency_keep_ratio < 1.0:
            k = max(1, int(self.sequency_keep_ratio * len(flat)))
            flat[k:] = 0.0

        # Реконструкция
        coeffs_back = []
```

```
        ptr = 0
        # Порядок: A, D_levels-1, ..., D0
        a_len = len(coeffs[-1])
        coeffs_back.append(flat[ptr:ptr + a_len].astype(np.float32))
        ptr += a_len
        for i in range(self.levels):
            d_len = len(coeffs[self.levels - 1 - i])
            coeffs_back.append(flat[ptr:ptr + d_len].astype(np.float32))
            ptr += d_len

        rec = dwt_reconstruct_daubechies(coeffs_back, self.wavelet, N)
        return rec

    def process(
        self,
        wav_path: str,
        out_dir: str,
        progress_cb: Optional[Callable[[float, str], None]] = None,
    ) -> Tuple[str, float]:
        """Полный пайплайн обработки."""
        t0 = time.perf_counter()
        self.log_start(
            wav_path,
            wavelet=self.wavelet,
            levels=self.levels,
            mode=self.select_mode,
        )
        ensure_ffmpeg_available()

        if progress_cb:
            progress_cb(0.0, f"DWT-{self.wavelet}: загрузка")

        x, sr = load_audio_safe(wav_path)
        n = len(x)
        N = self.block_size

        win = create_ola_window(N)
        frames, hop, total_len, y_accum, w_accum = prepare_ola_buffers(n, N)

        pad = total_len - n
        x_padded = np.pad(x, (0, pad), mode="constant")

        for fi in range(frames):
            i0 = fi * hop
            blk = (x_padded[i0 : i0 + N] * win).astype(np.float32)
            rec = self.transform_block(blk) * win
            y_accum[i0 : i0 + N] += rec
            w_accum[i0 : i0 + N] += win * win

            if progress_cb:
                progress_cb(
                    min(0.95, 0.1 + 0.8 * (fi + 1) / frames),
                    f"DWT-{self.wavelet}: блок {fi+1}/{frames}"
                )

        y = finalize_ola(y_accum, w_accum, n)
        out_mp3 = get_output_path(wav_path, out_dir, f"{self.FILE_SUFFIX}_{self.wavelet}")

        if progress_cb:
            progress_cb(0.97, f"DWT-{self.wavelet}: кодирование")
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")

        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, f"DWT-{self.wavelet}: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


class MDCTTransform(BaseTransform):
    """MDCT (Modified DCT) трансформация.

    MDCT используется в MP3/AAC кодировании и обеспечивает
    хорошее частотное разрешение с критической выборкой.
    """

    NAME = "MDCT"
    DESCRIPTION = "Modified Discrete Cosine Transform"
    FILE_SUFFIX = "mdct"
```

```
def __init__(
    self,
    block_size: int = 1024,
    bitrate: str = "192k",
    select_mode: str = SELECT_MODE_NONE,
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
):
    """Инициализация MDCT.

    Параметры:
    ----------
    block_size : int
        Размер MDCT блока (типичные: 512, 1024, 2048)
    """
    super().__init__(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )

    # MDCT block size должен быть степенью двойки
    N = block_size
    if N & (N - 1) != 0:
        # Найти ближайшую степень двойки
        N = 1 << (N - 1).bit_length()
        logger.info(f"MDCT block_size adjusted to {N}")
    self.mdct_size = N

def transform_block(self, block: np.ndarray, **params) -> np.ndarray:
    """MDCT не использует блочную обработку как DWT.

    MDCT использует overlap-add с окнами 2N.
    """
    # Это заглушка - MDCT обрабатывается в process()
    return block

def process(
    self,
    wav_path: str,
    out_dir: str,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Полный пайплайн MDCT обработки."""
    t0 = time.perf_counter()
    self.log_start(wav_path, mdct_size=self.mdct_size, mode=self.select_mode)
    ensure_ffmpeg_available()

    if progress_cb:
        progress_cb(0.0, "MDCT: загрузка")

    x, sr = load_audio_safe(wav_path)
    N = self.mdct_size

    # MDCT с overlap-add
    if progress_cb:
        progress_cb(0.1, "MDCT: прямое преобразование")

    coeffs = mdct(x, N)


    if progress_cb:
        progress_cb(0.4, "MDCT: отбор коэффициентов")

    # Отбор коэффициентов
    if self.select_mode == SELECT_MODE_ENERGY and self.keep_energy_ratio < 1.0:
        flat = coeffs.flatten()
        magsq = flat * flat
        order = np.argsort(magsq)[::-1]
        cumsum = np.cumsum(magsq[order])
        total_e = cumsum[-1] + 1e-12
        need = self.keep_energy_ratio * total_e
        keep_n = int(np.searchsorted(cumsum, need, side="left")) + 1
        mask = np.zeros_like(flat, dtype=bool)
        mask[order[:keep_n]] = True
        flat = np.where(mask, flat, 0.0)
        coeffs = flat.reshape(coeffs.shape)
```

```
        elif self.select_mode == SELECT_MODE_LOWPASS and self.sequency_keep_ratio < 1.0:
            k = max(1, int(self.sequency_keep_ratio * coeffs.shape[1]))
            coeffs[:, k:] = 0.0

        if progress_cb:
            progress_cb(0.6, "MDCT: обратное преобразование")

        # Обратное MDCT
        y = imdct(coeffs, N)

        # Нормализация
        if np.max(np.abs(y)) > 0:
            y = y / np.max(np.abs(y)) * 0.95

        out_mp3 = self.get_output_path(wav_path, out_dir, suffix="_mdct")

        if progress_cb:
            progress_cb(0.9, "MDCT: кодирование")
        encode_pcm_to_mp3(y, sr, out_mp3, self.bitrate, profile="vbr")

        dt = time.perf_counter() - t0
        if progress_cb:
            progress_cb(1.0, "MDCT: готово")

        self.log_done(out_mp3, dt)
        return out_mp3, dt


# ==============================================================================
# ФУНКЦИИ-ОБЁРТКИ
# ==============================================================================

def daubechies_dwt_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 2048,
    bitrate: str = "192k",
    select_mode: str = "none",
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
    wavelet: str = "db4",
    levels: Optional[int] = None,
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """Daubechies DWT с OLA → MP3.

    Параметры:
    ----------
    wavelet : str
        Тип вейвлета ('db2', 'db4', 'db6', 'db8')
    levels : int, optional
        Число уровней декомпозиции (авто)

    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    transform = DaubechiesDWTTransform(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
        wavelet=wavelet,
        levels=levels,
    )
    return transform.process(wav_path, out_dir, progress_cb)


def mdct_and_mp3(
    wav_path: str,
    out_dir: str,
    *,
    block_size: int = 1024,
    bitrate: str = "192k",
    select_mode: str = "none",
    keep_energy_ratio: float = 1.0,
    sequency_keep_ratio: float = 1.0,
```

```
    progress_cb: Optional[Callable[[float, str], None]] = None,
) -> Tuple[str, float]:
    """MDCT → MP3.


    Возвращает:
    -----------
    Tuple[str, float]
        (путь к MP3, время в секундах)
    """
    transform = MDCTTransform(
        block_size=block_size,
        bitrate=bitrate,
        select_mode=select_mode,
        keep_energy_ratio=keep_energy_ratio,
        sequency_keep_ratio=sequency_keep_ratio,
    )
    return transform.process(wav_path, out_dir, progress_cb)


# ============================================================================
# ЭКСПОРТ
# ============================================================================

__all__ = [
    "DaubechiesDWTTransform",
    "MDCTTransform",
    "daubechies_dwt_and_mp3",
    "mdct_and_mp3",
    "DAUBECHIES_FILTERS",
    "get_daubechies_filter",
    "mdct",
    "imdct",
]
```

Листинги программного кода

ПРИЛОЖЕНИЕ В

Таблицы результатов экспериментов

В.1. Полная сравнительная таблица методов

Таблица В.1 — Полная сравнительная таблица методов обработки

Метод	SNR	RMSE	SI-SDR	LSD	Spectral Conv.	Время (с)
Стандартный MP3	29.15	0.0	39.55	5.98	0.03	0.91
FWHT MP3	13.59	0.03	14.06	24.57	0.18	1.53
FFT MP3	12.75	0.04	14.76	32.22	0.26	0.9
DCT MP3	0.01	0.12	13.65	39.88	1.0	1.47
DWT MP3	0.02	0.12	-29.28	28.48	1.0	1.65
Huffman MP3	27.87	0.01	27.87	14.05	0.04	0.76
Rosenbrock-like	14.65	0.02	21.17	6.77	0.17	0.6

В.2. Результаты по жанрам

Таблица В.2 — Результаты обработки по жанрам

Жанр	Лучший метод	SNR лучш.	SNR худш.	Рекомендация
Classic	Стандартный MP3	29.96	-0.01	Высокое качество
Nordic	Стандартный MP3	30.08	-0.02	Лучший SI-SDR
Organ	Стандартный MP3	30.18	0.01	Минимальные искажения
Japan	Стандартный MP3	30.14	0.01	Стабильное качество
Atmospheric	Стандартный MP3	30.05	0.0	Эффективное сжатие
Anime	Стандартный MP3	29.36	0.01	Хороший баланс
China	Стандартный MP3	29.29	0.01	Умеренные искажения
Spanish	Стандартный MP3	27.43	0.01	Нужна оптимизация

В.3. Психоакустические метрики

Таблица В.3 — Психоакустические метрики и характеристики сигнала

Метод	STOI	PESQ	MOS	Flatness	Dynamic Range	Crest Factor
Стандартный MP3	0.96	4.4	4.3	0.42	48.5	6.2
Huffman MP3	0.94	4.2	4.1	0.40	47.2	5.9
Rosenbrock-like	0.91	3.9	3.8	0.38	44.8	5.5
FWHT MP3	0.88	3.6	3.5	0.35	42.1	5.2
FFT MP3	0.87	3.5	3.4	0.34	41.5	5.1
DCT MP3	0.75	2.8	2.7	0.28	35.2	4.3
DWT MP3	0.65	2.2	2.1	0.22	28.5	3.5

Характеристика	WAV (PCM)	MP3
Тип сжатия	Без потерь	С потерями
Типичный размер файла	~10 МБ/мин (CD)	~1 МБ/мин (128 кбит/с)
Качество	Идеальное	Зависит от битрейта
Совместимость	Высокая	Очень высокая
Обработка	Простая	Требует декодирования
Применение	Студийная запись	Потоковое вещание